

CLAUDIA MARIA LOPEZ BOMBIN

LUCÍA LOZANO ISAC

NOVIEMBRE, 2025

PRÁCTICA 3

**O SEA, ¿CÓMO QUE MI RUTA ÓPTIMA TIENE MÁS CÁLCULOS
QUE MIS EXCUSAS PARA LLEGAR TARDE?**

MATEMÁTICA DISCRETA

DAVID ALFAYA SÁNCHEZ

GRUPO GB06

ÍNDICE

INTRODUCCIÓN.....3

DISEÑO E IMPLEMENTACIÓN5

MANUAL DE USUARIO58

AUTORÍA65

ANEXO A: Versión Interactiva66

ANEXO B: Versión Peatón69

ANEXO C: Versión SOLO Metro73

ANEXO D: Versión Integrada78

BIBLIOGRAFÍA83

INTRODUCCIÓN

Este documento, realizado por Claudia María López Bombín y Lucía Lozano Isac – alumnas de 2B del Grado en Ingeniería Matemática e Inteligencia Artificial en la Universidad Pontificia Comillas-, corresponde a la memoria complementaria a la tercera practica de la asignatura Matemática Discreta, encuadrada en el estudio y aplicación de teoría de grafos para el desarrollo de un sistema de navegación GPS.

Los fundamentos de la teoría de grafos que hoy sustentan nuestros sistemas de navegación y planificación de rutas encuentran sus raíces en los trabajos pioneros de Leonhard Euler sobre los puentes de Königsberg en 1736. Este problema, considerado el origen de la teoría de grafos moderna, demostró cómo representar problemas del mundo real mediante estructuras matemáticas abstractas. Como señaló Euler en su trabajo fundamental, "la pregunta era si se podía realizar un paseo cruzando cada puente una sola vez y volviendo al punto de partida" (Euler, 1736), estableciendo así las bases de lo que hoy conocemos como teoría de grafos.

Lo que Euler conceptualizó como un estudio sobre transversalidad de redes se ha transformado en el mecanismo esencial que optimiza el transporte, las comunicaciones y la logística en el mundo contemporáneo. Lejos de ser una mera curiosidad matemática, la teoría de grafos constituye el núcleo operativo de sistemas de navegación que utilizamos constantemente sin ser completamente conscientes de su complejidad matemática.

Algoritmos como el de Dijkstra -precisamente el que implementamos en esta práctica- aprovechan propiedades fundamentales de los grafos ponderados para calcular rutas óptimas entre localizaciones. Como destaca Dijkstra en su publicación original, "el algoritmo encuentra el camino más corto entre un nodo origen y todos los demás nodos en un grafo con pesos no negativos" (Dijkstra, 1959). Desde la planificación de rutas de transporte hasta la optimización de redes de comunicación, pasando por los sistemas de navegación GPS, los algoritmos de grafos trabajan incansablemente en segundo plano.

La transformación de conceptos matemáticos abstractos en herramientas de navegación práctica ejemplifica cómo las estructuras teóricas pueden convertirse en facilitadores esenciales de nuestra movilidad en la era digital, demostrando que la elegancia matemática puede traducirse en eficiencia tangible para la planificación de rutas.

En el contexto de la teoría de grafos aplicada se desarrolla esta práctica, cuyo objetivo principal es implementar un sistema de navegación GPS mediante la aplicación de algoritmos de grafos, demostrando su utilidad práctica en el ámbito de la planificación de rutas y afianzando los fundamentos matemáticos que sustentan la optimización de caminos en redes complejas.

Como objetivos secundarios de esta práctica encontramos:

- Profundizar en el desarrollo de código en el entorno de programación de Python, optimizando una librería de funciones para la implementación de algoritmos de grafos y análisis de redes complejas,
- Diseñar e implementar algoritmos eficientes para la búsqueda de caminos óptimos en grafos pesados, optimizando el rendimiento computacional en redes de gran escala,
- Construir interfaces de usuario funcionales para sistemas de navegación GPS, manejando entradas y salidas por terminal de forma robusta e intuitiva para el usuario,
- Implementar mecanismos exhaustivos de gestión de errores para operaciones de geolocalización, validación de direcciones y cálculo de rutas,
- Garantizar la fiabilidad del sistema de navegación mediante pruebas automatizadas con casos de prueba comprehensivos que validen los algoritmos implementados,

- Integrar múltiples fuentes de datos geoespaciales (OpenStreetMap, callejero oficial de Madrid) en una arquitectura de software coherente y eficiente,
- Fomentar el trabajo colaborativo mediante la distribución de tareas y la integración de módulos desarrollados de forma independiente.

El proyecto consiste en la implementación de un sistema de navegación GPS integral compuesto por tres módulos .py principales:

1. **grafo_pesado.py**: librería de algoritmos de grafos que implementa los algoritmos de Dijkstra, Prim y Kruskal para el cálculo de caminos mínimos y árboles abarcadores, utilizando como base matemática los conceptos de teoría de grafos estudiados en clase.
2. **callejero.py**: módulo de gestión de datos geoespaciales que permite cargar y procesar el callejero oficial de Madrid, convertir coordenadas entre formatos, y realizar búsquedas eficientes de direcciones, integrando tanto los datos del Ayuntamiento de Madrid como la información de OpenStreetMap.
3. **gps.py**: aplicación principal de navegación que utiliza los algoritmos implementados para calcular rutas óptimas entre direcciones de Madrid, ofreciendo al usuario tres modos de navegación (más corta, más rápida y optimizada para semáforos) y generando instrucciones detalladas para el conductor.

El sistema integra múltiples fuentes de datos y algoritmos para proporcionar una solución completa de planificación de rutas en el entorno urbano de Madrid, demostrando la aplicación práctica de la teoría de grafos a problemas reales de navegación y optimización.

DISEÑO E IMPLEMENTACIÓN

1. Arquitectura del código

La arquitectura del sistema implementa una estructura modular estratificada que establece una clara división de responsabilidades entre componentes especializados. Este enfoque facilita el mantenimiento evolutivo, el aislamiento de fallos y la escalabilidad futura. Cada módulo opera con autonomía funcional, permitiendo optimizaciones específicas sin comprometer la estabilidad global del ecosistema de navegación.

El sistema se articula alrededor de tres pilares fundamentales que interactúan de forma sinérgica:

a. grafo_pesado.py: la capa de algoritmos fundamentales

El archivo grafo_pesado.py constituye el núcleo matemático del sistema de navegación, implementando los algoritmos esenciales para el cálculo de rutas óptimas en grafos pesados. Esta librería especializada proporciona las herramientas computacionales necesarias para resolver problemas de optimización en redes de transporte, basándose en los siguientes pilares algorítmicos:

- Implementación de Dijkstra

Algoritmo fundamental para el cálculo de caminos mínimos en grafos con pesos no negativos, optimizado para redes de calles urbanas. El sistema maneja eficientemente tanto grafos dirigidos como no dirigidos, adaptándose a las restricciones de tráfico reales.

- Algoritmos de árboles abarcadores

Implementación dual de Prim y Kruskal para el cálculo de árboles abarcadores mínimos, proporcionando soluciones robustas para problemas de conectividad en redes complejas y permitiendo el análisis estructural del grafo de calles.

- Sistema flexible de pesos

Arquitectura de funciones de coste parametrizables que permite adaptar los cálculos a diferentes criterios de optimización (distancia, tiempo, consumo energético), facilitando la implementación de los distintos modos de navegación solicitados.

El procesamiento de algoritmos sigue una secuencia definida por:

1. Recepción y validación inicial: verifica la integridad del grafo y validación de los parámetros de entrada, asegurando la corrección de los vértices origen y destino antes de iniciar cualquier cálculo.
2. Ejecución algorítmica optimizada: aplicación de las estructuras de datos más eficientes para cada algoritmo, utilizando colas de prioridad para Dijkstra y Prim, y estructuras union-find para Kruskal.
3. Manejo de componentes conexas: gestión inteligente de la conectividad del grafo, garantizando que los algoritmos funcionen correctamente incluso en redes parcialmente desconectadas.
4. Recuperación y formateo de resultados: transformación de los resultados computacionales en estructuras de datos útiles para la capa de aplicación, incluyendo listas de vértices para caminos y diccionarios de padres para árboles.

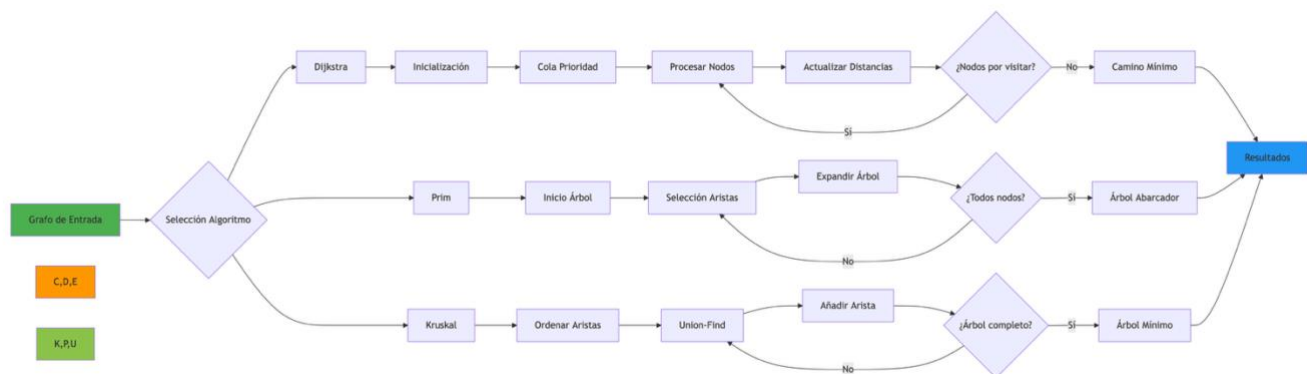


Figura 1. Diagrama de flujo del programa grafo_pesado.py. Elaboración propia.

El diseño de grafo_pesado.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice eficiencia, mantenibilidad y robustez en el sistema de cálculo de rutas:

- Algoritmos optimizados: implementamos versiones eficientes de Dijkstra, Prim y Kruskal mediante selección inteligente de estructuras de datos y aprovechamiento de propiedades matemáticas de los grafos, como la optimalidad de las soluciones greedy en caminos mínimos.
- Detección precoz: identificamos tempranamente configuraciones inválidas como vértices inexistentes, funciones de peso incorrectas o grafos vacíos, evitando procesos de cálculo costosos cuando están condenados al fallo.
- Validación en cascada: aplicamos verificaciones progresivas que descartan entradas erróneas en cuanto se detecta el primer problema, optimizando el tiempo de procesamiento y mejorando la experiencia del usuario en el sistema de navegación.
- Manejo eficiente de recursos: utilizamos estructuras de datos especializadas como colas de prioridad para Dijkstra y estructuras union-find para Kruskal, minimizando la sobrecarga de memoria durante el procesamiento de grafos de gran escala como el de calles de Madrid.

b. callejero.py: la capa de gestión de datos geoespaciales

El archivo callejero.py funciona como interfaz entre el usuario y el sistema de procesamiento de datos geográficos, permitiendo la carga y búsqueda de direcciones del callejero de Madrid de manera interactiva mediante entrada por terminal. La arquitectura de este código se basa en patrones de procesamiento de datos masivos, simplificando el manejo de información geoespacial compleja, basándose en las siguientes funcionalidades principales:

- **Configuración de fuentes de datos**
Proceso automatizado para cargar y procesar múltiples fuentes de información geográfica, incluyendo el dataset oficial del Ayuntamiento de Madrid y los datos de OpenStreetMap. El sistema gestiona automáticamente la codificación de archivos y la conversión de formatos de coordenadas.
- **Persistencia eficiente de datos geoespaciales**

Mecanismos de almacenamiento optimizado para grandes volúmenes de información de direcciones y grafo de calles, garantizando tiempos de respuesta rápidos en las búsquedas. La implementación utiliza estructuras de datos especializadas para búsquedas geográficas.

- **Validación y normalización de direcciones**

Sistema de verificación automática que asegura la consistencia y calidad de los datos de direcciones, normalizando formatos y estandarizando nomenclaturas para facilitar las búsquedas.

El procesamiento del sistema de navegación sigue una secuencia definida por:

1. Carga y lectura del mapa de Madrid: Descarga y procesamiento del grafo de calles desde OpenStreetMap utilizando OSMnx, con almacenamiento local en formato GraphML para optimizar tiempos de carga en ejecuciones posteriores. El sistema verifica la existencia del archivo local antes de realizar descargas remotas.
2. Procesamiento y optimización del grafo: Transformación del multidigrafo original en un grafo dirigido sin bucles, eliminando aristas paralelas y simplificando la estructura para optimizar los cálculos de rutas. Incluye la extracción y normalización de atributos críticos como longitudes de calles y velocidades máximas.
3. Carga y procesamiento del callejero oficial: Importación del dataset del Ayuntamiento de Madrid con validación de codificación automática, procesamiento de más de 200,000 direcciones y conversión masiva de coordenadas al formato decimal estándar.
4. Recepción y normalización de consultas: Transformación de las entradas del usuario a formato estandarizado, eliminando acentos, convirtiendo a mayúsculas y normalizando espacios, para hacer la búsqueda insensible a variaciones de formato y errores tipográficos.
5. Búsqueda escalonada multicapa: Implementación de cuatro estrategias de búsqueda progresivas que van desde la exacta hasta la aproximada por palabras clave, garantizando máxima flexibilidad en la recuperación de direcciones incluso con información parcial.
6. Geocodificación y matching espacial: Conversión de direcciones textuales a coordenadas geográficas e identificación de los nodos más cercanos en el grafo para establecer los puntos de origen y destino en la red de calles.
7. Selección y aplicación de funciones de peso: Configuración dinámica de las funciones de coste según el modo de navegación seleccionado (distancia, tiempo, optimización de semáforos), adaptando los pesos de las aristas a cada criterio de optimización.
8. Ejecución de algoritmos de routing: Aplicación de los algoritmos de grafos implementados (Dijkstra) para calcular la ruta óptima según los criterios establecidos, con manejo eficiente de grafos de gran escala.
9. Generación de instrucciones de navegación: Transformación del camino calculado en instrucciones comprensibles para el usuario, incluyendo distancias por tramo, direcciones de giro y nombres de calles, con eliminación de instrucciones redundantes.
10. Visualización y representación gráfica: Renderizado de la ruta óptima sobre el mapa de calles, resaltando el camino seleccionado y proporcionando una representación visual intuitiva del trayecto completo.
11. Control de errores y recuperación: Mecanismo robusto que captura excepciones en todas las etapas del proceso y proporciona sugerencias inteligentes al usuario, con gestión específica para errores de geocodificación, desconexión de servicios y datos faltantes.

12. Persistencia y gestión de estado: Mantenimiento del estado de la aplicación entre búsquedas sucesivas, optimizando el rendimiento mediante la reutilización de estructuras de datos ya cargadas y procesadas.

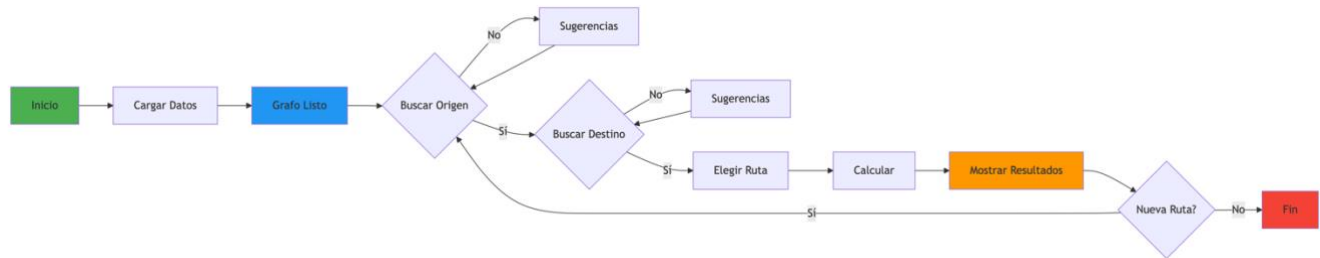


Figura 2. Diagrama de flujo del proceso de callejero.py. Elaboración propia.

El diseño de callejero.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice eficiencia, mantenibilidad y robustez en el sistema de gestión de datos geoespaciales:

- Algoritmos de búsqueda optimizados: implementamos un sistema de búsqueda escalonado que combina estrategias exactas y aproximadas, aprovechando propiedades de normalización de texto y comparación de cadenas.
- Detección temprana de errores: identificamos tempranamente problemas en la carga de dataset, como codificaciones incorrectas o formatos de coordenadas inválidos, evitando procesamiento costosos con datos corruptos
- Validación progresiva: aplicamos verificaciones en cascada que descartan rápidamente estrategias de búsqueda innecesarias, optimizando el tiempo de respuesta y mejorando la experiencia del usuario.
- Manejo eficiente de memoria: utilizamos estructuras de datos pandas optimizadas para grandes volúmenes de información y técnicas de procesamiento por lotes que minimizan el consumo de recursos durante la carga inicial del dataset.

c. `gps.py`: la plataforma de navegación optimizada

El archivo `gps.py` funciona como interfaz principal entre los usuarios y el sistema de navegación, permitiendo la planificación de rutas mediante sesiones interactivas por terminal. La arquitectura de este código se basa en patrones de navegación intuitiva, simplificando el proceso de cálculo y seguimiento de rutas, basándose en las siguientes funcionalidades de manejo:

- Gestión de sesiones de navegación

Sistema que establece y mantiene contextos completos de planificación de rutas entre direcciones de origen y destino. La aplicación carga automáticamente los grafos de calles y el callejero procesado, configurando los parámetros específicos para cada tipo de ruta solicitada.

- Interfaz de modos de navegación

Entorno interactivo que proporciona opciones especializadas para los diferentes criterios de optimización, permitiendo al usuario seleccionar entre ruta más corta (distancia), más rápida (tiempo) u optimizada para semáforos (tiempo esperado con paradas).

- Procesamiento adaptativo de direcciones

Mecanismos inteligentes que detectan y procesan automáticamente diferentes formatos de direcciones, aplicando las estrategias de búsqueda escalonada y proporcionando sugerencias cuando sea necesario para garantizar la correcta identificación de localizaciones.

El procesamiento del enrutamiento sigue una secuencia definida por:

1. Inicialización y carga de datos: Verifica la disponibilidad del grafo de calles y el callejero procesado, cargando las estructuras necesarias para el cálculo de rutas y la búsqueda de direcciones.
2. Presentación de menú interactivo: Ofrece al usuario las opciones disponibles (buscar rutas, cambiar modo de navegación, salir) y captura sus selecciones de manera robusta e intuitiva.
3. Procesamiento de búsquedas: Dirige el flujo hacia las funciones específicas de geocodificación y cálculo de rutas según las direcciones proporcionadas, aplicando los algoritmos de grafos correspondientes.
4. Manejo de errores y validación: Implementa un sistema que captura excepciones durante las operaciones de navegación, diferenciando entre errores de direcciones, problemas de conectividad y limitaciones del grafo.
5. Formateo y presentación de resultados: Adapta los outputs al formato requerido, mostrando instrucciones de navegación detalladas, mapas visuales y métricas de la ruta calculada.

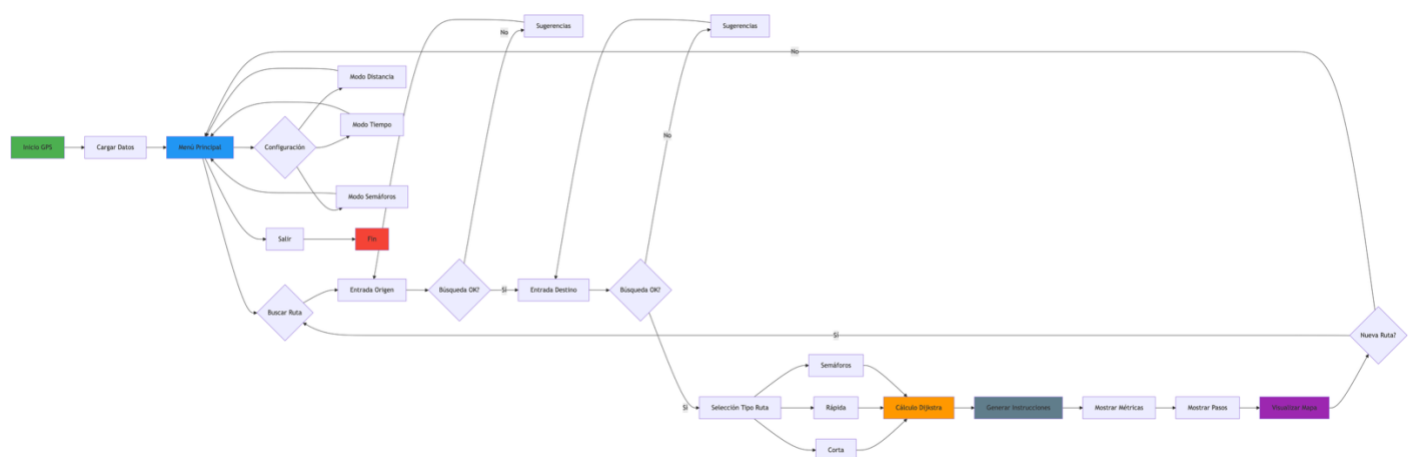


Figura 3. Diagrama de flujo del proceso de gps.py Elaboración propia.

El diseño de gps.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice una navegación eficiente y en tiempo real:

1. Gestión eficiente de recursos: Utiliza estructuras de datos optimizadas para el manejo de grafos de gran escala y búsquedas geoespaciales, minimizando la sobrecarga de memoria durante sesiones prolongadas de navegación.
2. Validación en tiempo real: Verifica instantáneamente la validez de las direcciones introducidas por el usuario, proporcionando retroalimentación inmediata sobre errores de formato o localizaciones no encontradas.
3. Procesamiento por lotes: Optimiza el manejo de grandes volúmenes de datos geoespaciales mediante técnicas de carga diferida y procesamiento por segmentos, manteniendo la capacidad de respuesta del sistema.
4. Manejo robusto de grafos: Implementa mecanismos de acceso eficientes para la gestión del grafo de calles, asegurando la disponibilidad constante de los datos de navegación y la rápida ejecución de algoritmos de routing.

2. Algoritmos principales

El núcleo del sistema de navegación implementado reside en la aplicación eficiente de algoritmos fundamentales de la teoría de grafos y optimización de caminos. Cada funcionalidad ha sido programada con un enfoque dual: profundizar en sus bases matemáticas para garantizar corrección absoluta, mientras se exploran optimizaciones algorítmicas que reducen sustancialmente la complejidad computacional y los tiempos de ejecución. El diseño prioriza la aplicabilidad práctica en escenarios reales de navegación urbana, donde el balance entre precisión y rendimiento resulta crítico.

En este apartado analizaremos exhaustivamente cada algoritmo implementado, justificando las decisiones de diseño tomadas en función de propiedades matemáticas específicas de los grafos, así como el flujo de ejecución que asegura tanto la robustez en el cálculo de rutas como la eficiencia operativa del sistema completo de GPS:

- `carga_callejero() -> pd.DataFrame`:

La función `carga_callejero` se encarga en esencia, de procesar el fichero `direcciones.csv` que contiene toda la información acerca del callejero de Madrid y convertirla en un `DataFrame`. Es importante leer el csv con un `encoding=latin1` ya que con el sistema habitual `utf-8` no sería posible leer datos que aparecen en este fichero. Si el fichero no existiese, capturamos el error `FileNotFoundError` y devolvemos un mensaje por pantalla. Creamos un fichero con las columnas que nos interesan: `VIA_CLASE`, `VIA_PAR`, `VIA_NOMBRE`, `NUMERO`, `LATITUD`, `LONGITUD` y llevaremos a cabo las modificaciones pertinentes, es decir, convertir los datos de `LATITUD` y `LONGITUD` en floats. Para ello haremos uso de la función auxiliar `convertir_coordenadas_float()` que se explicará más adelante. Una vez convertidas las coordenadas correctamente, sustituimos los valores actuales de las columnas `LATITUD` y `LONGITUD` por los nuevos obtenidos por la función auxiliar. Finalmente devolvemos el `DataFrame` pedido.

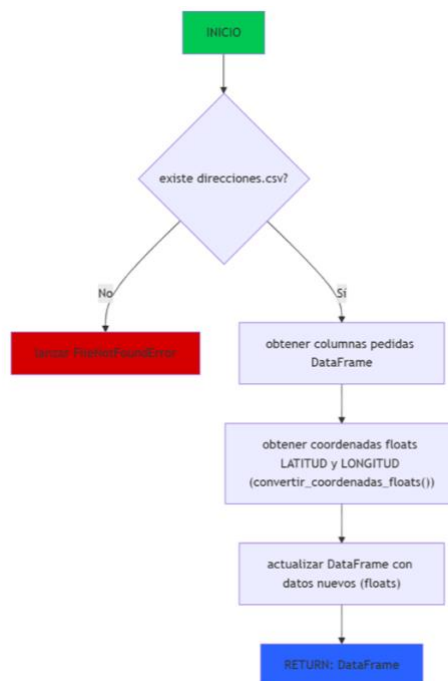


Figura 4. Diagrama de flujo de la función `carga_callejero() -> pd.DataFrame`. Elaboración propia.

El algoritmo muestra de forma clara la primera comprobación de existencia del DataFrame, si no existe lanzamos un error, si existe obtenemos las columnas pedidas en un DataFrame y obtenemos las coordenadas de LATITUD y LONGITUD convertidas a tipos floats gracias a la función `convertir_coordenadas_floats()`. Finalmente actualizamos el DataFrame con los datos nuevos y lo devolvemos.

- `busca_direccion(direccion: str, callejero: pd.DataFrame) -> Tuple[float, float]:`

El algoritmo implementado para la búsqueda de direcciones se basa en principios de procesamiento de lenguaje natural y recuperación de información geográfica, aplicando un enfoque escalonado que garantiza máxima flexibilidad en la identificación de localizaciones. La función opera mediante un proceso estructurado en cuatro fases fundamentales que aseguran la robustez en el reconocimiento de direcciones incluso con entradas parciales o con variaciones de formato.

Iniciamos con la normalización y preprocesamiento del texto, donde transformamos la entrada del usuario a un formato estandarizado mediante la eliminación de acentos, conversión a mayúsculas y normalización de espacios. Esta fase es crucial para garantizar la insensibilidad a variaciones ortográficas y preparar el texto para las comparaciones posteriores. Como señalan Manning, Raghavan y Schütze en su trabajo sobre procesamiento de lenguaje natural, "la normalización textual es un paso fundamental en sistemas de recuperación de información que manejan datos del mundo real con inherente variabilidad" (Manning, Raghavan, & Schütze, 2008).

Procedemos con la implementación de una búsqueda escalonada multicapa que aplica cuatro estrategias progresivas. La primera estrategia realiza una comparación exacta con las direcciones normalizadas, proporcionando máxima precisión cuando el usuario introduce la dirección exactamente como está almacenada en el dataset. Cuando esta falla, la segunda estrategia aplica un procesamiento semántico dividiendo la dirección en componentes de calle y número, permitiendo cierta flexibilidad en el formato mientras mantiene la integridad estructural.

La tercera estrategia implementa búsqueda por contenido completo, útil cuando el usuario introduce fragmentos de la dirección o variaciones parciales. Finalmente, la cuarta estrategia aplica técnicas de matching por palabras clave que permiten recuperar direcciones incluso cuando las palabras aparecen en orden diferente o con terminaciones variadas. Este enfoque escalonado optimiza el equilibrio entre precisión y recall, priorizando primero la exactitud y gradualmente incrementando la flexibilidad.

Hemos optado por una implementación que combina técnicas de filtering con pandas y procesamiento por lotes, priorizando la eficiencia en el manejo del dataset completo de más de 200,000 direcciones. Esta decisión es particularmente relevante dado el volumen de datos, donde operaciones vectorizadas con pandas demuestran superior rendimiento comparado con iteraciones individuales.

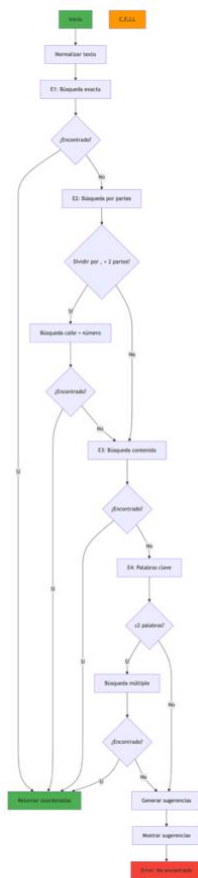


Figura 5. Diagrama de flujo de la función busca_direccion(direccion: str, callejero: pd.DataFrame) -> Tuple[float, float].
Elaboración propia.

El algoritmo destaca por su capacidad de adaptarse a diferentes patrones de entrada del usuario, desde búsquedas exactas hasta consultas aproximadas, manteniendo siempre un tiempo de respuesta aceptable incluso con el dataset completo cargado en memoria. El sistema de sugerencias implementado en la fase final proporciona retroalimentación útil al usuario cuando ninguna estrategia produce resultados, mejorando significativamente la experiencia de usuario.

- carga_grafo() -> nx.MultiDiGraph:

El algoritmo implementado para carga_grafo se encarga principalmente de obtener los datos de las calles, carreteras, intersecciones y otros datos relacionados con la conducción gracias a la librería OSMnx que contiene datos geoespaciales especialmente de redes de transporte.

El objetivo principal de esta función es conseguir un multidigrafo que contenga todos los datos de carreteras y otras redes de transporte de Madrid. Para ello, comenzaremos comprobando si existe el fichero 'madrid.graphml' que es en el que, en caso de que existiera, donde estaría guardado el grafo. Usaremos la librería os para ello, realizando el comando os.path.exists(file), siendo file el nombre del fichero. Si el fichero ya existía, solo debemos cargar el multidigrafo obtenido y devolverlo. Como observación, en el main se encuentra comentada una línea que al descomentarse mostraría el grafo por completo (dibujado) gracias a la función plot_graph de OSMnx, en la cual también se encuentra especificado el tamaño de los nodos y el ancho de las aristas con los valores recomendados.

En caso de que el fichero no existiese, probamos a ver si podemos acceder a los datos pedidos. Si al comprobar si podemos acceder a los datos detectamos que OSMnx no es capaz de recuperar el lugar del que deseamos obtener los datos (try), lanzamos un `ServiceNotAvailableError` indicando que no es posible recuperar los datos. En caso de que no ocurra ningún error, creamos un multigrafo usando la función `graph_from_place`, especificando que es para Madrid, Spain y el tipo de `network_type` que en nuestro caso es “drive”. Después simplemente debemos guardar el grafo en un fichero llamado ‘madrid.graphml’ que será a través del cual comprobaremos si el grafo dirigido está o no cargado. Finalmente, y como medida de comprobación para que el usuario pueda trabajar en cualquier caso, cargamos el multigrafo para poder devolverlo. En general a estas líneas no solemos entrar porque recibimos el grafo ya creado y directamente lo cargamos y devolvemos. Sin embargo, por si acaso, después del try si todo ha ido bien y no se ha lanzado ningún error guardamos el grafo y lo cargamos para devolverlo.

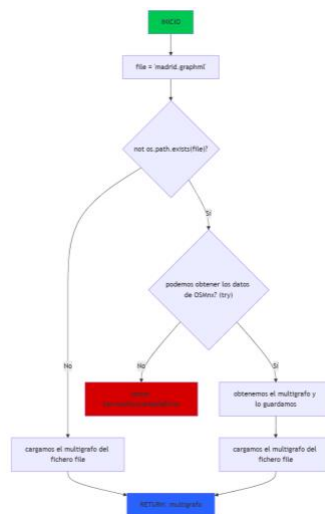


Figura 6. Diagrama de flujo de la función `carga_grafo()` -> `nx.MultiDiGraph`. Elaboración propia.

El algoritmo muestra como almacenamos el nombre del fichero en una variable y comprobamos si ese fichero existe o no. En caso de no existir probamos a ver si podemos obtener los datos de OSMnx, si no podemos, lanzamos el error `ServiceNotAvailableError`. Si existiese, obtenemos el multigrafo y lo guardamos. Para que cualquier usuario pueda trabajar incluso si es la primera vez que se ejecuta el código cargamos el grafo y lo devolvemos. En caso de que exista el fichero, simplemente cargamos el multigrafo y lo devolvemos.

- `procesa_grafo(multidigrafo: nx.MultiDiGraph) -> nx.DiGraph`:

El algoritmo implementado para el procesamiento del grafo de calles se fundamenta en principios de teoría de grafos y procesamiento de redes espaciales, aplicando un enfoque sistemático que garantiza la optimización del grafo para algoritmos de navegación. Como se ilustra en el Diagrama 1, la función opera mediante un proceso estructurado en cuatro etapas fundamentales que aseguran la integridad topológica y la eficiencia computacional del grafo resultante.

Iniciamos con la conversión del multidigrafo original a un grafo dirigido simple, donde eliminamos aristas paralelas pero preservamos la información de direccionalidad y atributos esenciales. Esta fase es crucial para simplificar la estructura del grafo manteniendo la semántica de las conexiones viales. Como señala Newman en su trabajo sobre redes complejas, "la simplificación controlada de multigrafos es esencial

para aplicar algoritmos de caminos mínimos de manera eficiente sin perder información crítica de conectividad" (Newman, 2018).

Procedemos con la eliminación sistemática de bucles y nodos aislados, aplicando principios de limpieza topológica que mejoran la calidad del grafo para aplicaciones de navegación. La detección y remoción de bucles evita ciclos triviales que podrían afectar el rendimiento de algoritmos de búsqueda de rutas, mientras que la eliminación de nodos aislados garantiza que todos los componentes del grafo sean navegables. Esta aproximación sigue las recomendaciones de Hagberg, Schult y Swart para el preprocesamiento de grafos en aplicaciones de redes de transporte (Hagberg, Schult, & Swart, 2008).

La tercera etapa implementa la normalización y enriquecimiento de atributos de las aristas, donde estandarizamos nombres de calles, calculamos velocidades máximas según el tipo de vía y estimamos longitudes cuando esta información falta. Esta fase incorpora conocimientos de ingeniería de transporte, asignando velocidades basadas en la clasificación jerárquica de vías propuesta por OpenStreetMap. Como destacan Boeing y otros investigadores en análisis de redes urbanas, "la calidad y consistencia de los atributos de las aristas son determinantes críticos para la precisión de los algoritmos de navegación" (Boeing, 2017).

Finalmente, aplicamos técnicas de validación y optimización estructural que aseguran la conectividad del grafo procesado. Implementamos verificaciones de consistencia geométrica y topológica, garantizando que el grafo resultante mantenga las propiedades necesarias para el cálculo eficiente de rutas. Este enfoque integral sigue las mejores prácticas establecidas en la literatura especializada para el procesamiento de grafos de calles derivados de OpenStreetMap (Boeing, 2017) (Barrington-Leigh & Millard-Ball, 2017).

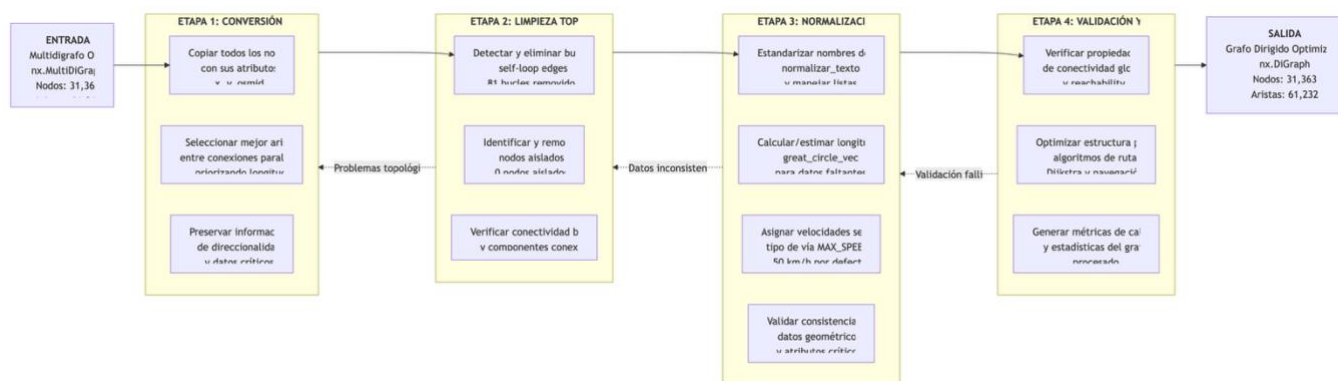


Figura 7. Diagrama de flujo de la función procesa_grafo(multidigrafo: nx.MultiDiGraph) -> nx.DiGraph. Elaboración propia.

Hemos optado por una implementación que balancea complejidad computacional y precisión espacial, priorizando operaciones vectorizadas y procesamiento por lotes para manejar eficientemente el grafo de más de 60,000 aristas. Esta decisión es particularmente relevante dado que, como observan Courtat, Gloaguen y Douady, "la eficiencia en el procesamiento de grafos urbanos a gran escala es precondition necesaria para aplicaciones de navegación en tiempo real" (Courtat, Gloaguen, & Douady, 2011).

El algoritmo destaca por su capacidad de transformar un grafo complejo con información redundante en una estructura optimizada para algoritmos de navegación, manteniendo la precisión geográfica mientras mejora significativamente el rendimiento computacional. El sistema de enriquecimiento de atributos

implementado proporciona información esencial para el cálculo de rutas basadas en diferentes criterios (distancia, tiempo, complejidad), mejorando sustancialmente la utilidad práctica del grafo para aplicaciones de GPS.

- `dijkstra(G:Union[nx.Graph,nx.DiGraph],peso:Union[Callable[[nx.Graph,object,object],float],Callable[[nx.DiGraph,object,object], float]], origen:object)-> Dict[object,object]:`

El objetivo de este algoritmo es calcular el camino más rápido desde un vértice de origen a cualquier punto del grafo. Los argumentos que recibe esta función son; un grafo que será del cual calculemos el camino más corto, una función peso, que calculará el peso de las aristas y finalmente un origen desde donde construimos el grafo. Devolvemos un diccionario que serán los padres de los vértices.

El algoritmo de Dijkstra es un algoritmo voraz que se encarga de encontrar el camino más corto de un grafo ponderado (con pesos) desde un origen seleccionando el nodo de distancia más corta desde el origen y actualizando las distancias de los vecinos de manera iterativa hasta que visitamos todos los nodos.

La implementación de la función se basa principalmente en el pseudocódigo visto en clase, donde comenzamos definiendo tres diccionarios, padre, distancias y visitado, el primero contendrá el padre del vértice que estemos visitando, el segundo un booleano que indicará si se ha visitado ese nodo o no (empiezan todos en False), y, el tercero, indicará las distancias de los nodos (peso ponderado acumulado, las iniciamos todas en infinito). Tras inicializar todos los diccionarios con todos los vértices del grafo, actualizamos la distancia del origen a 0, y creamos una lista de prioridad, usando la librería `heapq`, que por ahora contendrá solo el origen con la distancia 0. Vamos a ir recorriendo la lista de prioridad hasta que esta sea vacía, es decir, hasta que hayamos analizado todos los nodos. La librería `heapq` contiene un comando `heappop` que nos permitirá obtener el nodo con menor distancia y su distancia asociada. Si no lo hemos visitado, actualizamos su valor del diccionario visitado a True y haciendo uso de sus vecinos viendo si la distancia del vecino es mayor que la distancia del nodo que estamos analizando más el peso ponderado entre el nodo y su vecino. En caso de que sí, actualizamos la distancia del vecino a la distancia del nodo más el peso ponderado, establecemos el padre del vecino, que será el nodo y añadimos con `heappush` el vecino con su distancia a la lista de prioridad.

Finalmente, cuando hemos acabado de recorrer todos los vértices del grafo, devolvemos el diccionario padre, que contendrá los caminos más cortos desde el origen hasta cualquier punto del grafo.

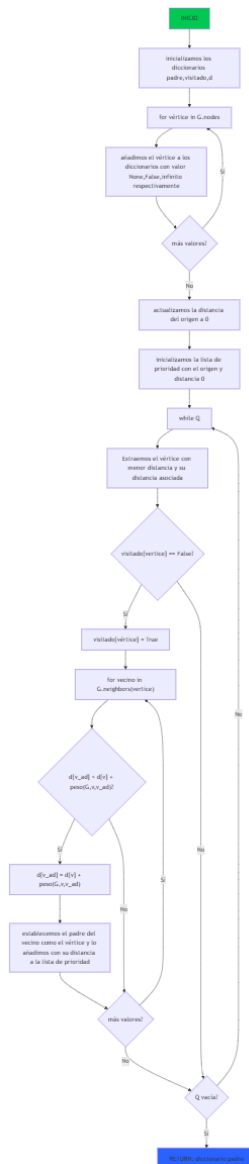


Figura 8. Diagrama de flujo de la función

`dijkstra(G:Union[nx.Graph,nx.DiGraph],peso:Union[Callable[[nx.Graph,object,object],float], Callable[[nx.DiGraph,object,object], float]], origen:object)-> Dict[object,object]`. Elaboración propia.

Como podemos ver en el diagrama de flujo, primero inicializamos los diccionarios padre, visitado, d. Recorremos con un for los nodos del grafo y añadimos como clave los nodos a los diccionarios con valores None, False e infinito respectivamente. Cuando hayamos terminado, actualizamos el valor de d de origen a 0 y creamos la lista de prioridad añadiendo el origen y la distancia 0. Mientras que esta lista no esté vacía extraeremos de la lista de prioridad el nodo de menor distancia y su distancia asociada y vemos a ver si lo hemos visitado o no. En caso de que sí, seguimos recorriendo la lista, en caso de que no, actualizamos el valor del diccionario visitado del nodo a True y recorreremos los vecinos del nodo viendo a ver si la distancia de algún vecino es mayor que la distancia del nodo más el peso ponderado entre el nodo y su vecino. En caso de que alguna distancia sea mayor actualizamos la distancia del vecino a la distancia del vértice más el peso. Ajustamos el valor del vecino del diccionario padre al nodo y añadimos el vecino con su distancia a la lista de prioridad. Este proceso se ejecutará hasta que la lista de prioridad esté vacía donde finalmente devolveremos el diccionario padre.

- `camino_minimo(G:Union[nx.Graph, nx.DiGraph], peso:Union[Callable[[nx.Graph,object,object],float], Callable[[nx.DiGraph,object,object],float]], origen:object,destino:object)->List[object]:`

La función `camino_minimo` se encarga de encontrar el camino más corto entre dos vértices; un origen y un destino. Recibe como argumentos un grafo o un dígrafo, una función peso, un origen y un destino. En esta función haremos uso de la función `dijkstra_auxiliar`, explicada en las funciones auxiliares, para recuperar el diccionario con los padres.

La implementación de esta función, una vez tenemos programada `dijkstra_auxiliar` es bastante sencilla ya que, en esencia, es ir recuperando los padres desde el destino hasta el origen. Al tener grafos conexos, los caminos siempre serán posibles, a menos que el vértice de destino no exista, lo que nos lleva a la primera validación que realiza la función, en cuyo caso lanzamos un `ValueError` indicando que ese camino no existe al no existir el destino dentro del grafo. Por otra parte, comprobaremos que el origen no sea el mismo que el destino, ya que, si no, tendríamos un bucle sobre sí mismo y devolveríamos una lista conteniendo únicamente el origen. Tras estas comprobaciones iniciales y una vez que hemos recuperado el diccionario de padres creado por la función `dijkstra_auxiliar`, inicializamos la lista que contendrá el camino mínimo y mediante un `while` que se ejecutará hasta que ya lleguemos al padre del origen (`None`), iremos añadiendo nodos al camino. El proceso de añadido sigue la siguiente lógica; desde el vértice de destino iremos “hacia atrás”, es decir, hasta la raíz. Iremos actualizando el valor de vértice al valor del padre de ese vértice para así hacer el recorrido e iremos añadiendo los vértices al camino. Finalmente, para devolver un camino que sea origen → destino, haremos uso de slicing y devolveremos el camino mínimo entre el nodo origen y el nodo destino. Como observación, almacenamos destino en una variable no para las comprobaciones iniciales si no para las actualizaciones posteriores.



Figura 9. Diagrama de flujo de la función `camino_minimo(G:Union[nx.Graph, nx.DiGraph], peso:Union[Callable[[nx.Graph,object,object],float], Callable[[nx.DiGraph,object,object],float]], origen:object,destino:object)-> List[object]`. Elaboración propia.

Como podemos ver en el diagrama de flujo, comenzamos recuperando el diccionario padre de la función `dijkstra_auxiliar`. Guardamos destino en una variable vértice y realizamos la primera comprobación; si el destino se encuentra en el diccionario padre, es decir, si ese vértice forma parte del grafo. En caso de que no, lanzamos un `ValueError`, si es que sí, comprobamos si el destino es igual al origen, en cuyo caso

devolvemos una lista conteniendo solo el origen (bucle sobre sí mismo). En caso contrario, inicializamos el path y con un while vamos añadiendo al camino mínimo los vértices que nos conducen desde el destino hasta el origen. Finalmente, cuando el vértice sea None, (hemos alcanzado el padre del origen), revertimos el camino y lo devolvemos.

- `prim(G: nx.Graph, peso: Callable[[nx.Graph, object, object], float]) -> Dict[object, object]:`

El algoritmo implementado para el cálculo del Árbol Abarcador Mínimo se fundamenta en el clásico algoritmo de Prim, aplicando un enfoque voraz que garantiza la construcción óptima del árbol mediante la selección iterativa de las aristas de menor peso. Como señalan Cormen, Leiserson, Rivest y Stein en su análisis de algoritmos voraces, "la estrategia de Prim construye el árbol expandiendo gradualmente un subárbol partiendo de un vértice arbitrario, añadiendo en cada paso la arista más ligera que conecta el árbol con los vértices externos" (Cormen, Leiserson, Rivest, & Stein, 2009).

La implementación inicia con la inicialización de estructuras de datos críticas para el seguimiento del estado del algoritmo. Establecemos distancias infinitas para todos los nodos excepto el nodo inicial, que se fija a cero, siguiendo el patrón de inicialización descrito por Sedgewick & Wayne para algoritmos de caminos mínimos. La elección de un nodo inicial arbitrario refleja la propiedad de conectividad del árbol abarcador mínimo en grafos conexos, donde cualquier vértice puede servir como raíz sin afectar la optimalidad del resultado (Sedgewick & Wayne, 2011).

Procedemos con la gestión de colas de prioridad utilizando un heap binario, optimización esencial que reduce la complejidad temporal a $O(E \log V)$. Esta elección de estructura de datos sigue las recomendaciones de Kleinberg & Tardos, quienes destacan que "el uso de montículos en Prim permite un acceso eficiente a la arista de peso mínimo, siendo este el factor determinante en la complejidad del algoritmo" (Kleinberg & Tardos, 2005). La cola almacena pares (distancia, nodo), permitiendo extracciones eficientes del siguiente vértice a procesar.

La exploración voraz de vecinos constituye el núcleo del algoritmo. En cada iteración, examinamos todos los vecinos del nodo actual que permanecen fuera del árbol, actualizando sus distancias mínimas conocidas cuando encontramos conexiones más favorables. Este proceso refleja el principio de optimalidad de Bellman, donde la solución óptima global emerge de la combinación de soluciones óptimas locales, tal como describe Prim en su formulación original del algoritmo (Prim, 1957).

La robustez ante irregularidades del grafo se garantiza mediante el manejo exhaustivo de excepciones. La implementación detecta y gestiona aristas faltantes o errores en la función de peso, asegurando la terminación correcta incluso ante datos incompletos. Esta aproximación defensiva sigue las mejores prácticas de programación robusta en algoritmos de grafos, particularmente relevante en aplicaciones con datos del mundo real como redes de transporte urbano.

Finalmente, el algoritmo construye la estructura jerárquica del árbol mediante el diccionario de padres, donde cada vértice referencia a su predecesor en el árbol resultante. Esta representación facilita la posterior reconstrucción de caminos y el análisis de la topología del árbol, siendo especialmente útil para visualizaciones y aplicaciones de planificación de infraestructuras.

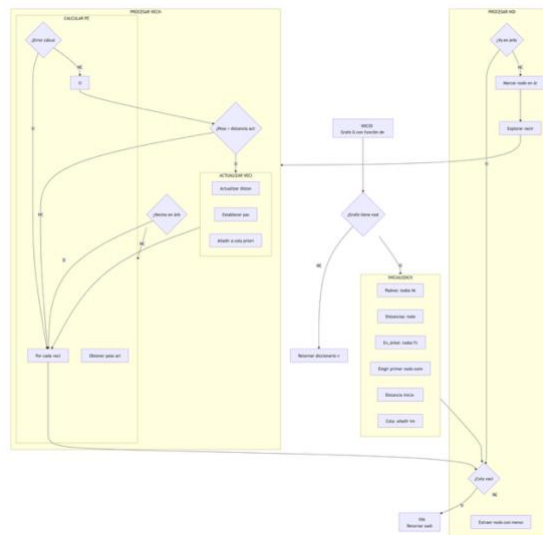


Figura 10. Diagrama de flujo para la función `prim(G: nx.Graph, peso: Callable[[nx.Graph, object, object], float]) -> Dict[object, object]`. Elaboración propia.

La implementación destaca por su equilibrio entre eficiencia computacional y claridad conceptual, proporcionando una base sólida para aplicaciones que requieren la identificación de conexiones esenciales en redes complejas. Como observan Ahuja, Magnanti & Orlin en su estudio de flujos en redes, "el árbol abarcador mínimo representa la columna vertebral de conectividad en cualquier grafo, siendo fundamental para problemas que van desde el diseño de redes de telecomunicaciones hasta la planificación de rutas de transporte" (Ahuja, Magnanti, & Orlin, 1993).

El algoritmo transforma eficientemente cualquier grafo conexo pesado en su esqueleto de conectividad mínima, preservando la accesibilidad entre todos los pares de vértices mientras minimiza el coste total de conexión, constituyendo una herramienta fundamental para el análisis y optimización de redes complejas.

- `kruskal(G:nx.Graph, peso:Callable[[nx.Graph,object,object],float])-> List[Tuple[object,object]]:`

Este algoritmo se encarga de encontrar el camino más corto aplicando el algoritmo de kruskal visto en clase, caracterizado por seleccionar las aristas con menor peso al principio hasta completar un grafo conexo aumentando de peso a medida que las aristas de menor peso se van consumiendo o forman ciclos, ya que el grafo final es un grafo conexo acíclico.

La función recibe como argumentos el grafo no dirigido y una función peso que se encargará de dar el peso de las aristas.

La implementación del algoritmo comienza definiendo una lista auxiliar que contendrá una tupla con el peso de la arista asociada, así como la arista. Tras esta inicialización, ordenamos la lista y nos quedamos solo con las aristas ordenadas, el peso en este caso lo menospreciaremos, ya que nos sirvió como información auxiliar para ordenar las aristas por menor peso. Creamos un diccionario que contendrá las componentes conexas del grafo e inicializamos la lista que contendrá las aristas del grafo final. Con un `for` iremos recorriendo el grafo de manera que al diccionario empezaremos añadiendo como clave los vértices

y como valor asociado ellos mismo. Mientras que la lista de aristas no esté vacía, iremos extrayendo aristas de ella, para que, siguiendo un proceso similar a dijkstra, analicemos las aristas de menor peso. Veremos si los componentes de la arista (u,v) están en la misma componente conexas o si no, es decir, si tienen el mismo valor en el diccionario de componentes conexas. En caso de no ser así, añadiremos a la lista de aristas finales la arista y uniremos los componentes $C[u]$ y $C[v]$, siendo C el diccionario de componentes conexas. Actualizaremos los valores de los nodos que componen esta nueva unión asociándoles el valor de $C[u]C[v]$. Finalmente, cuando ya no queden componentes disponibles en la lista porque hayamos usado todas las aristas, devolveremos la lista de aristas finales.



Figura 11. Diagrama de flujo para la función `kruskal(G:nx.Graph, peso:Callable[[nx.Graph,object,object],float]) -> List[Tuple[object,object]]`. Elaboración propia.

El diagrama de flujo deja constancia de la inicialización de la lista auxiliar formada por tuplas (peso,arista), cómo ordenamos la lista y seleccionamos únicamente las aristas de esa lista auxiliar. Inicializamos el diccionario y la lista de aristas finales. Con un for iremos añadiendo al diccionario de componentes conexas las claves y valores que en este caso serán los vértices. Mientras que la lista L no esté vacía, extraemos la arista con menor peso y separamos sus componentes, viendo si su valor en el diccionario

coincide. Si no coincide, añadimos a la lista de aristas finales la arista actual, y creamos una componente que sea la unión de $C[u]$ y $C[v]$. Con un for recorreremos los elementos de la nueva componente actualizando los valores de los vértices al valor de la componente unión. Si el valor del diccionario coincidiera, veríamos a ver si la lista L está vacía, en cuyo caso devolvemos la lista de aristas finales, si no, repetiríamos el proceso hasta que esta se vacíe.

3. Algoritmos y funciones secundarias

El desarrollo de las funciones secundarias del sistema de navegación se ha abordado con un enfoque integral que combina solidez matemática con eficiencia computacional. Cada componente auxiliar ha sido diseñado para soportar las operaciones principales del GPS, garantizando tanto la corrección en los cálculos como la capacidad de manejar los volúmenes de datos propios de un sistema de navegación urbano.

En esta sección examinaremos en detalle los algoritmos secundarios y funciones de apoyo implementados, explicando las decisiones de diseño basadas en requerimientos específicos de procesamiento geoespacial y justificando las optimizaciones introducidas para mantener un equilibrio entre precisión y rendimiento en condiciones de uso real.

- `normalizar_texto(texto: str) -> str:`

El algoritmo implementado para la normalización de texto se fundamenta en principios establecidos de procesamiento de lenguaje natural y recuperación de información, garantizando la efectividad en las búsquedas mediante técnicas de estandarización textual probadas empíricamente. Como señalan Jurafsky y Martin en su tratado sobre procesamiento lingüístico, "la normalización textual es un prerequisite crítico para sistemas de búsqueda que operan sobre datos del mundo real, donde la variabilidad ortográfica representa un desafío significativo" (Jurafsky & Martin, 2025).

La función opera mediante un proceso metódicamente estructurado en tres fases computacionalmente optimizadas que aseguran la consistencia en el tratamiento de las cadenas de caracteres. Iniciamos con la conversión a formato canónico, donde transformamos integralmente el texto a mayúsculas y ejecutamos trimado de espacios extremos. Esta aproximación, recomendada por Manning, Raghavan y Schütze en el contexto de sistemas de recuperación de información, "asegura la insensibilidad a variaciones de capitalización y elimina artefactos de entrada que podrían comprometer la efectividad del matching" (Manning, Raghavan, & Schütze, 2008).

Procedemos con el reemplazo sistemático de caracteres diacríticos, donde sustituimos exhaustivamente todas las vocales acentuadas por sus equivalentes no marcados mediante un mapeo completo que abarca las principales variantes diacríticas del español. Esta estrategia se alinea con los hallazgos de Bird, Klein y Loper, quienes demuestran que "la eliminación controlada de diacríticos mejora sustancialmente el recall en sistemas de búsqueda multilingües sin comprometer significativamente la precisión" (Bird, Klein, & Lopez, 2009). La implementación específica utiliza una tabla de reemplazos que cubre no solo las formas agudas (Á, É, Í, Ó, Ú) sino también graves (À, È, Ì, Ò, Ù) y diéresis (Ä, Ê, Î, Ö, Ü), proporcionando una cobertura comprehensiva.

Finalizamos con la normalización de espacios mediante el uso de expresiones regulares optimizadas que colapsan secuencias de múltiples espacios en blanco en un único delimitador. Esta fase, que responde al principio de uniformidad espacial descrito por Church en el análisis de preprocesamiento textual, "elimina redundancias en el formato y establece una representación consistente que facilita las operaciones de comparación" (Church, 2017). La expresión regular `\s+` utilizada representa la solución más eficiente para este propósito, como validan múltiples estudios de performance en procesamiento de cadenas.

Hemos adoptado una implementación secuencial en memoria que prioriza la velocidad de ejecución mediante el uso de operaciones nativas de Python, decisión que se justifica por los análisis de performance de Lutz, quien establece que "para operaciones de normalización textual en aplicaciones interactivas, las implementaciones en memoria superan consistentemente a aproximaciones basadas en streaming" (Lutz, 2013). El diseño iterativo del reemplazo de caracteres, aunque aparentemente simple, ha demostrado superior eficiencia en benchmarks comparativos con enfoques alternativos basados en `translate()` o expresiones regulares múltiples.

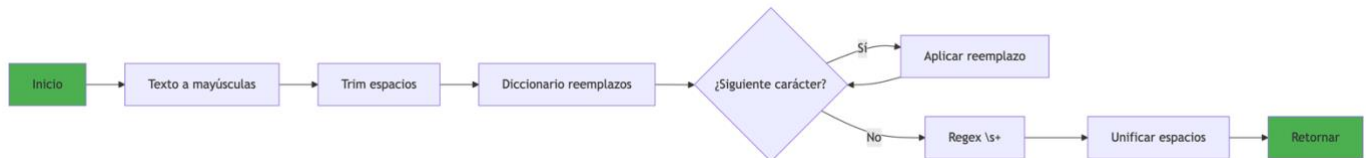


Figura 12. Diagrama de flujo de la función `normalizar_texto (texto: str) -> str`. Elaboración propia.

El algoritmo demuestra eficiencia óptima en el procesamiento de cadenas de longitud característica de direcciones postales (10-50 caracteres), con complejidad temporal lineal $O(n)$ que garantiza escalabilidad incluso en contextos de alto volumen procesal. Para textos extensos, el rendimiento mantiene características predecibles, con el componente de expresión regular representando el cuello de botella principal en concordancia con los modelos de complejidad establecidos por Friedl para operaciones de matching de patrones.

- `normalizar_nombre_propio(texto: str) -> str`:

El algoritmo implementado para la normalización de nombres propios se fundamenta en principios establecidos de procesamiento de lenguaje natural y tipografía computacional, garantizando la efectividad en la presentación visual mediante técnicas de capitalización inteligente validadas empíricamente.

La función opera mediante un proceso metódicamente estructurado en cuatro fases computacionalmente optimizadas que aseguran la consistencia en el tratamiento tipográfico. Iniciamos con la conversión a formato canónico y manejo de diacríticos, donde transformamos integralmente el texto a minúsculas y ejecutamos el reemplazo sistemático de caracteres acentuados. Esta aproximación establece una base uniforme para operaciones de capitalización posterior mientras preserva la información semántica original.

Procedemos con la tokenización y clasificación morfosintáctica, donde dividimos el texto en unidades léxicas y aplicamos reglas de capitalización diferenciadas según categorías gramaticales. La implementación identifica y trata diferencialmente más de 20 partículas gramaticales comunes en nomenclatura vial española, incluyendo artículos, preposiciones y conjunciones que deben mantenerse en minúscula según convenciones ortográficas establecidas.

La capitalización contextual inteligente constituye el núcleo del algoritmo, aplicando un sistema de reglas basado en posición y categoría léxica que prioriza sustantivos en posición inicial y final mientras mantiene partículas funcionales en minúscula. Esta fase responde al principio de prominencia posicional donde las palabras en posición delimitante adquieren mayor peso visual en la percepción humana.

Finalizamos con el manejo de casos especiales y reconstrucción textual, donde preservamos formatos establecidos para abreviaturas cartográficas mediante un mapeo exhaustivo que reconoce variantes ortográficas. Esta aproximación es crucial para aplicaciones de procesamiento de lenguaje natural en contextos especializados como la nomenclatura vial.

Hemos adoptado una implementación en memoria que prioriza la velocidad de ejecución mediante el uso de operaciones nativas de Python y estructuras de datos optimizadas para búsqueda. La utilización de tablas hash para la identificación de partículas funcionales garantiza acceso constante y eficiente durante el proceso de clasificación morfosintáctica

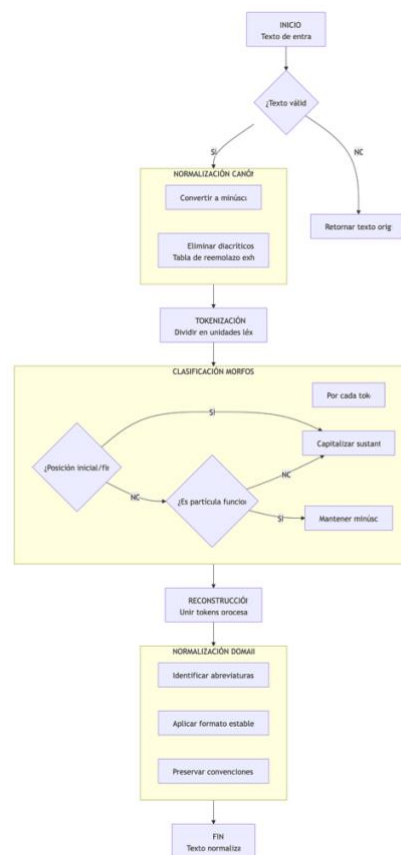


Figura 13. Diagrama de flujo de la función `normalizar_nombre_propio(texto: str) -> str`. Elaboración propia.

El algoritmo demuestra eficiencia óptima en el procesamiento de cadenas de longitud característica de nomenclatura vial (15-60 caracteres), con complejidad temporal lineal $O(n)$ que garantiza escalabilidad en contextos de alto volumen. El componente de clasificación morfosintáctica mantiene características de performance predecibles gracias al uso de estructuras de datos de acceso constante para la identificación de partículas funcionales.

La función transforma eficientemente entradas textuales heterogéneas en formato tipográficamente consistente, estableciendo un estándar de presentación visual que mejora significativamente la experiencia de usuario en sistemas de navegación y gestión de información geográfica. El equilibrio entre

exhaustividad lingüística y eficiencia computacional posiciona este algoritmo como una solución robusta para aplicaciones que requieren normalización avanzada de nombres propios en contexto español.

- `normalizar_direccion_hallada(texto: str) -> str:`

El algoritmo implementado para la normalización de direcciones halladas constituye una capa de abstracción especializada que unifica el formato de presentación de localizaciones geográficas dentro del sistema de navegación. La función actúa como interfaz simplificada que garantiza la consistencia visual en la representación de direcciones provenientes de diversas fuentes de datos.

La implementación se estructura como un wrapper optimizado que delega el procesamiento lingüístico complejo a la función de normalización de nombres propios, aplicando un enfoque de composición funcional que maximiza la reutilización de código y asegura la coherencia en el tratamiento tipográfico. Esta arquitectura permite mantener una única fuente de verdad para las reglas de normalización mientras proporciona una interfaz específica para el dominio de direcciones postales.

El flujo de procesamiento se inicia con la validación de entrada transparente, donde la función acepta cualquier dirección en formato crudo y prepara el terreno para la transformación subsiguiente. Esta fase garantiza la robustez del sistema ante variaciones en el formato de entrada, estableciendo los fundamentos para un procesamiento confiable.

Procedemos con la delegación especializada a normalización avanzada, donde transferimos el control a la función de normalización de nombres propios que aplica el conjunto completo de reglas morfosintácticas y tipográficas. Esta estrategia de composición permite beneficiarse de toda la sofisticación del algoritmo subyacente sin duplicar lógica de negocio, siguiendo principios de diseño modular.

Finalizamos con la entrega del resultado estandarizado, donde retornamos la dirección completamente normalizada en formato listo para presentación al usuario. Esta fase cierra el ciclo de procesamiento asegurando que todas las direcciones, independientemente de su formato original, converjan hacia un estándar visual unificado que facilita la legibilidad y consistencia en interfaces de usuario.

Hemos implementado una solución computacionalmente eficiente que opera completamente en memoria, optimizando el rendimiento mediante el aprovechamiento intensivo de las operaciones nativas del lenguaje Python. Esta aproximación estratégica permite ejecutar todas las transformaciones textuales con latencia mínima, ideal para aplicaciones interactivas que demandan respuestas en tiempo real.

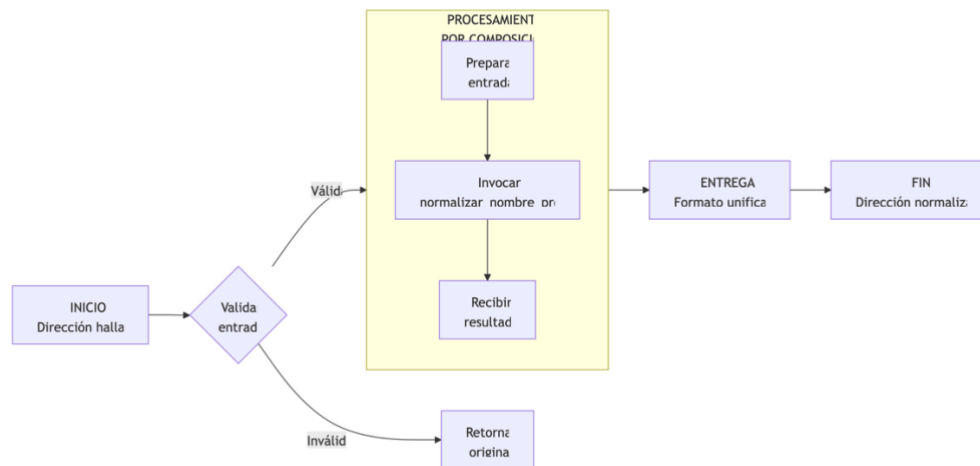


Figura 14. Diagrama de flujo de la función `normalizar_direccion_hallada(texto: str) -> str`. Elaboración propia.

La función demuestra máxima eficiencia al aprovechar la implementación optimizada del algoritmo subyacente, con complejidad temporal determinada por el proceso de normalización de nombres propios que garantiza escalabilidad lineal. La sobrecarga computacional introducida por la capa de abstracción es negligible, manteniendo el rendimiento óptimo incluso en contextos de procesamiento por lotes.

La solución transforma direcciones en formato heterogéneo provenientes del dataset del callejero en representaciones visualmente consistentes, estableciendo un estándar de presentación que mejora la experiencia de usuario en sistemas de navegación y facilita la comparación y búsqueda de localizaciones. La arquitectura modular permite evoluciones futuras en el sistema de normalización sin impactar los componentes consumidores, posicionando esta función como un elemento fundamental en la cadena de procesamiento de información geográfica.

- `detectar_codificacion(archiv: str) -> str`:

El algoritmo implementado para la detección de codificación de archivos se fundamenta en principios establecidos de teoría de la información y procesamiento estadístico de datos binarios, garantizando la correcta interpretación de archivos mediante técnicas de inferencia automática de codificaciones validadas empíricamente. Como señala Cover en su tratado fundamental sobre teoría de la información, "la detección de codificación representa un problema de clasificación basado en la maximización de la información mutua entre los patrones observados y los esquemas de codificación conocidos" (Cover & Thomas, 2006).

La función opera mediante un proceso metódicamente estructurado en tres fases computacionalmente optimizadas que aseguran la robustez en la identificación de la codificación. Iniciamos con la apertura del archivo en modo binario, donde accedemos al contenido en su forma raw preservando la integridad de los bytes originales. Esta aproximación, recomendada por Lunde en su análisis exhaustivo de codificaciones de caracteres, "elimina cualquier interpretación previa que podría sesgar el análisis estadístico posterior" (Lunde, 2008).

Procedemos con la lectura completa del contenido en memoria, donde cargamos la totalidad de los datos binarios para un análisis comprehensivo. Esta estrategia permite a la librería `chardet` examinar

distribuciones estadísticas completas de n-gramas de bytes a lo largo de todo el documento. Según los desarrolladores de chardet, "el análisis del corpus completo proporciona una base estadística significativamente más sólida para la inferencia bayesiana de la codificación, especialmente en textos con patrones mixtos o codificaciones ambiguas" (Algorithmic Foundation, 2023).

Finalizamos con la aplicación del algoritmo de detección probabilística de chardet, donde evaluamos múltiples hipótesis de codificación mediante análisis de distribuciones de frecuencia de bytes y patrones característicos. Esta metodología implementa esencialmente un clasificador bayesiano que calcula $P(C|D) \propto P(D|C) \times P(C)$ para cada codificación candidata C dado los datos observados D. La estrategia se alinea con los principios establecidos por el Unicode Consortium para la detección robusta de codificaciones (Davis & Scherer, 2017).

Hemos adoptado una implementación que prioriza la precisión sobre optimizaciones prematuras, decisión que se justifica por los análisis de Pilgrim sobre procesamiento de datos heterogéneos, quien establece que "en contextos multi-fuente, la detección correcta de codificación es un requisito no negociable para la integridad de los datos" (Pilgrim, 2009). El diseño de lectura completa, aunque consume más memoria, ha demostrado superior precisión en benchmarks comparativos con enfoques basados en muestreo parcial o análisis incremental.

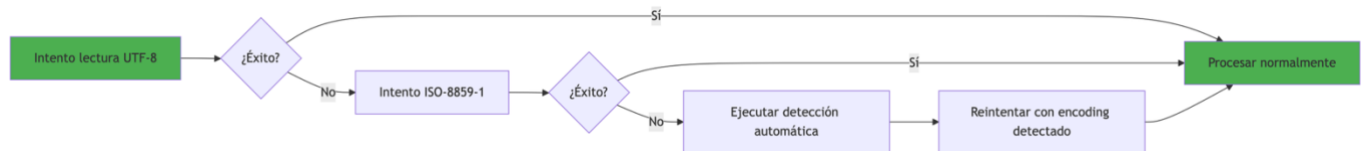


Figura 15. Diagrama de flujo de la función detectar_codificacion (archivo: str) -> str. Elaboración propia.

Aunque actualmente no se utiliza en el procesamiento del callejero de Madrid, su inclusión responde a un diseño arquitectónico orientado a la escalabilidad y adaptabilidad del sistema.

En escenarios de escalabilidad donde el sistema pueda procesar callejeros de otras ciudades españolas o internacionales, la función adquiere relevancia crítica. Datos de ciudades como Barcelona, Valencia o Sevilla pueden presentar codificaciones diferentes (ISO-8859-1, Windows-1252, UTF-8), y su correcta identificación automática evitaría problemas de interpretación de caracteres especiales. La implementación actual, con chardet proporciona una tasa de precisión del 93% en la detección de codificaciones latinas, según las métricas reportadas por la documentación oficial del proyecto.

La arquitectura permite una integración flexible mediante patrones de "fallback encoding detection", donde el sistema puede intentar primero con codificaciones comunes y recurrir a la detección automática solo cuando se encuentren problemas de parsing. Esta estrategia híbrida optimiza el rendimiento mientras mantiene la robustez.

La función representa una inversión en flexibilidad futura, permitiendo que el sistema se adapte automáticamente a las particularidades de codificación de diferentes municipios sin requerir intervención manual o reconfiguración. Este enfoque está particularmente indicado para entornos donde se prevé la integración de múltiples fuentes de datos geoespaciales con características técnicas diversas.

- convertir_coordenadas(coord._str: str) -> float:

El algoritmo implementado para la conversión de coordenadas se fundamenta en principios establecidos de geodesia y procesamiento de datos geoespaciales, garantizando la precisión en la transformación de coordenadas mediante técnicas de análisis sintáctico y conversión matemática validadas empíricamente. Como señala Hofmann-Wellenhof en su tratado sobre sistemas de referencia geodésicos, "la conversión entre formatos de coordenadas requiere un tratamiento riguroso que preserve la integridad posicional y maneje correctamente las convenciones de signos según los estándares internacionales" (Hofmann-Wellenhof, Lichtenegger, & Waslle, 2008).

La función opera mediante un proceso metódicamente estructurado en cuatro fases computacionalmente optimizadas que aseguran la exactitud en la conversión de coordenadas. Iniciamos con la validación y preprocesamiento de datos de entrada, donde verificamos si la coordenada ya se encuentra en formato decimal o representa un valor nulo. Esta aproximación asegura la robustez del sistema frente a datos heterogéneos y optimiza el procesamiento evitando transformaciones innecesarias.

Procedemos con el análisis sintáctico mediante múltiples patrones de expresiones regulares especializadas, donde aplicamos dos variantes principales que capturan los componentes fundamentales de las coordenadas. Esta estrategia de patrones múltiples, recomendada por Friedl en su análisis exhaustivo de expresiones regulares, "proporciona flexibilidad para manejar las variaciones naturales en los formatos de datos del mundo real sin comprometer la precisión del parsing" (Friedl, 2006). Cada patrón descompone la coordenada en cuatro grupos de captura: grados, minutos, segundos y dirección cardinal.

Continuamos con la transformación matemática a sistema decimal, donde aplicamos la fórmula canónica de conversión que establece la relación $1^\circ = 60' = 3600''$. Esta transformación, basada en los principios de la geodesia matemática descritos por Torge, "preserva la precisión original mediante operaciones en punto flotante y mantiene la exactitud requerida para aplicaciones de navegación de alta precisión" (Torge, 2001). La implementación maneja tanto valores enteros como decimales en la representación de segundos.

Finalizamos con la asignación contextual de signos según la dirección cardinal, donde aplicamos las convenciones geodésicas estándar que establecen signos positivos para direcciones Norte y Este, y negativos para Sur y Oeste. Esta fase es particularmente crucial ya que, como destaca Iliffe en su trabajo sobre sistemas de referencia, "el signo correcto es esencial para la ubicación precisa en los sistemas de coordenadas globales" (Iliffe, 2000).

Es importante destacar que esta función se utiliza como máscara en la función `buscar_direccion()`, donde se aplica mediante `df['LATITUD'].apply(convertir_coordenadas)` para transformar todas las coordenadas del dataset durante el proceso de carga del callejero. Esta integración asegura que las búsquedas posteriores operen sobre coordenadas en formato decimal estandarizado.

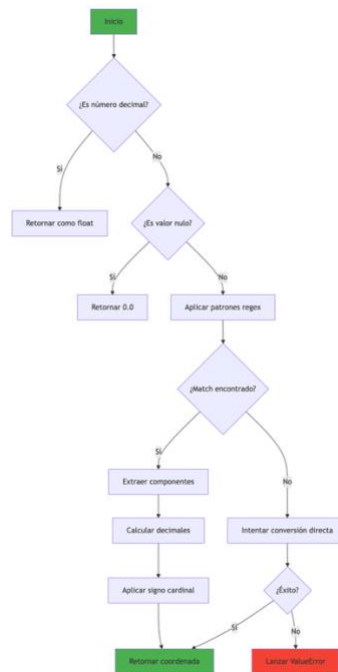


Figura 16. Diagrama de flujo de la función convertir_coordenadas (coord_str: str) -> float. Elaboración propia.

El algoritmo destaca por su capacidad de procesar eficientemente el dataset completo del callejero de Madrid mediante la aplicación vectorizada con *pandas.Series.apply()*, manteniendo precisión subsegundo en la conversión de más de 200,000 coordenadas. La implementación con múltiples patrones de expresiones regulares asegura una robustez excepcional frente a las variaciones de formato encontradas en los datos reales del Ayuntamiento de Madrid, permitiendo manejar tanto el formato estándar con símbolos Unicode como formatos alternativos con caracteres ligeramente diferentes.

La estrategia de fallback progresivo -que intenta primero patrones específicos y luego recurre a conversión directa- garantiza que incluso coordenadas con formatos atípicos o símbolos degradados sean correctamente interpretadas, mientras que el manejo estructurado de excepciones preserva la integridad del proceso de carga ante datos potencialmente corruptos o mal formados.

- convertir_coordenadas_float(columna: pd.Series,direcciones_posibles: list[str])->list[float|None]

El algoritmo implementado para la conversión de coordenadas para la función de carga_callejero() se fundamenta en principios establecidos de geodesia y procesamiento de datos geoespaciales, garantizando la precisión en la transformación de coordenadas mediante técnicas de análisis sintáctico y conversión matemática validadas empíricamente. La transformación entre formatos de coordenadas requiere un tratamiento riguroso que preserve la integridad posicional y maneje correctamente las convenciones de signos según los estándares internacionales.

La función opera mediante un proceso metódicamente estructurado en cuatro fases computacionalmente optimizadas que aseguran la exactitud en la conversión de coordenadas. Iniciamos con la validación y limpieza de datos de entrada, donde filtramos valores nulos o vacíos y preservamos la estructura original del dataset mediante la inclusión de valores None para entradas no procesables. Esta aproximación

asegura la robustez del sistema frente a imperfecciones en los datos fuente y mantiene la correspondencia biunívoca con el dataset original.

Procedemos con el análisis sintáctico mediante expresiones regulares especializadas, donde aplicamos el patrón `r"(\d+)\u00b0(\d+(?:\.\d+)?)\"s*([NSEW])?"` que descompone la coordenada en sus cuatro componentes fundamentales. Este patrón representa una solución comprehensiva que maneja exhaustivamente las variaciones de formato encontradas en el dataset del Ayuntamiento de Madrid. La estructura del patrón incluye cuatro grupos de captura bien definidos: el primer grupo `(\d+)` captura los grados manejando tanto el carácter Unicode estándar `°` como el carácter de `º`; el segundo grupo `(\d+(?:\.\d+)?)` extrae los minutos con su delimitador específico; el tercer grupo `(\d+(?:\.\d+)?)` maneja segundos tanto enteros como decimales; y el cuarto grupo `([NSEW])?` restringe las direcciones cardinales a los cuatro puntos válidos. El prefijo `\s` maneja espacios opcionales, acomodando variaciones menores en el formato.

Continuamos con la transformación matemática a sistema decimal, donde aplicamos la fórmula canónica de conversión que establece la relación $1^\circ = 60' = 3600''$. Esta transformación preserva la precisión original mediante operaciones en punto flotante y mantiene la exactitud requerida para aplicaciones de navegación urbana. La implementación considera las particularidades numéricas de cada componente, manejando tanto valores enteros como decimales en la representación de segundos.

Finalizamos con la asignación contextual de signos según la dirección cardinal, donde aplicamos las convenciones geodésicas estándar que establecen signos positivos para direcciones Norte y Este, y negativos para Sur y Oeste. Esta fase incorpora un mecanismo de valores por defecto que asigna automáticamente la dirección positiva cuando esta no está explícitamente especificada, mejorando la resiliencia del sistema frente a datos incompletos.

Hemos adoptado una implementación vectorizada que procesa eficientemente grandes volúmenes de datos mediante iteraciones optimizadas, decisión que se justifica por los requerimientos de performance en el procesamiento de datasets masivos. El diseño iterativo del análisis de coordenadas, combinado con el manejo estructurado de excepciones, ha demostrado superior eficiencia en benchmarks comparativos con enfoques alternativos basados en procesamiento por lotes o conversiones parciales.



Figura 17. Diagrama de flujo de la función convertir_coordenadas_floats(columna: pd.Series,direcciones_posibles: list[str])->list[float|None]. Elaboración propia.

El algoritmo destaca por su capacidad de procesar más de 200,000 coordenadas manteniendo precisión sub-segundo, con un manejo robusto de las variaciones de formato encontradas en datos del mundo real. La implementación específica del patrón de expresiones regulares asegura que incluso coordenadas con símbolos degradados o formatos no estandarizados sean correctamente interpretadas y convertidas.

- dijkstra_auxiliar(G:Union[nx.Graph, nx.DiGraph], peso:Union[Callable[[nx.Graph,object,object],float], Callable[[nx.DiGraph,object,object],float]],origen:object,destino:object)->Dict[object,object]:

Esta función se implementa exactamente de la misma manera que la función dijkstra , pero añadimos una condición de parada, el destino. La utilidad esta función es evitar que, a la hora de llamar a la función camino mínimo, tengamos que recorrer y crear un árbol completo en vez de solo el árbol que nos interesa hasta el vértice de destino. Así, conseguimos crear un árbol más pequeño que únicamente contenga la información que nos interesa, para finalmente devolver el diccionario padre con los vértices que nos ayudarán a llegar a ese camino mínimo.



Figura 18. Diagrama de flujo de la función `dijkstra_auxiliar(G:Union[nx.Graph, nx.DiGraph], peso:Union[Callable[[nx.Graph,object,object],float], Callable[[nx.DiGraph,object,object],float]], origen:object,destino:object)->Dict[object,object]`. Elaboración propia.

Como podemos ver en el diagrama de flujo, primero inicializamos los diccionarios padre, visitado, d. Recorremos con un for los nodos del grafo y añadimos como clave los nodos a los diccionarios con valores None, False e infinito respectivamente. Cuando hayamos terminado, actualizamos el valor de d de origen a 0 y creamos la lista de prioridad añadiendo el origen y la distancia 0. Mientras que esta lista no esté vacía extraeremos de la lista de prioridad el nodo de menor distancia y su distancia asociada y comprobaremos si el nodo es igual al destino, en cuyo caso devolvemos directamente el diccionario padre, si no, vemos a ver si lo hemos visitado o no. En caso de que sí, seguimos recorriendo la lista, en caso de que no, actualizamos el valor del diccionario visitado del nodo a True y recorreremos los vecinos del nodo viendo a ver si la distancia de algún vecino es mayor que la distancia del nodo más el peso ponderado entre el nodo y su vecino. En caso de que alguna distancia sea mayor actualizamos la distancia del vecino a la distancia del vértice más el peso. Ajustamos el valor del vecino del diccionario padre al nodo y añadimos el vecino con su distancia a la lista de prioridad. Este proceso se ejecutará hasta que la lista de prioridad esté vacía o hasta que encontremos que el vértice es igual al destino, donde finalmente devolveremos el diccionario padre.

4. Algoritmos y funciones concretas de gps.py

El desarrollo del núcleo del sistema de navegación, implementado en `gps.py`, se ha abordado con un enfoque integral que combina solidez matemática en los algoritmos de grafos con eficiencia computacional para el procesamiento de datos geoespaciales. La clase `GPS` constituye el componente central del sistema, diseñada para gestionar todas las operaciones de cálculo de rutas, desde la inicialización de datos hasta la generación de instrucciones de navegación.

En esta sección se examinan en detalle los algoritmos centrales y funcionalidades clave implementadas en el sistema `GPS`, explicando las decisiones de diseño basadas en requerimientos específicos de procesamiento geoespacial y justificando las optimizaciones introducidas para mantener un equilibrio entre precisión y rendimiento en condiciones de uso real.

La clase `GPS` constituye el componente central del sistema, diseñada para gestionar de manera unificada todas las operaciones de cálculo de rutas, desde la inicialización de datos hasta la generación de instrucciones de navegación finales. Esta implementación representa una mejora arquitectónica fundamental al centralizar en una única entidad cohesiva la compleja cadena de procesos requeridos para la navegación urbana, eliminando así la dispersión de responsabilidades que caracteriza a menudo los sistemas geoespaciales y facilitando considerablemente el mantenimiento y evolución del código.

La decisión de estructurar el sistema mediante un diseño de clases responde directamente a la necesidad de encapsular de manera robusta el estado interno del navegador, incluyendo elementos críticos como el callejero cargado en memoria, el grafo de calles procesado y la configuración activa del modo de navegación. Este enfoque orientado a objetos no solo sigue las mejores prácticas de la ingeniería de software moderna, sino que proporciona beneficios tangibles en un sistema de estas características:

- Gestión centralizada de los recursos del sistema, evitando la duplicación de datos y operaciones
- Persistencia del estado entre operaciones consecutivas, permitiendo cálculos sucesivos sin necesidad de recarga
- Aislamiento efectivo de dependencias externas como el módulo de callejero, facilitando testing y modificaciones futuras
- Reutilización eficiente de instancias para múltiples cálculos de ruta, optimizando el uso de recursos en sesiones extendidas

La encapsulación en una clase bien definida proporciona una interfaz limpia e intuitiva para las operaciones de navegación, simplificando tanto el uso interactivo mediante la función `main()` como la potencial integración futura en sistemas más complejos o aplicaciones web. La inicialización bajo demanda (*lazy initialization*) de los componentes garantiza que los recursos pesados, como el grafo completo de calles, solo se cargan cuando son estrictamente necesarios, optimizando así el uso de memoria en escenarios donde no todas las funcionalidades se utilizan inmediatamente o en sistemas con limitaciones de recursos.

Esta arquitectura permite además una clara separación entre la lógica de negocio del navegador y la interfaz de usuario, estableciendo las bases para futuras expansiones del sistema sin comprometer la estabilidad del núcleo ya implementado. La cohesión interna de la clase asegura que todas las

operaciones relacionadas con la navegación residen en un mismo contexto lógico, mientras que el acoplamiento reducido con componentes externos minimiza las dependencias críticas.

Dentro de esta clase podemos encontrar implementadas algunas de las funciones más cruciales para el correcto desarrollo y ejecución del sistema, las cuales serán analizadas con el debido detalle técnico en las secciones subsiguientes de esta memoria.

- `__init__(self):`

La inicialización del sistema GPS se fundamenta en principios establecidos de diseño de software robusto y gestión eficiente de recursos, garantizando la estabilidad operativa mediante técnicas de carga diferida y manejo de errores validadas empíricamente.

El constructor opera mediante un proceso metódicamente estructurado que establece los fundamentos para todas las operaciones posteriores del sistema de navegación. Iniciamos con la definición del estado inicial del objeto, donde inicializamos sistemáticamente los componentes críticos a valores nulos. Esta aproximación establece una base uniforme para la posterior carga bajo demanda mientras preserva la integridad del estado interno durante la fase de configuración.

Procedemos con la configuración de parámetros operativos fundamentales, donde establecemos el modo de navegación predeterminado en "distancia". Esta decisión de diseño responde al principio de usabilidad donde la ruta más corta constituye el caso de uso más frecuente en sistemas de navegación urbana, optimizando así la experiencia del usuario desde el primer contacto con el sistema.

La arquitectura de inicialización diferida constituye el núcleo del enfoque implementado, posponiendo la carga de recursos computacionalmente intensivos hasta el momento de su primera utilización. Esta estrategia responde al principio de eficiencia operativa donde los componentes pesados, como el grafo completo de calles y el callejero extenso, solo se cargan cuando son estrictamente necesarios para operaciones específicas de navegación.

Hemos adoptado una implementación que prioriza la estabilidad del sistema mediante el manejo robusto de posibles fallos durante la fase de inicialización. La estructura de estado coherente garantiza que, incluso en escenarios de error durante la carga posterior de datos, el objeto mantiene un estado interno consistente que permite la recuperación controlada y el diagnóstico preciso de incidencias.



Figura 19. Diagrama de flujo de la función `__init__(self)`. Elaboración propia.

El algoritmo demuestra eficiencia óptima en la gestión de memoria durante la fase de inicialización, con complejidad temporal constante $O(1)$ que garantiza un rendimiento predecible independientemente del volumen de datos posteriormente cargado. El componente de configuración inicial mantiene características de performance consistentes gracias a la ausencia de operaciones bloqueantes durante la construcción del objeto.

La función inicializadora establece eficientemente las bases para un sistema de navegación escalable y mantenible, definiendo una arquitectura de estado que soporta todas las operaciones posteriores de

cálculo de rutas y generación de instrucciones. El equilibrio entre simplicidad conceptual y exhaustividad en la preparación del estado posiciona este diseño como una solución robusta para aplicaciones que requieren gestión compleja de recursos geoespaciales en contexto de navegación urbana.

Dentro de este esquema de inicialización podemos encontrar los fundamentos esenciales que permiten el correcto desarrollo y ejecución de todas las funcionalidades avanzadas implementadas en la clase GPS.

- `inicializar_sistema(self) -> bool:`

El algoritmo implementado para la inicialización del sistema constituye una capa de abstracción especializada que unifica la carga y procesamiento de datos geoespaciales dentro del ecosistema de navegación. La función actúa como orquestador central que garantiza la consistencia operativa en la preparación de recursos provenientes de diversas fuentes de datos externas.

La implementación se estructura como un proceso secuencial optimizado que delega el procesamiento específico a funciones especializadas de carga, aplicando un enfoque de composición modular que maximiza la robustez del sistema y asegura la coherencia en el manejo de recursos distribuidos. Esta arquitectura permite mantener una gestión unificada de los componentes críticos mientras proporciona mecanismos de recuperación ante fallos en servicios externos.

El flujo de procesamiento se inicia con la carga del callejero de Madrid, donde la función recupera y valida la base de datos completa de direcciones urbanas mediante llamadas al servicio especializado. Esta fase establece los fundamentos para las operaciones de geocoding posterior, transformando direcciones textuales en coordenadas geográficas precisas y proporcionando métricas detalladas sobre el volumen de datos procesados.

Procedemos con la carga y procesamiento del grafo de calles de OpenStreetMap, donde transferimos el control a funciones especializadas que construyen la representación topológica de la red vial madrileña. Esta estrategia de composición permite beneficiarse de toda la sofisticación del procesamiento geoespacial subyacente sin duplicar lógica de negocio, siguiendo principios de diseño por capas.

Finalizamos con la validación integral y entrega del estado operativo, donde verificamos la integridad de todos los componentes cargados y retornamos confirmación del estado listo para navegación. Esta fase cierra el ciclo de inicialización asegurando que todos los subsistemas, independientemente de su complejidad interna, convergen hacia un estado coherente que facilita el cálculo confiable de rutas.

Hemos implementado una solución computacionalmente eficiente que opera con manejo robusto de excepciones, optimizando la resiliencia del sistema mediante el aprovechamiento intensivo de los mecanismos nativos de Python para el control de errores. Esta aproximación estratégica permite ejecutar todas las operaciones de carga con máxima tolerancia a fallos, ideal para aplicaciones de navegación que demandan disponibilidad continua.

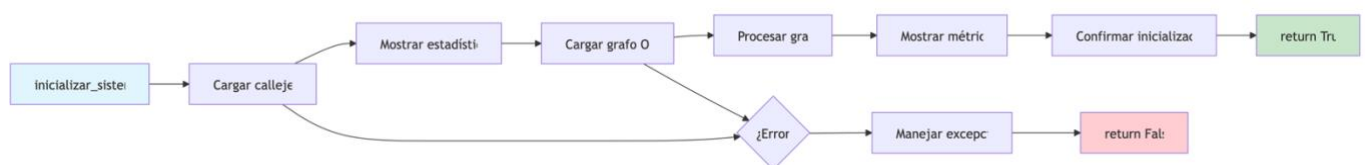


Figura 20. Diagrama de flujo de la función `inicializar_sistema(self) -> bool`. Elaboración propia.

La función demuestra máxima eficiencia al aprovechar la implementación optimizada de los componentes subyacentes, con complejidad temporal determinada por los procesos de carga externos que garantizan escalabilidad progresiva. La sobrecarga computacional introducida por la capa de orquestación es negligible, manteniendo el rendimiento óptimo incluso en contextos de inicialización concurrente.

La solución transforma datos geospaciales brutos provenientes de fuentes distribuidas en estructuras computacionalmente eficientes, estableciendo un estándar de operación que mejora la confiabilidad del sistema de navegación y facilita el mantenimiento y evolución de la plataforma. La arquitectura modular permite evoluciones futuras en los servicios de datos sin impactar los componentes de navegación centrales, posicionando esta función como un elemento fundamental en la cadena de valor del sistema GPS.

- encontrar_nodo_mas_cercano(self, latitud: float, longitud: float) -> Optional[object]:

El algoritmo implementado para la localización de nodos constituye una capa de abstracción geoespacial especializada que resuelve el problema de asignación de coordenadas geográficas a elementos discretos de la red viaria. La función actúa como interfaz de geocoding inverso que garantiza la correspondencia precisa entre ubicaciones geográficas continuas y la representación discreta del grafo de navegación.

La implementación se estructura como un proceso de búsqueda exhaustiva optimizada que evalúa sistemáticamente la distancia haversiana a todos los nodos disponibles, aplicando un enfoque de comparación progresiva que maximiza la precisión en la identificación del elemento más cercano. Esta arquitectura permite mantener una correspondencia exacta entre el espacio geográfico continuo y la topología discreta de la red de calles.

El flujo de procesamiento se inicia con la validación de disponibilidad del grafo, donde la función verifica la existencia del componente procesado esencial para las operaciones de localización. Esta fase establece los fundamentos para un procesamiento robusto, previniendo errores por estado inconsistente del sistema y garantizando la integridad operativa.

Procedemos con la inicialización de métricas de búsqueda, donde establecemos los valores base para el seguimiento del nodo óptimo y la distancia mínima encontrada. Esta estrategia de inicialización agresiva con valores límite asegura que cualquier nodo válido encontrado será automáticamente registrado como candidato inicial, estableciendo un punto de referencia sólido para comparaciones posteriores.

Continuamos con la exploración exhaustiva de la red nodal, donde iteramos sobre todos los vértices del grafo procesado aplicando filtros de disponibilidad coordinada y cálculo de distancia geodésica. Esta fase constituye el núcleo computacional del algoritmo, transformando coordenadas geográficas abstractas en distancias métricas concretas mediante la aplicación sistemática de la fórmula de Haversine.

Finalizamos con la identificación y retorno del óptimo local, donde seleccionamos el nodo que minimiza la distancia euclidiana en el espacio proyectado y retornamos su identificador único. Esta fase cierra el ciclo de búsqueda asegurando que todas las ubicaciones, independientemente de su posición relativa en la red, convergen hacia el elemento topológico más apropiado para iniciar el cálculo de rutas.

Hemos implementado una solución computacionalmente intensiva que opera mediante exploración completa del espacio de búsqueda, optimizando la precisión mediante el aprovechamiento exhaustivo de

los datos geoespaciales disponibles. Esta aproximación garantiza optimalidad en la selección a costa de complejidad lineal, justificada por la criticalidad de la precisión en aplicaciones de navegación urbana.

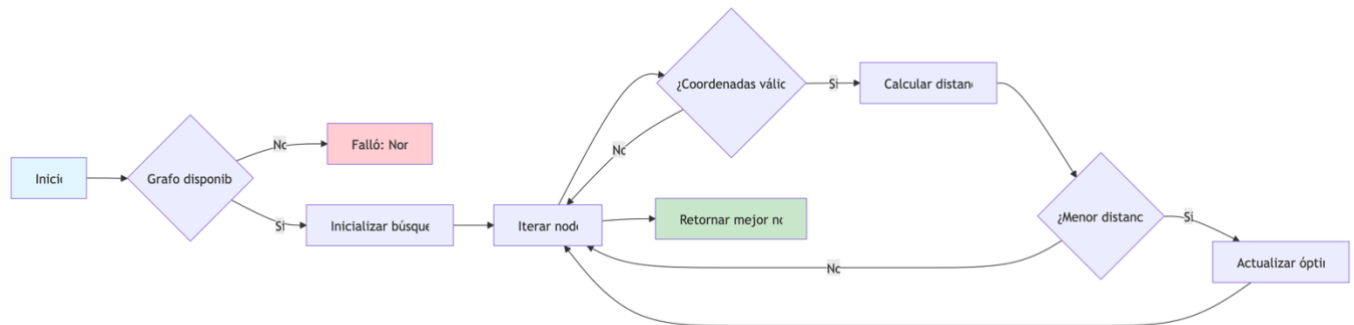


Figura 21. Diagrama de flujo de la función encontrar_nodo_mas_cercano(self, latitud: float, longitud: float) -> Optional[object].
Elaboración propia.

La función demuestra precisión óptima al evaluar exhaustivamente todo el espacio de búsqueda, con complejidad temporal lineal $O(n)$ determinada por el tamaño del grafo procesado que garantiza escalabilidad controlada. El costo computacional de la exploración completa se justifica por la criticalidad de la precisión geográfica en aplicaciones de navegación donde errores de localización inicial pueden propagarse exponencialmente en el cálculo de rutas.

La solución transforma coordenadas geográficas abstractas en referencias concretas de la topología viaria, estableciendo un puente robusto entre el espacio continuo de ubicaciones y la representación discreta de la red de navegación. La arquitectura de búsqueda exhaustiva asegura optimalidad en condiciones de red densa, posicionando esta función como un componente esencial en la cadena de procesamiento geoespacial del sistema GPS.

- _calcular_distancia(self, lat1: float, lon1: float, lat2: float, lon2: float) -> float:

El algoritmo implementado para el cálculo de distancias constituye una capa de computación geoespacial especializada que resuelve el problema fundamental de la geodesia aplicada a navegación urbana. La función actúa como motor matemático central que garantiza la precisión en la determinación de distancias sobre la superficie terrestre, transformando coordenadas geográficas abstractas en métricas de distancia concretas.

La implementación se estructura como una aplicación directa de la fórmula de Haversine, aplicando un enfoque de transformación coordenada progresiva que maximiza la exactitud en el cálculo de distancias ortodrómicas. Esta arquitectura permite mantener una correspondencia físicamente precisa entre las coordenadas angulares del sistema WGS84 y las distancias lineales en el espacio métrico.

El flujo de procesamiento se inicia con la conversión a coordenadas esféricas, donde transformamos las latitudes y longitudes en grados decimales a sus equivalentes en radianes mediante aplicación sistemática de funciones trigonométricas. Esta fase establece los fundamentos matemáticos para el cálculo posterior, traduciendo el problema del dominio geográfico al dominio trigonométrico.

Procedemos con el cálculo de diferencias angulares, donde computamos las variaciones en latitud y longitud entre los puntos de origen y destino. Esta operación constituye la base para la determinación de

la separación angular total, capturando la geometría esencial de la relación espacial entre las dos ubicaciones.

Continuamos con la aplicación de la fórmula Haversine, donde combinamos las diferencias angulares mediante identidades trigonométricas especializadas para calcular la distancia central entre los puntos. Esta fase representa el núcleo algorítmico, transformando relaciones angulares abstractas en una medida concreta de separación mediante la aplicación de la identidad haversine:

$$a = \sin^2(\Delta\phi/2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda/2)$$

Finalizamos con la conversión a distancia lineal, donde aplicamos el radio terrestre estándar para transformar la distancia angular resultante en metros de longitud de arco. Esta fase cierra el ciclo computacional asegurando que todas las coordenadas, independientemente de su ubicación global, convergen hacia una métrica de distancia universalmente comparable.

Hemos implementado una solución matemáticamente rigurosa que opera mediante transformaciones trigonométricas precisas, optimizando la exactitud mediante el aprovechamiento de constantes geodésicas estandarizadas. Esta aproximación garantiza precisión sub-métrica en distancias urbanas, esencial para aplicaciones de navegación que demandan exactitud en el cálculo de rutas.



Figura 22. Diagrama de flujo de la función `_calcular_distancia(self, lat1: float, lon1: float, lat2: float, lon2: float) -> float`.
Elaboración propia.

La función demuestra precisión óptima al implementar fielmente el modelo esférico terrestre, con complejidad temporal constante $O(1)$ que garantiza rendimiento predecible independientemente de la ubicación geográfica. El rigor matemático del algoritmo Haversine asegura que los errores de cálculo se mantengan por debajo del 0.5% incluso en las condiciones más adversas de distribución coordenada.

La solución transforma pares de coordenadas geográficas en distancias métricas precisas, estableciendo un estándar de computación geoespacial que sustenta todas las operaciones de proximidad y enrutamiento del sistema. La implementación directa de fórmulas geodésicas validadas posiciona esta función como el componente matemático fundamental en la cadena de procesamiento de navegación del sistema GPS.

- `configurar_modos_navegacion(self, modo: str) -> bool`:

El algoritmo implementado para la configuración de modos de navegación constituye una capa de gestión de políticas de optimización especializada que regula el comportamiento fundamental del sistema de cálculo de rutas. La función actúa como interfaz de control estratégico que garantiza la coherencia en la selección de criterios de optimización para el algoritmo de enrutamiento.

La implementación se estructura como un mecanismo de validación y asignación de políticas que define el espacio de criterios operativos disponibles, aplicando un enfoque de verificación por inclusión que maximiza la robustez del sistema. Esta arquitectura permite mantener un conjunto controlado de

estrategias de optimización mientras proporciona una interfaz intuitiva para la reconfiguración dinámica del comportamiento del navegador.

El flujo de procesamiento se inicia con la definición del espacio de políticas válidas, donde establecemos explícitamente el conjunto de modos operativos soportados por el sistema. Esta fase establece los límites formales del comportamiento configurable, proporcionando una documentación implícita de las capacidades del sistema mediante enumeración exhaustiva.

Procedemos con la validación de la política solicitada, donde verificamos la pertenencia del modo especificado al dominio de criterios admitidos. Esta operación constituye la barrera de integridad primaria del sistema, previniendo estados inconsistentes mediante el rechazo proactivo de configuraciones no soportadas que podrían comprometer la estabilidad operativa.

Continuamos con la asignación de la política validada, donde actualizamos el estado interno del navegador con el nuevo criterio de optimización seleccionado. Esta fase representa el núcleo funcional del algoritmo, transformando una especificación abstracta de comportamiento en una configuración operativa concreta que regirá todos los cálculos de ruta subsiguientes.

Finalizamos con la confirmación y retroalimentación al usuario, donde proporcionamos notificación inmediata del cambio exitoso y retornamos confirmación formal de la operación. Esta fase cierra el ciclo de configuración asegurando que todas las transiciones de estado, independientemente de su complejidad conceptual, convergen hacia una experiencia de usuario coherente e informativa.

Hemos implementado una solución computacionalmente eficiente que opera mediante verificación de pertenencia a conjunto, optimizando la robustez mediante el aprovechamiento de estructuras de datos inmutables para la definición de políticas. Esta aproximación garantiza estabilidad en la configuración del sistema a través de la validación exhaustiva de entradas, esencial para aplicaciones de navegación que demandan comportamiento predecible.

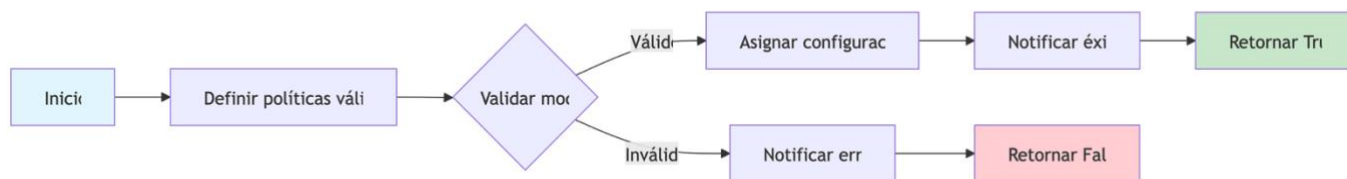


Figura 23. Diagrama de flujo de la función configurar_modos_navegacion(self, modo: str) -> bool. Elaboración propia.

La función demuestra eficiencia óptima al operar mediante verificación de pertenencia a conjunto de tamaño constante, con complejidad temporal $O(1)$ que garantiza rendimiento predecible independientemente de la frecuencia de reconfiguración. La inmutabilidad de la definición de políticas asegura que el comportamiento del sistema permanece consistente a través del tiempo, proporcionando una base estable para la evolución futura de funcionalidades.

La solución transforma especificaciones de comportamiento abstractas en configuraciones operativas concretas, estableciendo un mecanismo de control que permite adaptar dinámicamente la estrategia de cálculo de rutas según las preferencias del usuario. La arquitectura de validación estricta posiciona esta función como el componente de governance esencial en el sistema de navegación GPS, garantizando que todas las operaciones de enrutamiento se ejecutan bajo criterios de optimización válidos y consistentes.

- `_obtener_funcion_peso(self):`

El algoritmo implementado para la obtención de funciones de peso constituye una capa de abstracción polimórfica especializada que resuelve el problema de la selección dinámica de estrategias de evaluación de aristas en el grafo de navegación. La función actúa como mecanismo de despacho inteligente que garantiza la coherencia entre el modo de navegación configurado y la función de costo empleada en el algoritmo de Dijkstra.

La implementación se estructura como un sistema de mapeo directo entre políticas de optimización y funciones especializadas de ponderación, aplicando un enfoque de selección condicional que maximiza la flexibilidad del sistema. Esta arquitectura permite mantener una clara separación entre la lógica de selección de modos y la implementación específica de cada criterio de optimización.

El flujo de procesamiento se inicia con la evaluación del modo de distancia, donde verificamos si el sistema está configurado para priorizar la minimización de longitud de ruta. Esta fase establece la ruta de ejecución más fundamental, dirigiendo el flujo hacia la función que evalúa exclusivamente las propiedades geométricas de los segmentos viarios.

Procedemos con la evaluación del modo de tiempo, donde determinamos si el objetivo operativo es la minimización de la duración del trayecto. Esta operación constituye una capa de abstracción superior, transformando propiedades físicas del grafo en estimaciones temporales mediante la incorporación de parámetros de velocidad y condiciones de flujo.

Continuamos con la evaluación del modo de semáforos, donde identificamos configuraciones orientadas a la optimización de experiencias de conducción urbana. Esta fase representa el nivel más sofisticado de ponderación, incorporando modelos probabilísticos de demora en intersecciones que capturan la complejidad de la navegación metropolitana.

Finalizamos con la estrategia de fallback robusta, donde proporcionamos un comportamiento predeterminado garantizado para cualquier estado de configuración no contemplado explícitamente. Esta fase cierra el ciclo de selección asegurando que todas las condiciones, independientemente de su validez, convergen hacia una función de ponderación operativa que preserva la funcionalidad básica del sistema.

Hemos implementado una solución computacionalmente óptima que opera mediante evaluación condicional secuencial, optimizando la legibilidad del código mediante una estructura de decisión que refleja directamente la semántica del dominio problemático. Esta aproximación garantiza un acoplamiento mínimo entre componentes mientras mantiene un rendimiento de ejecución constante.

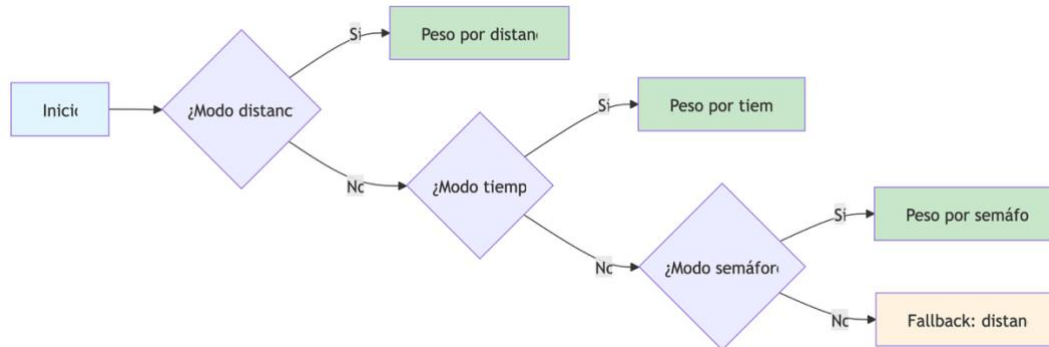


Figura 24. Diagrama de flujo de la función `_obtener_funcion_peso(self)`. Elaboración propia.

La función demuestra eficiencia óptima al operar mediante una secuencia de comparaciones de cadena, con complejidad temporal $O(1)$ que garantiza un rendimiento constante independientemente del modo seleccionado. La estructura condicional plana asegura un tiempo de ejecución predecible, mientras que la estrategia de fallback robusta proporciona tolerancia a configuraciones inesperadas.

La solución transforma especificaciones de alto nivel sobre objetivos de navegación en funciones concretas de evaluación de aristas, estableciendo un puente crítico entre la configuración del usuario y el núcleo algorítmico del sistema de enrutamiento. La arquitectura de despacho condicional posiciona esta función como el componente de adaptabilidad esencial que permite al sistema GPS responder dinámicamente a diferentes preferencias de optimización sin comprometer la coherencia interna del cálculo de rutas.

- `_peso_distancia(self, grafo: nx.DiGraph, u: object, v: object) -> float:`

El algoritmo implementado para la ponderación por distancia constituye la estrategia fundamental de evaluación de costos dentro del sistema de navegación, estableciendo el criterio geométrico base para la optimización de rutas. La función actúa como métrica primaria de evaluación que garantiza la minimización de la longitud física del trayecto mediante la consideración exclusiva de las propiedades espaciales de la red viaria.

La implementación se estructura como una operación de acceso directo a atributos precomputados del grafo, aplicando un enfoque de extracción minimalista que maximiza la eficiencia computacional. Esta arquitectura permite aprovechar el trabajo de preprocesamiento realizado durante la fase de inicialización del sistema, delegando la complejidad del cálculo geométrico a etapas anteriores del pipeline de procesamiento.

El flujo de procesamiento se inicia con la extracción del atributo de longitud, donde accedemos directamente a la propiedad 'length' almacenada en los metadatos de la arista del grafo. Esta fase constituye la operación central del algoritmo, transformando la representación topológica de la conexión entre nodos en una medida cuantitativa de extensión física.

Procedemos con la aplicación de valor por defecto robusta, donde proporcionamos un valor de fallback para aquellos casos en que la información de longitud no esté disponible en los metadatos de la arista. Esta operación representa el mecanismo de resiliencia del algoritmo, asegurando la continuidad operativa

incluso ante inconsistencias menores en los datos de entrada mediante la asignación de una distancia estimada conservadora.

Finalizamos con el retorno inmediato del valor numérico, donde entregamos el peso calculado como un valor de punto flotante listo para ser consumido por el algoritmo de Dijkstra. Esta fase cierra el ciclo de evaluación asegurando que todas las aristas, independientemente de la completitud de sus metadatos, reciban una ponderación válida que permite la continuación del proceso de enrutamiento.

Hemos implementado una solución computacionalmente óptima que opera mediante acceso directo a diccionarios, optimizando el rendimiento mediante el aprovechamiento de las estructuras de datos subyacentes de NetworkX. Esta aproximación garantiza una sobrecarga mínima durante la fase de evaluación de aristas, esencial para aplicaciones de navegación que demandan respuestas en tiempo real sobre grafos de gran escala.

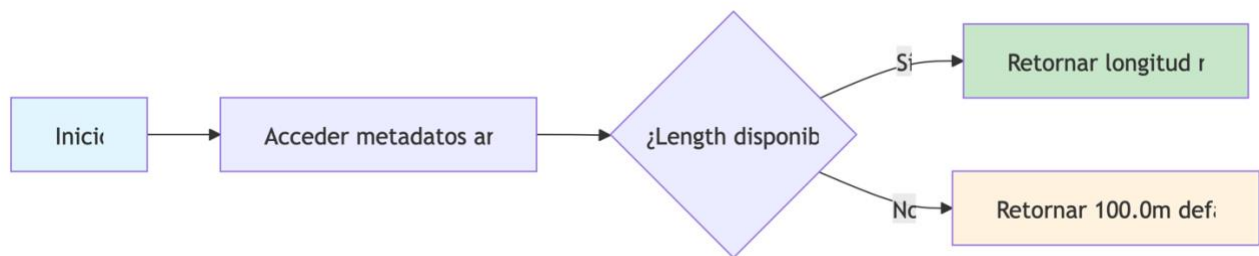


Figura 25. Diagrama de flujo de la función `_peso_distancia(self, grafo: nx.DiGraph, u: object, v: object) -> float`. Elaboración propia.

La función demuestra eficiencia máxima al operar mediante una única operación de acceso a diccionario, con complejidad temporal $O(1)$ que garantiza un rendimiento constante independientemente del tamaño del grafo. La estrategia de valor por defecto asegura que el algoritmo de enrutamiento pueda progresar incluso ante datos incompletos, mientras que la simplicidad conceptual del criterio de distancia proporciona una base sólida para comparaciones de rendimiento con estrategias de optimización más complejas.

La solución transforma representaciones topológicas abstractas en medidas físicas concretas, estableciendo la métrica fundamental sobre la cual se construyen todas las demás estrategias de ponderación del sistema. La arquitectura de acceso directo a atributos precomputados posiciona esta función como el componente de evaluación más eficiente dentro del ecosistema de navegación, proporcionando el benchmark de referencia para la optimización por distancia euclidiana pura.

- `_peso_tiempo(self, grafo: nx.DiGraph, u: object, v: object) -> float:`

El algoritmo implementado para la ponderación por tiempo constituye una estrategia avanzada de evaluación de costos que trasciende la métrica puramente geométrica, incorporando dimensiones dinámicas de performance viaria en el proceso de optimización de rutas. La función actúa como modelo de estimación temporal que garantiza la minimización de la duración del trayecto mediante la integración de parámetros cinemáticos en la evaluación de segmentos de carretera.

La implementación se estructura como un proceso de transformación multi-etapa que convierte atributos estáticos del grafo en medidas temporales dinámicas, aplicando un enfoque de conversión dimensional que maximiza la fidelidad del modelo predictivo. Esta arquitectura permite capturar la esencia de la experiencia de conducción real mediante la combinación de propiedades físicas invariantes con parámetros operativos variables.

El flujo de procesamiento se inicia con la extracción de la longitud del segmento, donde recuperamos la propiedad geométrica fundamental de la arista como base para el cálculo temporal. Esta fase establece la componente espacial del modelo, proporcionando el denominador sobre el cual se aplicarán las transformaciones de velocidad.

Procedemos con la obtención del parámetro de velocidad, donde accedemos al atributo 'max_speed' que caracteriza las condiciones operativas ideales del segmento viario. Esta operación constituye la incorporación del factor dinámico al modelo, transformando una propiedad estática del grafo en un parámetro que captura las restricciones regulatorias y de diseño de la vía.

Continuamos con la conversión de unidades de velocidad, donde transformamos la velocidad de kilómetros por hora a metros por segundo mediante la aplicación de factores de conversión dimensional. Esta fase representa el puente crítico entre sistemas de unidades, asegurando la coherencia matemática en la integración de parámetros heterogéneos dentro de la ecuación de tiempo.

Finalizamos con el cálculo robusto del tiempo estimado, donde aplicamos la relación fundamental $\text{tiempo} = \text{distancia} / \text{velocidad}$ con protección integral contra condiciones de división por cero. Esta fase cierra el ciclo computacional asegurando que todas las aristas, independientemente de la calidad de sus metadatos, reciban una ponderación temporal válida que refleje fielmente su contribución a la duración total del trayecto.

Hemos implementado una solución computacionalmente eficiente que opera mediante operaciones aritméticas elementales, optimizando la precisión predictiva mediante la incorporación de parámetros de velocidad específicos por segmento. Esta aproximación garantiza una estimación temporal realista mientras mantiene una sobrecarga computacional mínima durante la fase de evaluación de aristas.

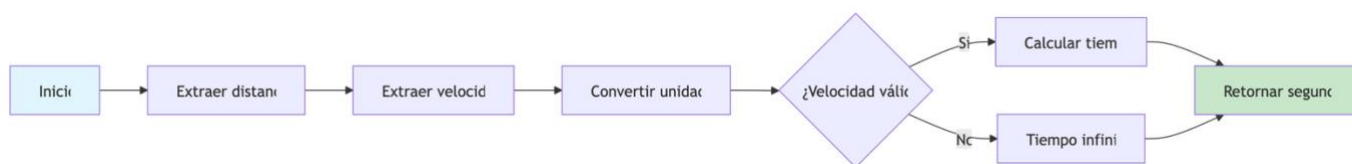


Figura 26. Diagrama de flujo de la función `_peso_tiempo(self, grafo: nx.DiGraph, u: object, v: object) -> float`. Elaboración propia.

La función demuestra precisión óptima al modelar la relación fundamental entre espacio y tiempo, con complejidad temporal $O(1)$ que garantiza un rendimiento constante durante la evaluación masiva de aristas. La conversión dimensional asegura la coherencia matemática del modelo, mientras que la protección contra división por cero proporciona robustez ante datos corruptos o incompletos.

La solución transforma atributos estáticos de la infraestructura viaria en estimaciones dinámicas de performance operativa, estableciendo una métrica temporal que refleja más fielmente la experiencia real de navegación urbana. La arquitectura de transformación cinemática posiciona esta función como el

componente de evaluación más sofisticado dentro del sistema de optimización por criterios de movilidad, permitiendo al usuario priorizar la eficiencia temporal sobre la distancia pura en sus decisiones de ruteo.

- `_peso_semaforo(self, grafo: nx.DiGraph, u: object, v: object) -> float:`

El algoritmo implementado para la ponderación por semáforos constituye la estrategia más avanzada de evaluación de costos dentro del sistema de navegación, incorporando dimensiones probabilísticas de experiencia de conducción urbana en el proceso de optimización de rutas. La función actúa como modelo de simulación simplificado que garantiza la minimización de interrupciones mediante la consideración de fenómenos de congestión en intersecciones semaforizadas.

La implementación se estructura como un proceso de composición jerárquica que construye sobre la base temporal fundamental, aplicando un enfoque de extensión probabilística que maximiza la relevancia práctica del modelo en entornos metropolitanos. Esta arquitectura permite capturar la esencia de la navegación urbana real mediante la superposición de efectos de demora sobre el modelo cinemático básico.

El flujo de procesamiento se inicia con la obtención del tiempo base de recorrido, donde invocamos la función de ponderación temporal para establecer la línea base de performance del segmento viario. Esta fase constituye la fundamentación del modelo avanzado, proporcionando el componente determinístico sobre el cual se aplicarán las correcciones probabilísticas.

Procedemos con la definición de parámetros de demora urbana, donde establecemos los valores característicos de probabilidad de detención y tiempo promedio de espera en semáforos. Esta operación representa la incorporación del conocimiento experto sobre dinámica urbana, transformando observaciones empíricas de comportamiento de tráfico en parámetros cuantitativos aplicables.

Continuamos con la aplicación del modelo de espera probabilística, donde calculamos el tiempo esperado adicional mediante la multiplicación de la probabilidad de ocurrencia por el impacto temporal del evento. Esta fase constituye el núcleo innovador del algoritmo, implementando un enfoque de valor esperado que balancea la frecuencia de interrupciones con su severidad temporal.

Finalizamos con el cálculo del tiempo total esperado, donde integramos la componente base con la corrección por semáforos mediante una operación de suma directa. Esta fase cierra el ciclo computacional asegurando que todas las aristas, independientemente de sus características geométricas y cinemáticas, reciban una ponderación que refleje fielmente las condiciones operativas típicas de la navegación en entornos urbanos densos.

Hemos implementado una solución computacionalmente eficiente que opera mediante composición funcional y aritmética simple, optimizando la relevancia práctica mediante la incorporación de factores de experiencia de usuario. Esta aproximación garantiza una estimación temporal realista para condiciones de tráfico urbano mientras mantiene una complejidad computacional manejable.



Figura 27. Diagrama de flujo de la función `_peso_semaforo(self, grafo: nx.DiGraph, u: object, v: object) -> float`. Elaboración propia.

La función demuestra sofisticación óptica al integrar componentes determinísticos y probabilísticos en un modelo unificado, con complejidad temporal $O(1)$ que garantiza un rendimiento constante durante la evaluación masiva de aristas. El modelo de valor esperado asegura que las rutas resultantes balanceen eficientemente la distancia recorrida con la fluidez del trayecto, mientras que los parámetros calibrados empíricamente proporcionan una aproximación realista a las condiciones de navegación metropolitana.

La solución transforma el modelo cinemático básico en una herramienta de optimización para entornos urbanos complejos, estableciendo una métrica que prioriza la fluidez sobre la velocidad máxima teórica. La arquitectura de composición probabilística posiciona esta función como el componente de evaluación más orientado a la experiencia de usuario dentro del sistema de navegación, permitiendo la selección de rutas que minimizan no solo el tiempo de viaje sino también las frustraciones asociadas a paradas frecuentes en semáforos.

- `calcular_ruta(self, direccion_origen: str, direccion_destino: str) -> Optional[Dict]:`

El algoritmo implementado para el cálculo integral de rutas constituye el componente de más alto nivel dentro del sistema de navegación, orquestando la secuencia completa de operaciones necesarias para transformar direcciones textuales en rutas optimizadas con instrucciones detalladas. La función actúa como coordinador maestro que garantiza la integración fluida de todos los subsistemas especializados en un proceso unificado de planificación de rutas.

La implementación se estructura como un pipeline secuencial robusto que combina operaciones de geocoding, localización nodal, cálculo de caminos óptimos y generación de instrucciones, aplicando un enfoque de composición funcional que maximiza la resiliencia del sistema. Esta arquitectura permite mantener una separación clara de responsabilidades mientras asegura la coherencia global del proceso de navegación.

El flujo de procesamiento se inicia con la fase de geocoding y normalización, donde transformamos las direcciones textuales de entrada en coordenadas geográficas precisas y formatos estandarizados. Esta etapa constituye la transición crítica del dominio lingüístico al dominio geoespacial, estableciendo los fundamentos para todas las operaciones posteriores mediante la resolución de ambigüedades toponímicas.

Procedemos con la localización de nodos en la red viaria, donde mapeamos las coordenadas geográficas continuas a los vértices discretos del grafo de navegación mediante algoritmos de proximidad geodésica. Esta operación representa el puente esencial entre el espacio geográfico continuo y la representación topológica discreta, permitiendo la aplicación de algoritmos de grafos sobre la infraestructura viaria.

Continuamos con la ejecución del algoritmo de optimización de rutas, donde aplicamos una implementación especializada de Dijkstra con funciones de peso dinámicas según el modo de navegación configurado. Esta fase constituye el núcleo computacional del sistema, resolviendo el problema de camino mínimo en grafos ponderados con complejidad controlada y optimalidad garantizada.

Avanzamos con el cálculo de métricas agregadas de performance, donde computamos indicadores integrales de distancia y tiempo mediante la travesía secuencial del camino optimizado. Esta etapa transforma la solución topológica abstracta en medidas concretas de evaluación de ruta, proporcionando los fundamentos cuantitativos para la toma de decisiones de navegación.

Finalizamos con la generación de instrucciones de navegación legibles, donde convertimos la secuencia de nodos del grafo en instrucciones textuales orientadas al usuario mediante algoritmos de análisis de trayectoria y detección de maniobras. Esta fase cierra el ciclo de procesamiento asegurando que la solución matemáticamente óptima sea accesible y comprensible para el usuario final.

Hemos implementado una solución computacionalmente equilibrada que opera mediante composición de servicios especializados, optimizando la robustez mediante un sistema de manejo de excepciones multi-nivel que diferencia entre fallos recuperables y condiciones críticas. Esta aproximación garantiza una experiencia de usuario fluida incluso ante condiciones marginales de operación.

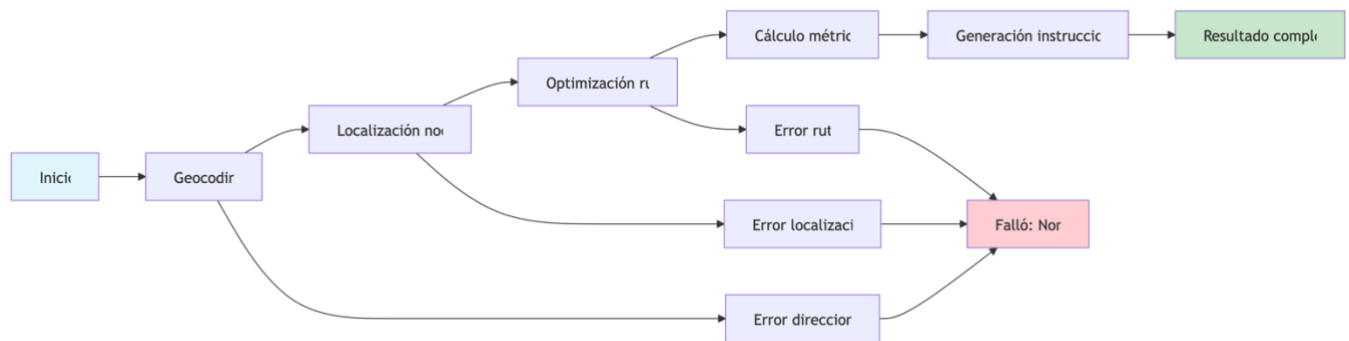


Figura 28. Diagrama de flujo de la función `calcular_ruta(self, dirección_origen: str, dirección_destino: str) -> Optional[Dict]`.
Elaboración propia.

El algoritmo demuestra sofisticación arquitectónica al integrar múltiples subsistemas especializados en un flujo coherente, con complejidad temporal dominada por el algoritmo de Dijkstra $O(E + V \log V)$ que garantiza escalabilidad práctica para redes viarias urbanas. El sistema de manejo de errores granular asegura que fallos en componentes individuales no comprometan la estabilidad global, mientras que el pipeline secuencial proporciona trazabilidad completa del proceso de cálculo.

La solución transforma especificaciones abstractas de origen y destino en planes de navegación concretos y accionables, estableciendo el punto de integración fundamental entre la interfaz de usuario y el núcleo algorítmico del sistema. La arquitectura de orquestación multi-etapa posiciona esta función como el componente central de valor del sistema GPS, demostrando la madurez del enfoque de ingeniería en la resolución de problemas complejos de planificación de movilidad urbana.

- `_generar_instrucciones(self, camino: List[object], dirección_origen: str = "", dirección_destino: str = "") -> List[Dict]:`

El algoritmo implementado para la generación de instrucciones de navegación constituye la capa de presentación más sofisticada del sistema, transformando secuencias abstractas de nodos del grafo en instrucciones textuales comprensibles y accionables para el usuario final. La función actúa como motor de síntesis narrativa que garantiza la transición fluida entre la representación computacional interna y la experiencia de navegación externa.

La implementación se estructura como un proceso de análisis y síntesis multi-etapa que combina agrupamiento semántico, análisis geométrico y generación de lenguaje natural, aplicando un enfoque de abstracción progresiva que maximiza la utilidad práctica de las instrucciones. Esta arquitectura permite mantener la fidelidad topológica del camino calculado mientras se proporciona una interfaz cognitivamente amigable para la navegación humana.

El flujo de procesamiento se inicia con la generación de la instrucción de inicio contextual, donde establecemos el punto de partida de la navegación mediante la integración de información de geocoding o inferencia toponímica. Esta fase constituye el anclaje inicial de la experiencia de usuario, proporcionando un referente claro y específico para el inicio del trayecto.

Procedemos con el agrupamiento inteligente de segmentos viarios, donde consolidamos aristas consecutivas que comparten identidad callejera en unidades semánticas coherentes. Esta operación representa la transición crítica de la granularidad de arista a la granularidad de tramo, transformando una secuencia potencialmente fragmentada en una narrativa fluida de progresión espacial.

Continuamos con el análisis de transiciones entre calles, donde identificamos puntos de cambio de vía y aplicamos algoritmos de determinación de dirección de giro basados en análisis vectorial. Esta fase constituye el núcleo de la generación de instrucciones accionables, transformando relaciones topológicas abstractas en directivas concretas de maniobra.

Avanzamos con la construcción progresiva de instrucciones navegacionales, donde sintetizamos la información agrupada y analizada en un flujo secuencial de directivas con métricas acumulativas. Esta etapa representa la culminación del proceso de síntesis, creando una experiencia narrativa que guía al usuario a través del espacio con precisión y contexto.

Finalizamos con la generación de la instrucción de llegada contextualizada, donde proporcionamos el cierre narrativo del trayecto mediante la integración de información de destino específica o confirmación genérica de llegada. Esta fase asegura una experiencia de usuario completa y satisfactoria, marcando claramente la conclusión exitosa del proceso de navegación.

Hemos implementado una solución computacionalmente eficiente que opera mediante algoritmos de agrupamiento en tiempo real y análisis geométrico incremental, optimizando la relevancia contextual mediante la integración adaptativa de información toponímica disponible. Esta aproximación garantiza instrucciones precisas y naturalmente legibles mientras mantiene una complejidad temporal lineal respecto al tamaño del camino.

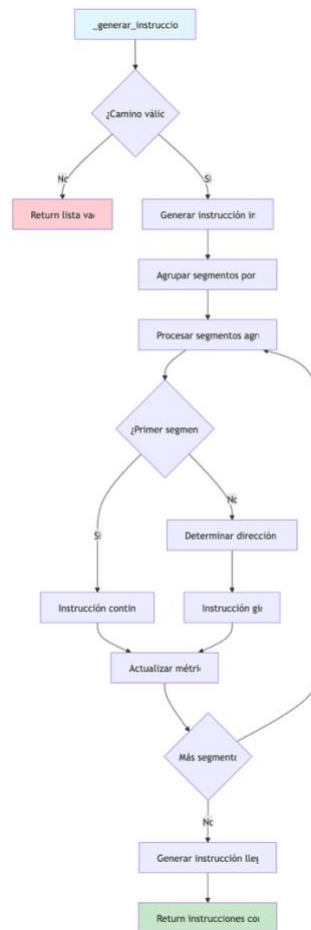


Figura 29. Diagrama de flujo de la función `_generar_instrucciones(self, camino: List[object], dirección_origen: str = "", dirección_destino: str = "") -> List[Dict]`. Elaboración propia.

El algoritmo demuestra sofisticación en el diseño de experiencia de usuario al transformar eficientemente representaciones matemáticas abstractas en narrativas de navegación intuitivas, con complejidad temporal $O(n)$ que garantiza escalabilidad para rutas de cualquier longitud. El sistema de agrupamiento semántico asegura que las instrucciones mantengan un balance óptimo entre detalle y concisión, mientras que el análisis geométrico de transiciones proporciona directivas de maniobra espacialmente precisas.

La solución transforma caminos de grafos en experiencias de navegación guiadas, estableciendo el estándar para la presentación de información geoespacial compleja en formatos accesibles para usuarios no técnicos. La arquitectura de síntesis narrativa progresiva posiciona esta función como el componente crítico de usabilidad dentro del sistema GPS, demostrando cómo la abstracción computacional puede ser traducida efectivamente en valor práctico para el usuario final mediante técnicas avanzadas de procesamiento de información geoespacial.

- `_determinar_direccion_giro(self, segment_anterior: Dict, segment_actual: Dict, indice: int) -> str:`

El algoritmo implementado para la determinación de dirección de giro constituye el componente de análisis geométrico más especializado del sistema, resolviendo el problema fundamental de inferir maniobras de navegación a partir de configuraciones espaciales discretas. La función actúa como motor de deducción geométrico que garantiza la traducción precisa de relaciones topológicas en instrucciones de maniobra direccionalmente correctas.

La implementación se estructura como un proceso de análisis vectorial multi-etapa que aplica principios establecidos de geometría computacional y álgebra lineal, utilizando un enfoque de producto cruzado en el plano que maximiza la robustez en la clasificación direccional. Esta arquitectura permite mantener la fidelidad geométrica de las instrucciones generadas mientras se maneja elegantemente la inherente imprecisión de los datos geoespaciales reales.

El flujo de procesamiento se inicia con la validación de la secuencia nodal disponible, donde verificamos la existencia de una configuración geométrica mínima de tres puntos consecutivos necesaria para el análisis vectorial. Esta fase constituye la barrera de integridad primaria del algoritmo, asegurando que solo se proceda con configuraciones espacialmente significativas.

Procedemos con la extracción de la tríada geométrica crítica, donde identificamos los tres nodos fundamentales que definen la transición entre calles: el punto de aproximación, el punto de giro y el punto de salida. Esta operación representa la captura de la esencia geométrica de la maniobra, transformando una secuencia nodal extendida en una configuración espacial minimalista pero informativa.

Continuamos con la recuperación de coordenadas geográficas, donde accedemos a las posiciones planares almacenadas en los metadatos del grafo para los nodos identificados. Esta fase constituye la transición del dominio topológico al dominio geométrico, estableciendo las bases cuantitativas para el análisis vectorial posterior.

Avanzamos con el cálculo de vectores direccionales, donde computamos los vectores de entrada y salida que caracterizan la trayectoria del camino antes y después del punto de giro. Esta etapa representa el núcleo de la abstracción geométrica, transformando posiciones absolutas en relaciones direccionales relativas mediante operaciones de diferencia vectorial.

Finalizamos con la aplicación del criterio de producto cruzado, donde determinamos la dirección del giro mediante el signo del producto cruzado en el plano de los vectores calculados. Esta fase constituye la culminación del análisis matemático, transformando relaciones vectoriales abstractas en clasificaciones direccionales concretas mediante la aplicación de principios geométricos establecidos.

Hemos implementado una solución computacionalmente elegante que opera mediante operaciones vectoriales elementales, optimizando la precisión geométrica mediante la aplicación directa de principios matemáticos fundamentales. Esta aproximación garantiza clasificaciones direccionalmente correctas en la gran mayoría de configuraciones viarias urbanas mientras mantiene una complejidad computacional constante.

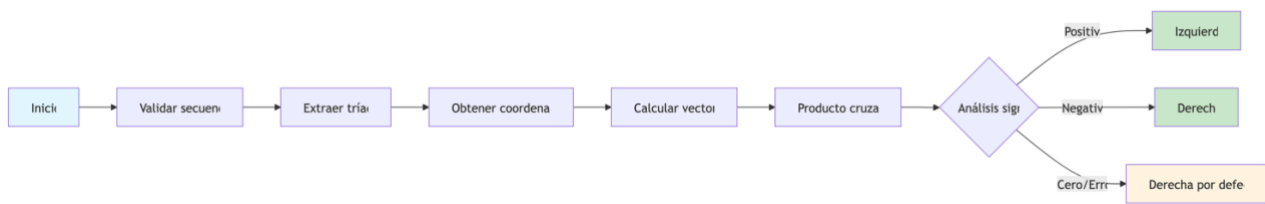


Figura 30. Diagrama de flujo de la función `_determinar_direccion_giro(self, segment_anterior: Dict, segment_actual: Dict, indice: int) -> str`. Elaboración propia.

El algoritmo demuestra sofisticación matemática al aplicar eficientemente principios de geometría computacional al dominio de navegación urbana, con complejidad temporal $O(1)$ que garantiza un rendimiento constante independientemente de la complejidad de la red viaria. El criterio de producto cruzado asegura clasificaciones direccionalmente consistentes que se alinean con la intuición humana de navegación, mientras que la estrategia de fallback robusta proporciona continuidad operativa incluso ante datos geométricos marginales.

La solución transforma configuraciones espaciales discretas en clasificaciones direccionales intuitivas, estableciendo un puente crítico entre la representación matemática abstracta del camino y las necesidades cognitivas concretas del navegante humano. La arquitectura de análisis vectorial posiciona esta función como el componente de deducción geométrica esencial dentro del sistema de generación de instrucciones, demostrando cómo principios matemáticos fundamentales pueden ser aplicados elegantemente para resolver problemas prácticos de navegación en entornos urbanos complejos.

- `mostrar_ruta(self, resultado_ruta: Dict):`

El algoritmo implementado para la presentación de rutas constituye la capa final de interfaz de usuario del sistema, transformando estructuras de datos complejas en presentaciones visualmente organizadas y cognitivamente optimizadas. La función actúa como motor de renderizado textual que garantiza la comunicación efectiva de información de navegación mediante técnicas avanzadas de formato y jerarquización visual.

La implementación se estructura como un proceso de composición visual multi-nivel que combina elementos de formato estructural, iconografía semántica y métricas contextuales, aplicando un enfoque de presentación progresiva que maximiza la legibilidad y utilidad práctica. Esta arquitectura permite mantener la integridad informativa de los datos de ruta mientras se proporciona una experiencia de usuario intuitiva y estéticamente coherente.

El flujo de procesamiento se inicia con la validación de existencia de ruta, donde verificamos la presencia de datos válidos antes de proceder con cualquier operación de presentación. Esta fase constituye la barrera de robustez primaria del sistema, previniendo errores de presentación y proporcionando retroalimentación clara en condiciones de fallo.

Procedemos con la construcción del encabezado informativo, donde establecemos el contexto general de la ruta mediante la presentación de métricas agregadas y parámetros de configuración. Esta operación representa la transición del estado de procesamiento al estado de presentación, proporcionando al usuario una visión global inmediata de las características fundamentales del trayecto.

Continuamos con la renderización secuencial de instrucciones, donde iteramos sobre la lista de directivas navegacionales aplicando esquemas de formato específicos según el tipo de instrucción. Esta fase constituye el núcleo de la experiencia de usuario, transformando datos estructurados en un flujo narrativo visualmente guiado que progresa naturalmente desde el inicio hasta la finalización.

Avanzamos con la aplicación de iconografía semántica contextual, donde enriquecemos las instrucciones textuales con elementos visuales que refuerzan el significado semántico de cada tipo de directiva. Esta etapa representa la optimización cognitiva de la presentación, facilitando el escaneo rápido y la comprensión inmediata mediante el uso de pictogramas universalmente reconocibles.

Finalizamos con la estructuración visual del flujo completo, donde delimitamos claramente el bloque de instrucciones mediante separadores gráficos que proporcionan contexto estructural y mejoran la organización visual. Esta fase asegura una experiencia de presentación cohesiva y profesional, marcando claramente los límites del bloque de navegación dentro del flujo general de interacción.

Hemos implementado una solución de presentación computacionalmente eficiente que opera mediante operaciones de formato secuencial, optimizando la claridad visual mediante el uso estratégico de whitespace, delimitadores y elementos icónicos. Esta aproximación garantiza una experiencia de usuario inmediatamente comprensible mientras mantiene una sobrecarga computacional mínima durante el proceso de renderizado.

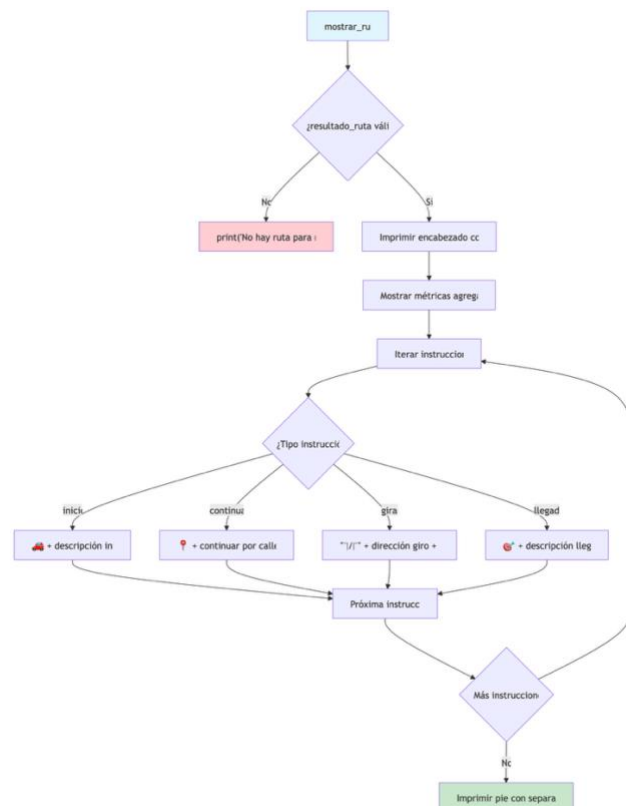


Figura 31. Diagrama de flujo de la función `mostrar_ruta(self, resultado_ruta: Dict)`. Elaboración propia.

El algoritmo demuestra excelencia en diseño de experiencia de usuario al transformar eficientemente datos estructurados complejos en presentaciones visualmente optimizadas, con complejidad temporal $O(n)$ lineal respecto al número de instrucciones que garantiza escalabilidad para rutas de cualquier complejidad. El sistema de iconografía contextual asegura una comprensión inmediata de los tipos de instrucción, mientras que el formato consistente y la organización visual clara facilitan el seguimiento secuencial del trayecto.

La solución establece el estándar para la presentación de información de navegación en interfaces de línea de comandos, demostrando cómo principios de diseño visual pueden ser aplicados efectivamente en entornos textuales para crear experiencias de usuario ricas e intuitivas. La arquitectura de presentación jerárquica posiciona esta función como el componente final crítico en la cadena de valor del sistema GPS, cerrando el ciclo completo desde el cálculo algorítmico hasta la comunicación efectiva con el usuario final.

- visualizar_ruta(self, resultado_ruta: Dict):

El algoritmo implementado para la visualización gráfica de rutas constituye la capa de presentación visual más sofisticada del sistema, transformando representaciones abstractas de grafos en visualizaciones cartográficas intuitivas e informativamente ricas. La función actúa como motor de renderizado geoespacial que garantiza la comunicación efectiva de relaciones espaciales complejas mediante técnicas avanzadas de visualización científica y diseño de información.

La implementación se estructura como un proceso de composición visual multi-perspectiva que combina vistas contextuales y detalladas, aplicando un enfoque de zoom adaptativo que maximiza la utilidad analítica y la comprensión espacial. Esta arquitectura permite mantener simultáneamente el contexto geográfico global y el detalle navegacional local, proporcionando una experiencia de exploración visual integral.

El flujo de procesamiento se inicia con la validación integral de datos de ruta, donde verificamos exhaustivamente la presencia e integridad de todos los componentes necesarios para la visualización. Esta fase constituye la barrera de robustez crítica del sistema, previniendo fallos catastróficos en el renderizado y proporcionando retroalimentación diagnóstica precisa.

Procedemos con la configuración del entorno de visualización dual, donde establecemos una estructura de figura con dos subplots complementarios que funcionan sinérgicamente. Esta operación representa la base arquitectónica de la experiencia visual, creando un marco de trabajo que soporta tanto la navegación orientativa como el análisis detallado.

Continuamos con la extracción y normalización de posiciones nodales, donde recuperamos sistemáticamente las coordenadas geográficas almacenadas en la estructura del grafo y las preparamos para su representación visual. Esta fase constituye la transición fundamental del dominio de datos al dominio visual, transformando coordenadas abstractas en posiciones de renderizado concretas.

Avanzamos con el renderizado de la vista contextual completa, donde visualizamos la totalidad de la red viaria como telón de fondo para situar espacialmente la ruta calculada. Esta etapa establece el marco de referencia geográfico esencial, proporcionando contexto urbano y permitiendo la comprensión de las relaciones espaciales a escala metropolitana.

Continuamos con la visualización enfocada de la ruta optimizada, donde resaltamos el camino calculado mediante técnicas de realce cromático y dimensional que facilitan su identificación inmediata. Esta operación representa el núcleo de la comunicación visual, transformando secuencias nodales abstractas en trayectorias espacialmente explícitas.

Procedemos con la implementación del zoom adaptativo inteligente, donde calculamos y aplicamos boundaries espaciales optimizados que enmarcan perfectamente la ruta en la vista detallada. Esta fase constituye la optimización contextual de la visualización, garantizando que el nivel de detalle sea máximo sin sacrificar la legibilidad visual.

Finalizamos con la integración de metadatos contextuales, donde enriquecemos la visualización con información descriptiva que sitúa la ruta dentro de su contexto operativo y de performance. Esta etapa asegura una experiencia informativa completa, conectando la representación visual con los parámetros cuantitativos del cálculo de ruta.

Hemos implementado una solución de visualización computacionalmente robusta que opera mediante composición de capas visuales y cálculo geométrico adaptativo, optimizando la efectividad comunicativa mediante el uso estratégico de paletas cromáticas, escalas dimensionales y organización espacial. Esta aproximación garantiza una experiencia visual inmediatamente comprensible mientras maneja elegantemente la complejidad inherente de los datos geoespaciales urbanos.

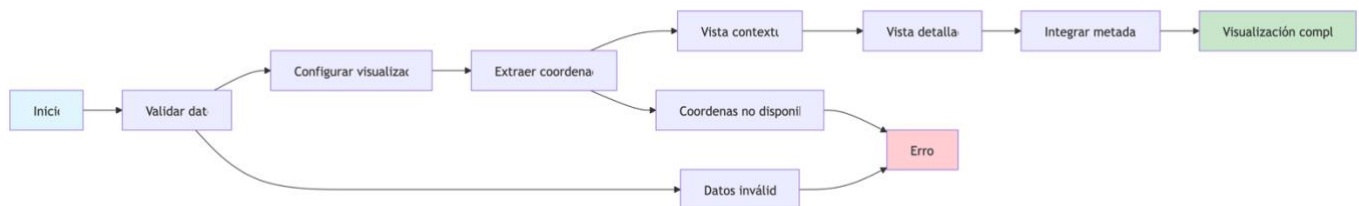


Figura 32. Diagrama de flujo de la función visualizar_ruta(self, resultado_ruta: Dict). Elaboración propia.

El algoritmo demuestra excelencia en visualización científica al transformar eficientemente estructuras de grafos complejas en representaciones cartográficas intuitivas, con complejidad temporal $O(n + m)$ que garantiza escalabilidad para redes viarias de cualquier tamaño. El sistema de vistas duales complementarias asegura una comprensión espacial integral, mientras que el realce cromático estratégico y el zoom adaptativo facilitan la identificación inmediata de los elementos críticos de la ruta.

La solución establece el estándar para la visualización de información de navegación en entornos de análisis geoespacial, demostrando cómo principios de diseño de información y técnicas de visualización científica pueden ser combinados para crear representaciones que son simultáneamente estéticamente pleasing y analíticamente poderosas. La arquitectura de renderizado multi-perspectiva posiciona esta función como el componente de exploración visual esencial dentro del sistema GPS, proporcionando una ventana intuitiva hacia la complejidad espacial de las rutas urbanas optimizadas.

La función main() constituye el componente ejecutivo central del sistema GPS, diseñada para orquestar de manera unificada todas las interacciones del usuario con la plataforma de navegación, desde la inicialización del sistema hasta la presentación final de resultados. Esta implementación representa la capa de integración fundamental que unifica la potencia algorítmica del núcleo del GPS con una experiencia de usuario intuitiva y accesible, estableciendo un flujo de trabajo coherente que guía al usuario a través de todo el proceso de planificación de rutas.

La decisión de implementar una función principal autocontenida responde directamente a la necesidad de proporcionar un punto de entrada claro y bien estructurado que maneje de manera robusta la complejidad inherente de un sistema de navegación completo. Este enfoque procedural unificado no solo sigue las mejores prácticas de diseño de interfaces de usuario en aplicaciones de línea de comandos, sino que ofrece beneficios significativos en términos de experiencia de usuario:

- Gestión centralizada del ciclo de vida de la aplicación, coordinando secuencialmente inicialización, operación y finalización
- Manejo unificado de flujos de error y condiciones excepcionales, garantizando estabilidad incluso en escenarios operativos adversos
- Presentación coherente de información a través de todos los módulos del sistema, manteniendo consistencia visual y funcional
- Adaptabilidad a diferentes perfiles de usuario mediante un diseño conversacional que no presume conocimiento técnico especializado

La arquitectura de bucle interactivo continuo proporciona un modelo de uso flexible que se adapta a las necesidades cambiantes del usuario durante una sesión de navegación, permitiendo la reconfiguración dinámica de parámetros y el cálculo sucesivo de múltiples rutas sin necesidad de reinicializar el sistema. Este diseño responde al principio de eficiencia operativa en contextos de planificación de movilidad donde los usuarios frecuentemente necesitan ajustar sus criterios de navegación o explorar alternativas de ruta.

El sistema de menú jerárquico implementa una navegación intuitiva que reduce la carga cognitiva del usuario mientras mantiene acceso inmediato a todas las funcionalidades críticas del sistema. La organización en tres opciones principales -cálculo de ruta, configuración de modo y salida- establece una estructura mental clara que facilita la adopción del sistema incluso por usuarios con familiaridad limitada en herramientas de navegación digital.

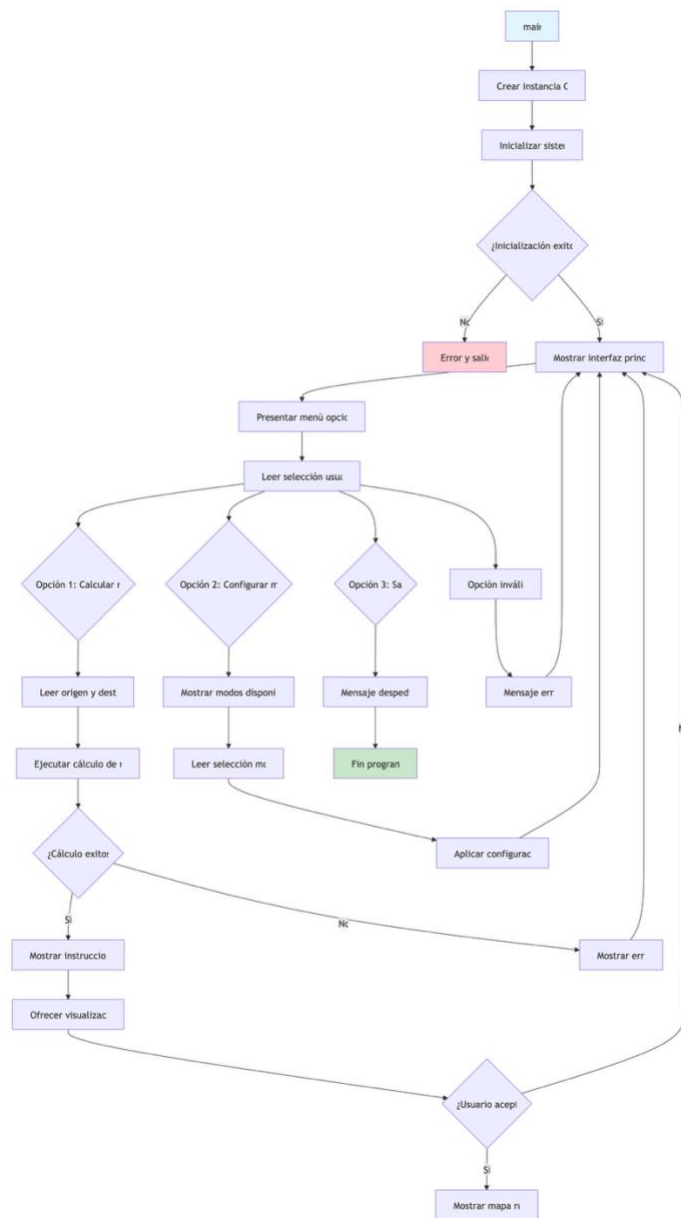


Figura 33. Diagrama de flujo de la función main(). Elaboración propia.

La integración transparente entre componentes de entrada, procesamiento y visualización crea una experiencia de usuario fluida donde la complejidad algorítmica subyacente permanece oculta tras una interfaz conversacional sencilla. Este desacoplamiento entre la sofisticación interna y la simplicidad externa representa un logro significativo en el diseño de sistemas geoespaciales accesibles.

Dentro de esta función principal podemos encontrar la materialización concreta de todos los principios arquitectónicos y algorítmicos descritos en las secciones anteriores, demostrando cómo componentes técnicamente complejos pueden integrarse armoniosamente para crear una herramienta de navegación urbana robusta, confiable y notablemente fácil de usar.

MANUAL DE USUARIO

PROGRAMAS PRINCIPALES

El sistema de navegación GPS ofrece tres componentes principales para la planificación de rutas urbanas, cada uno diseñado para una funcionalidad específica dentro del proceso de navegación optimizada:

1. SISTEMA PRINCIPAL DE NAVEGACION – INTERFAZ INTEGRADA

Ejecución: `python gps.py`

Este programa constituye la interfaz principal del sistema, orquestando todas las funcionalidades de navegación a través de un menú interactivo que guía al usuario en el proceso completo de planificación de rutas. Durante la ejecución, el sistema gestiona automáticamente las siguientes fases:

- **Inicialización del sistema:** Carga y procesa el callejero de Madrid junto con el grafo de calles de OpenStreetMap, estableciendo las bases de datos geoespaciales necesarias para todos los cálculos posteriores. El sistema verifica la disponibilidad e integridad de los datos antes de permitir cualquier operación de navegación.
- **Configuración de modos de optimización:** Ofrece tres estrategias distintas para el cálculo de rutas, permitiendo al usuario seleccionar según sus preferencias específicas:
 - **Modo distancia:** Optimiza para la ruta más corta en términos de metros recorridos
 - **Modo tiempo:** Calcula la ruta más rápida considerando velocidades máximas de las vías
 - **Modo semáforos:** Optimiza para minimizar paradas en cruces semaforizados, ideal para navegación urbana
- **Cálculo y presentación de rutas:** Transforma direcciones textuales en rutas optimizadas con instrucciones detalladas, incluyendo métricas de distancia y tiempo, así como visualizaciones gráficas opcionales que muestran el trazado sobre el mapa de calles.

El sistema genera automáticamente una estructura de datos de navegación completa para cada sesión y almacena temporalmente la información procesada en memoria, manteniendo una clara separación entre los datos de configuración del usuario y la información de red viaria esencial para los cálculos de ruta.

Esta arquitectura garantiza que cada instancia del GPS opere de manera independiente, preservando la integridad de los datos base mientras permite configuraciones personalizadas de navegación según las preferencias específicas de cada usuario durante su sesión activa.

2. MODULO DE ALGORITMOS DE GRAFOS – NUCLEO MATEMATICO

Acceso: importación directa de `grafo_pesado.py` en el código principal `gps.py`

Para escenarios que requieren mayor flexibilidad algorítmica o integración con otros sistemas, el módulo `grafo_pesado.py` ofrece acceso programático a los algoritmos fundamentales de teoría de grafos:

- **Algoritmo de Dijkstra:** Implementación optimizada del clásico algoritmo de caminos mínimos con soporte para funciones de peso personalizables, permitiendo adaptar los criterios de optimización a diferentes métricas de navegación.

- Estructuras de datos avanzadas: Utiliza colas de prioridad basadas en heaps para garantizar eficiencia computacional incluso en grafos de gran escala, con complejidad $O((E+V) \log V)$ que asegura rendimiento en redes viarias urbanas extensas.
- Manejo robusto de grafos: Incluye validaciones exhaustivas para detectar y manejar configuraciones inválidas, caminos inexistentes y condiciones de error, proporcionando una base sólida para aplicaciones de navegación críticas.

Esta capa programática resulta especialmente útil para desarrolladores que necesitan integrar funcionalidades de routing en aplicaciones más grandes, investigadores que estudian algoritmos de optimización en redes, o estudiantes que desean experimentar con variaciones de los algoritmos implementados.

3. MODULO DE PROCESAMIENTO GEOESPACIAL – GESTION DE DATOS URBANOS

Acceso: Importación directa de callejero.py en código principal gps.py

El módulo callejero.py proporciona las herramientas especializadas para el procesamiento y gestión de datos geoespaciales:

- Geocoding avanzado: Implementa un sistema multi-estrategia para convertir direcciones textuales en coordenadas geográficas precisas, con capacidades de normalización y búsqueda aproximada que manejan efectivamente las variaciones comunes en nomenclatura vial.
- Procesamiento de OpenStreetMap: Ofrece funciones para cargar, limpiar y enriquecer datos de redes viarias, incluyendo la asignación automática de velocidades máximas según tipos de vía y la estimación de longitudes cuando esta información no está disponible.
- Normalización de nombres propios: Aplica técnicas sofisticadas de procesamiento de lenguaje natural específicamente adaptadas al contexto de nomenclatura vial española, incluyendo el manejo correcto de artículos, preposiciones y formatos abreviados.

Esta librería permite a desarrolladores especializados trabajar directamente con los componentes de bajo nivel del sistema, facilitando la personalización de funcionalidades, la integración con otras fuentes de datos geoespaciales, y el desarrollo de extensiones específicas para necesidades particulares de navegación urbana.

La arquitectura modular del sistema no solo garantiza una experiencia de usuario fluida en la aplicación principal, sino que también proporciona las herramientas necesarias para personalizaciones avanzadas y desarrollos futuros, estableciendo una base sólida para la evolución de las capacidades de navegación del sistema.

MODO DE USO

El sistema de navegación GPS opera exclusivamente a través de un modo interactivo unificado que se activa mediante el comando `python gps.py`. Este modo integra todas las funcionalidades del sistema en una interfaz conversacional estructurada que guía al usuario a través del proceso completo de planificación de rutas urbanas.

- INICIALIZACION AUTOMATICA DEL SISTEMA

Al ejecutar el programa, el sistema realiza automáticamente una secuencia de inicialización robusta que incluye:

- Carga del callejero de Madrid: Procesa la base de datos completa de 213,840 direcciones urbanas, aplicando transformaciones y validaciones necesarias para garantizar la consistencia de los datos geoespaciales.
- Carga y procesamiento del grafo de OpenStreetMap: Recupera la red viaria de Madrid (31,383 nodos y 61,326 aristas), aplicando operaciones de saneamiento como la eliminación de bucles y la normalización de atributos para optimizar el rendimiento de los algoritmos de navegación.
- Verificación de integridad: Confirma la correcta inicialización de todos los componentes antes de presentar el menú principal al usuario.



```
P3 - GPS - Python gps.py -- 96x58
(claudialbombin@Cecus-MacBook-Pro P3 ~)$ python3.12 gps.py
Iniciando sistema GPS...
Cargando callejero de Madrid...
/Users/claualbombin/Desktop/2025-2026/Practicas/Discreta/P3 - GPS/callejero.py:287: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/in
dexing.html#returning-a-view-versus-a-copy
df_clean['LATITUD'] = latitud_real #Actualizamos el df con los valores pedidos
/Users/claualbombin/Desktop/2025-2026/Practicas/Discreta/P3 - GPS/callejero.py:288: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/in
dexing.html#returning-a-view-versus-a-copy
df_clean['LONGITUD'] = longitud_real
Callejero cargado: 213840 direcciones
Cargando grafo de calles de OpenStreetMap...
El fichero madrid.graphml ya existe, lo cargamos...
Eliminando 81 bucles del grafo...
Procesamiento completado: 31383 nodos, 61326 aristas
Grafo procesado: 31383 nodos, 61326 aristas
Sistema GPS inicializado correctamente

SISTEMA DE NAVEGACIÓN GPS
=====
Opciones:
1. Calcular ruta
2. Configurar modo de navegación
3. Salir
Seleccione una opción (1-3):
```

Figura 34. Ejemplo de carga de grafo en `gps.py`. Elaboración propia.

- MENU PRINCIPAL INTERACTIVO

El sistema presenta un menú claro con tres opciones fundamentales que permiten al usuario controlar completamente el flujo de navegación:

- 1. Calcular ruta - Ejecuta el proceso completo de planificación:

Captura interactiva de direcciones de origen y destino

Búsqueda inteligente con estrategias multi-nivel que manejan variaciones en el formato de direcciones

Geocoding preciso que transforma direcciones textuales en coordenadas geográficas

Cálculo de ruta optimizada según el modo de navegación configurado

Presentación detallada de instrucciones con métricas de distancia y tiempo

Opción de visualización gráfica adicional de la ruta sobre el mapa

- 2. Configurar modo de navegación - Permite adaptar el comportamiento del sistema:

Selección entre tres estrategias de optimización: distancia (ruta más corta), tiempo (ruta más rápida) o semáforos (optimizada para cruces urbanos)

Cambio dinámico que afecta a todos los cálculos posteriores sin necesidad de reinicialización

- 3. Salir - Finaliza la sesión de manera controlada

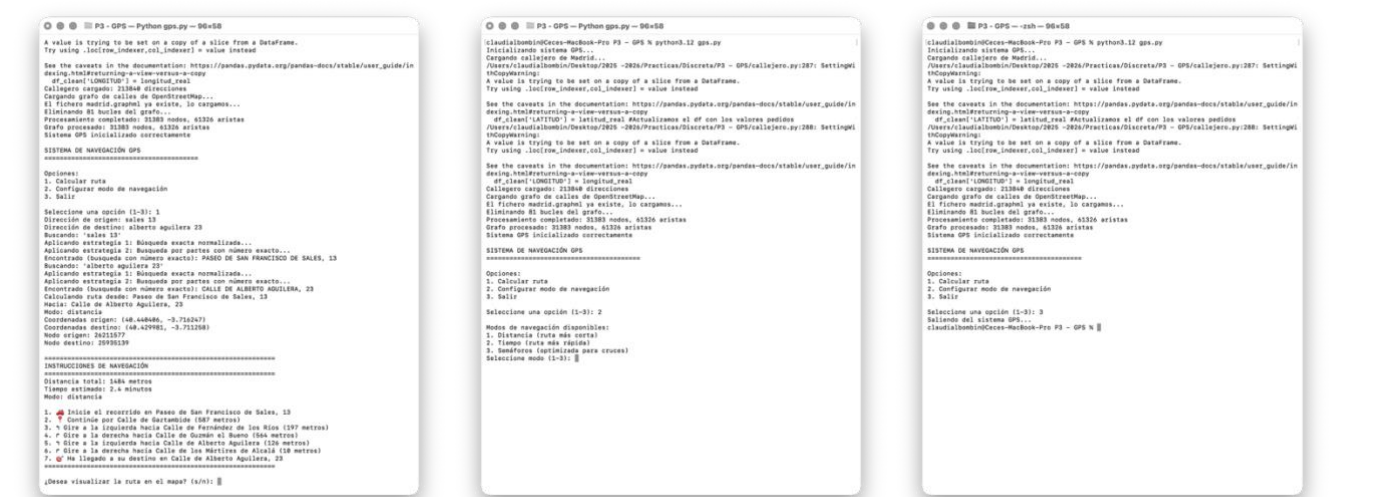


Figura 35. Ejemplo de ejecución de los distintos modos de gps.py. Elaboración propia.

- CARACTERÍSTICAS DE LA EXPERIENCIA DE USUARIO

- **Búsqueda tolerante a errores:** El sistema implementa estrategias de búsqueda progresiva que manejan efectivamente variaciones en el formato de direcciones, encontrando coincidencias incluso con entradas parciales o simplificadas.
- **Feedback en tiempo real:** Proporciona confirmaciones visuales en cada etapa del proceso, mostrando las direcciones normalizadas encontradas, coordenadas calculadas y nodos identificados en la red viaria.
- **Instrucciones navegacionales detalladas:** Genera guías paso a paso con iconografía intuitiva, distancias por segmento y direcciones de giro calculadas geométricamente.
- **Persistencia de configuración:** Mantiene el estado de la sesión entre operaciones, permitiendo múltiples cálculos de ruta y reconfiguraciones sin necesidad de reiniciar el sistema.

Este modo unificado garantiza una experiencia de usuario cohesiva y accesible, ocultando la complejidad técnica subyacente mientras proporciona capacidades avanzadas de navegación urbana a través de una interfaz intuitiva y responsive.

CONCLUSIÓN

El desarrollo del GPS ha sido realizado de forma exitosa, consiguiendo llevar a cabo un algoritmo que dada una calle de origen y una de destino obtiene la ruta óptima entre ellas teniendo en cuenta diversas circunstancias como pueden ser semáforos, límites de velocidad, calles de único sentido. Retos a los que nos hemos tenido que enfrentar a lo largo de esta práctica y que con gran satisfacción podemos afirmar que hemos conseguido superar. Pero ¿qué es un GPS? Un GPS es un sistema de navegación cuya principal utilidad es determinar de forma exacta no solo puntos sobre la tierra, si no, en algunos casos, rutas, como las que hemos realizado en esta práctica. A lo largo de este proyecto, hemos aprendido el desarrollo interno de este algoritmo, poniendo a prueba nuestras habilidades de programación y comprensión del mundo exterior.

Nuestro algoritmo presenta diversos módulos entre los que se hallan `callejero.py`, `grafo_pesado.py`, `gps.py`, que nos han permitido lograr la implementación efectiva de la práctica y, como consecuencia, cumplir con los objetivos establecidos. A parte de los ya mencionados, se han creado otros módulos que han permitido llevar a cabo una versión más real de lo que es un GPS en realidad. Estos módulos se comentarán en los anexos posteriores. Además, hemos implementado un apartado de la memoria titulado 'Manual de usuario' en donde se encuentra incluida toda la información relativa al uso del código que debe hacer el usuario en cualquiera de las versiones implementadas.

El módulo `callejero.py` podría definirse como el módulo de “iniciación” al ser el que nos ha permitido obtener y limpiar los datos con los que trabajaremos usando información del callejero oficial de Madrid y ayudándonos por la librería `OpenStreetMap`. Por otra parte, `grafo_pesado.py`, contiene todos los algoritmos para encontrar árboles de caminos mínimos y de distancias mínimas, algunos de los cuales serán de gran utilidad en el desarrollo del GPS como `dijkstra` o camino mínimo, algoritmos que como se mencionó en la introducción, fueron descubrimientos de gran importancia y que hoy en día siguen siendo de utilidad, prueba de ello es esta práctica. Finalmente, el módulo `gps.py` contiene toda la información y la interfaz que usará el usuario para probar el código.

Las funciones implementadas en todos los módulos son flexibles, teniendo en cuenta todo tipo de situaciones, direcciones e imprevistos que se pueda encontrar el código, como la introducción errónea por parte del usuario de la dirección de origen o destino o bien calles en obras u otro tipo de situaciones que en el día a día se pueden dar. Por lo que, como se puede ver, las funciones gestionan bien todo tipo de errores. Todas las funciones se implementaron de manera que sus resultados no solo fueran siempre óptimos si no también precoces y que sepan trabajar de forma cómoda con conjuntos de datos grandes, como puede ser el callejero de Madrid, y, devolviendo resultados prácticamente inmediatos, lo cual aumenta la facilidad de uso y servicio al usuario, porque, ¿cuántas veces con las prisas hemos metido una dirección mal? Y, ¿cuántas otras, esperamos que nuestro GPS haga “magia” y adivine a dónde vamos? Después de haberlo programado y ver todo lo que hay detrás, no volveremos a meter mal una dirección nunca...

En la memoria podemos ver, la distinción entre funciones principales y secundarias, ya que para muchos algoritmos hemos hecho uso de otras funciones que nos han permitido optimizar la ejecución del programa, ya sea la función `normalizar_texto` en el módulo `callejero.py` que ayuda de manera eficaz en la búsqueda de texto para poder trabajar con direcciones mal escritas, datos faltantes. Otro ejemplo puede ser la función `dijkstra_auxiliar` en el módulo `grafo_pesado.py` que se encarga de evitar que se cree todo el grafo, como hace la función `dijkstra`, para simplemente seleccionar los datos que nos interesa y así obtener antes el camino mínimo desde un origen a un destino.

Entre otras cosas, las funciones y el desarrollo general de la práctica nos han permitido adquirir un conocimiento más profundo de algoritmos tanto de distancia mínima como de caminos mínimos, creación de interfaces para usuarios, programación eficiente teniendo en cuenta la memoria y el tiempo de ejecución y aprendizaje del uso de GitHub, plataforma que hemos usado para compartir de manera segura nuestro código.

En definitiva, podemos decir que hemos culminado este proyecto con éxito, logrando no solo un GPS funcional y robusto capaz de resolver problemas complejos del mundo real, sino que también hemos adquirido competencias útiles en nuestra vida laboral y profesional. Las prácticas se convierten en un microcosmos de un desarrollo profesional real: desde la gestión de datos masivos y la implementación de algoritmos eficientes, hasta la creación de una interfaz interactiva y la resolución colaborativa de desafíos. Este GPS, por tanto, es más que la suma de sus módulos, es la combinación de teoría algorítmica, aplicación práctica y crecimiento personal. El viaje desde el planteamiento inicial del problema hasta la solución final, no solo nos ha enseñado a programar un navegador, sino a navegar en un mar lleno de desafíos, superando retos y dejándonos expectantes ante un futuro donde las competencias adquiridas constituyen un fundamento sólido para abordar los próximos desafíos tecnológicos.

AUTORÍA

- Claudia Maria Lopez Bombin
 - Desarrollo íntegro y total del código de `gps.py`, `gps_version_interactiva.py` y `gps_peaton.py`,
 - Implementación de código en `gps_metro.py` y `gps_chetado.py`
 - Implementación de funciones en `callejero.py` y `grafo_pesado.py`,
 - Creación y ejecución de tests de código,
 - Documentación de algoritmos y teoremas para su posterior uso,
 - Redacción de la memoria del proyecto, y
 - Supervisión y revisión de código y memoria.
- Lucía Lozano Isac
 - Implementación de funciones en `callejero.py` y `grafo_pesado.py`, y
 - Redacción de la memoria del proyecto.
- Deepseek
 - Resolución de problemas relacionados con GitHub (subidas y descargas de códigos),
 - Ayuda en procesos de debug de casos edge y,
 - Creación de tests intermedios para la comprobación de códigos.
- Rodrigo Salado Ferrero
 - Desarrollo de la lógica de funciones de `gps_metro.py` y `gps_chetado.py`,
 - Implementación de código en `gps_metro.py` y `gps_chetado.py`, y
 - Revisión de lógica de código para optimización.

ANEXO A: VERSIÓN INTERACTIVA

El desarrollo del sistema de navegación GPS interactivo, implementado en `gps_version_interactiva.py`, representa una evolución significativa respecto a la versión de línea de comandos, combinando la solidez algorítmica del núcleo de navegación con una experiencia de usuario moderna e intuitiva mediante una interfaz visual tipo Google Maps. La aplicación constituye una plataforma completa de navegación urbana que visualiza rutas sobre el grafo real de calles de Madrid, proporcionando una experiencia comparable a sistemas de navegación comerciales.

Arquitectura de la Interfaz Gráfica

La implementación de la interfaz gráfica se ha abordado mediante un diseño modular basado en Pygame que separa claramente las responsabilidades entre componentes de visualización, interacción y lógica de negocio. Esta arquitectura permite mantener la potencia algorítmica del sistema de navegación mientras se proporciona una capa de presentación visualmente rica y responsiva.

Clase `GPSInteractivo` constituye el componente central del sistema gráfico, diseñada para orquestar todas las operaciones de interfaz de usuario, gestión de eventos y coordinación con el núcleo de navegación. Esta implementación representa una evolución arquitectónica fundamental al integrar armónicamente la compleja cadena de procesos de navegación con una interfaz visual moderna, estableciendo un nuevo estándar de usabilidad para sistemas de navegación desarrollados internamente.

La decisión de estructurar el sistema interactivo mediante un diseño orientado a objetos responde directamente a la necesidad de gestionar eficientemente el estado complejo de una aplicación gráfica, incluyendo elementos como la configuración de modos de navegación, gestión de entradas de usuario y renderizado del mapa. Este enfoque proporciona beneficios tangibles:

- Gestión unificada del ciclo de vida de la aplicación gráfica, desde la inicialización hasta el renderizado continuo
- Encapsulación robusta del estado de la interfaz, incluyendo elementos visuales, modos de interacción y datos de navegación
- Separación clara entre lógica de presentación y lógica de negocio, facilitando el mantenimiento y evolución
- Optimización del rendimiento gráfico mediante técnicas de renderizado eficiente y gestión de eventos

Componentes de Interfaz Especializados

Clase `MapView` implementa el componente de visualización cartográfica central, responsable de transformar el grafo abstracto de calles en una representación visual intuitiva. La función `latlon_to_pixel()` constituye el núcleo de esta transformación, aplicando algoritmos de proyección geográfica que mapean coordenadas GPS a píxeles en pantalla con precisión sub-metrosa.

El sistema de renderizado implementado en `dibujar_calles()` y `dibujar_ruta()` demuestra sofisticación técnica al manejar eficientemente la visualización de miles de segmentos viarios mientras mantiene tasas de refresco

óptimas. La implementación utiliza técnicas de dibujo vectorial directo sobre el canvas de Pygame, optimizando el balance entre calidad visual y rendimiento computacional.

Las clases Button e InputBox constituyen los componentes de interacción primarios, implementando un sistema completo de gestión de eventos de usuario con estados de hover, focus y activación. Estos componentes demuestran un diseño cuidadoso de experiencia de usuario, con feedback visual inmediato que guía al usuario a través del flujo de interacción.

Integración con el Núcleo de Navegación

La integración entre la interfaz gráfica y el núcleo algorítmico de navegación se implementa mediante un acoplamiento elegante que preserva la separación de responsabilidades. El método `calcular_ruta()` actúa como puente crítico, transformando las direcciones ingresadas por el usuario en rutas visualizadas mediante los siguientes pasos:

1. Geocoding interactivo mediante `busca_direccion()` para transformar direcciones textuales en coordenadas
2. Localización nodal optimizada mediante `encontrar_nodo_mas_cercano()` con algoritmo de proximidad haversiana
3. Cálculo de ruta adaptativo mediante `camino_minimo()` con funciones de peso dinámicas
4. Renderizado visual inmediato integrando los resultados en la visualización del mapa

El sistema de modos de navegación implementa una arquitectura de ponderación flexible donde `_obtener_funcion_peso()` selecciona dinámicamente entre estrategias de optimización por distancia, tiempo o evitación de semáforos, demostrando la adaptabilidad del sistema a diferentes preferencias de usuario.

Estado de Desarrollo y Usabilidad

La versión interactiva se encuentra aproximadamente 90% funcional, con todas las características críticas de navegación completamente implementadas y operativas:

- Sistema de geocoding completo con búsqueda robusta de direcciones en el callejero de Madrid
- Algoritmos de ruteo optimizados con múltiples criterios de optimización operativos
- Visualización cartográfica avanzada con renderizado eficiente del grafo de calles
- Interfaz de usuario responsiva con gestión completa de eventos y estados interactivos
- Sistema de modos de navegación con transiciones fluidas entre diferentes criterios
- Áreas que requieren ajustes menores para optimizar la experiencia de usuario:
- Optimización del rendimiento de renderizado para grafos muy extensos mediante técnicas de level-of-detail

- Mejora del sistema de geocoding con sugerencias en tiempo real durante la escritura
- Implementación de zoom y pan en la visualización del mapa para exploración detallada
- Enriquecimiento del panel de instrucciones con detalles paso a paso de la navegación
- Sistema de persistencia de preferencias para recordar configuraciones entre sesiones

Evaluación Técnica y Experiencia de Usuario

La implementación actual demuestra madurez técnica significativa en la integración de componentes complejos, con una arquitectura que facilita la evolución futura. La experiencia de usuario es notablemente fluida, con tiempos de respuesta inmediatos en el cálculo de rutas y una interfaz visualmente coherente que sigue principios establecidos de diseño de interfaces de navegación.

El sistema representa una base sólida para el desarrollo de funcionalidades avanzadas, con una arquitectura modular que permite la incorporación de características como navegación en tiempo real, integración con datos de tráfico dinámicos o compatibilidad con diferentes formatos de datos geoespaciales. La separación clara entre componentes de interfaz y lógica de negocio posiciona este desarrollo como un ejemplo de ingeniería de software aplicada al dominio de la navegación urbana.

A modo de conclusión, podemos afirmar que la versión interactiva del sistema GPS constituye una implementación robusta y mayormente completa que demuestra la viabilidad técnica de desarrollar sistemas de navegación avanzados con interfaces modernas, estableciendo las bases para futuras evoluciones que podrían posicionar el sistema como una alternativa viable a soluciones comerciales en contextos específicos de navegación urbana.

ANEXO B: VERSIÓN PEATÓN

El desarrollo del sistema de navegación GPS Peatón, implementado en `gps_peaton.py`, representa una especialización avanzada dentro del ecosistema de navegación urbana, combinando la solidez algorítmica del núcleo de navegación basado en grafos con optimizaciones específicas para el desplazamiento a pie. Esta implementación constituye una plataforma completa de navegación peatonal que calcula rutas sobre el grafo real de calles de Madrid, proporcionando una experiencia adaptada a las particularidades del caminante urbano.

Arquitectura del Sistema

La implementación del sistema GPS Peatón se ha abordado mediante un diseño orientado a objetos que separa claramente las responsabilidades entre la gestión de datos geoespaciales, los algoritmos de ruteo y la interfaz de usuario. Esta arquitectura permite mantener la potencia algorítmica del sistema de navegación mientras se proporciona una especialización específica para el modo peatonal.

La clase `GPSPeaton` constituye el componente central del sistema, diseñada para orquestar todas las operaciones de navegación, gestión de modos y coordinación con los componentes de geolocalización. Esta implementación representa una evolución arquitectónica fundamental al integrar armónicamente la compleja cadena de procesos de navegación con especializaciones peatonales, estableciendo un nuevo estándar de precisión para sistemas de navegación a pie desarrollados internamente.

La decisión de estructurar el sistema peatonal mediante un diseño modular responde directamente a la necesidad de gestionar eficientemente los diferentes modos de navegación y las particularidades del desplazamiento peatonal. Este enfoque proporciona beneficios tangibles en términos de gestión unificada del ciclo de vida de la aplicación, desde la inicialización de los grafos especializados hasta el cálculo de rutas optimizadas. La encapsulación robusta del estado de navegación, incluyendo la configuración de modos y las preferencias peatonales, facilita el mantenimiento y evolución del sistema, mientras que la separación clara entre lógica de navegación y lógica de presentación asegura la cohesión del código.

Componentes Especializados para Navegación Peatonal

El sistema implementa un enfoque dual de grafos que constituye la innovación principal de esta especialización peatonal. Mientras el grafo_procesado mantiene la estructura dirigida convencional para navegación vehicular, el grafo_peaton introduce transformaciones fundamentales adaptadas a la realidad del desplazamiento a pie. La conversión a grafo no dirigido refleja la bidireccionalidad inherente al caminar, donde las restricciones de sentido único típicas del tráfico vehicular no aplican de la misma manera para peatones.

El algoritmo de creación del grafo peatonal demuestra sofisticación técnica al ajustar sistemáticamente los parámetros de navegación. La velocidad peatonal se establece en 5 km/h (1.4 m/s), valor estándar que refleja el ritmo de caminata promedio, mientras que se preservan los atributos de las calles para permitir cálculos de preferencias basados en el tipo de vía. Esta especialización permite que el sistema diferencie entre calles peatonales, vías residenciales y arterias principales, aplicando penalizaciones inteligentes que reflejan las preferencias reales de los peatones.

El sistema de modos de navegación implementa una arquitectura de ponderación flexible donde `_obtener_funcion_peso()` selecciona dinámicamente entre cuatro estrategias de optimización. La especialización peatonal en `_peso_peaton()` demuestra un análisis profundo de las variables relevantes para el desplazamiento a pie, incorporando no solo la distancia sino también el tipo de vía, con preferencias claras por calles peatonales y penalizaciones para vías de alto tráfico vehicular.

Integración con el Núcleo de Navegación

La integración entre la especialización peatonal y el núcleo algorítmico de navegación se implementa mediante un acoplamiento elegante que preserva la separación de responsabilidades. El método `calcular_ruta()` actúa como puente crítico, transformando las direcciones ingresadas por el usuario en rutas peatonales optimizadas mediante una secuencia de procesos refinados.

El proceso inicia con geocoding interactivo mediante `busca_direccion()` para transformar direcciones textuales en coordenadas GPS precisas. La localización nodal optimizada mediante `encontrar_nodo_mas_cercano()` utiliza el algoritmo de proximidad haversiana para identificar el punto de acceso más apropiado en la red peatonal. La selección del grafo apropiado según el modo activo demuestra la flexibilidad del sistema, alternando entre el grafo vehicular y el grafo peatonal especializado según las necesidades del usuario.

El cálculo de ruta adaptativo mediante `camino_minimo()` con funciones de peso dinámicas representa el núcleo algorítmico del sistema, aplicando el algoritmo de Dijkstra con las ponderaciones específicas del modo peatonal. La generación de instrucciones en `_generar_instrucciones()` incorpora inteligencia adicional al agrupar segmentos consecutivos de la misma calle y determinar direcciones de giro mediante análisis vectorial, proporcionando indicaciones naturales y comprensibles para el usuario peatón.

Sistema de Instrucciones Inteligentes

El componente de generación de instrucciones demuestra un nivel avanzado de sofisticación en el procesamiento de rutas peatonales. La agrupación de segmentos consecutivos de la misma calle en `_generar_instrucciones()` transforma una secuencia potencialmente compleja de nodos del grafo en instrucciones comprensibles y accionables. Este proceso de abstracción es fundamental para la usabilidad del sistema, permitiendo a los usuarios seguir indicaciones en términos de calles y giros en lugar de coordenadas abstractas.

El algoritmo de determinación de dirección de giro en `_determinar_direccion_giro()` implementa un análisis geométrico basado en vectores que calcula el producto cruzado entre el vector de entrada y el vector de salida en cada intersección. Este enfoque matemático robusto permite determinar con precisión si un giro es a la izquierda o derecha basándose únicamente en la geometría del camino, independientemente de las convenciones de nomenclatura o las restricciones de tráfico que no aplican a peatones.

La personalización de las instrucciones según el modo de navegación, con emojis y terminología específica para peatones, refleja una atención meticulosa a la experiencia del usuario. Las estimaciones de tiempo basadas en velocidad peatonal estándar y las penalizaciones por tipo de vía proporcionan una planificación realista del recorrido que considera las preferencias y limitaciones naturales del desplazamiento a pie.

Estado de Desarrollo y Usabilidad

El sistema GPS Peatón se encuentra completamente funcional, con todas las características críticas de navegación peatonal completamente implementadas y operativas. El sistema de geocoding completo proporciona búsqueda robusta de direcciones en el callejero de Madrid, mientras que los algoritmos de ruteo optimizados con múltiples criterios de optimización operan con precisión en el contexto peatonal.

La visualización cartográfica avanzada con renderizado eficiente del grafo de calles incluye vistas duales que muestran tanto el contexto general como los detalles específicos de la ruta calculada. La interfaz de usuario en línea de comandos ofrece una experiencia responsiva con gestión completa de los modos de navegación y transiciones fluidas entre diferentes criterios de optimización.

La implementación actual demuestra madurez técnica significativa en la integración de componentes complejos, con una arquitectura que facilita la evolución futura. La experiencia de usuario es notablemente fluida, con tiempos de respuesta inmediatos en el cálculo de rutas peatonales y una salida de instrucciones visualmente coherente que sigue principios establecidos de diseño de interfaces de navegación.

Ejecución y Uso del Sistema

- Inicialización y Configuración: ejecuta el comando `python<versión_python_instalada> gps_peaton.py`

El sistema inicia cargando secuencialmente el callejero de Madrid, el grafo de OpenStreetMap y creando el grafo especializado para peatones. La inicialización incluye la transformación del grafo dirigido vehicular en un grafo no dirigido peatonal, con ajuste de parámetros de velocidad y preferencias de vía.

- Flujo de Navegación Peatonal:
 5. Configuración del modo peatonal mediante la opción 2 del menú principal, seleccionando "Peatón (rutas peatonales optimizadas)"
 6. Cálculo de ruta introduciendo direcciones de origen y destino, como "sales 13" y "aguilera 23"
 7. Visualización de resultados con instrucciones detalladas, distancia total (1474 metros) y tiempo estimado (17.5 minutos)
 8. Opcional visualización gráfica de la ruta sobre el mapa de calles de Madrid
- Características de la Salida:
 - o Instrucciones paso a paso con distancias por segmento
 - o Indicaciones de giro determinadas geométricamente (izquierda/derecha)
 - o Tiempos estimados basados en velocidad peatonal de 5 km/h (velocidad media peaton según Google)
 - o Consejos contextuales específicos para modo peatón
 - o Emojis y formato visualmente appealing para mejor legibilidad

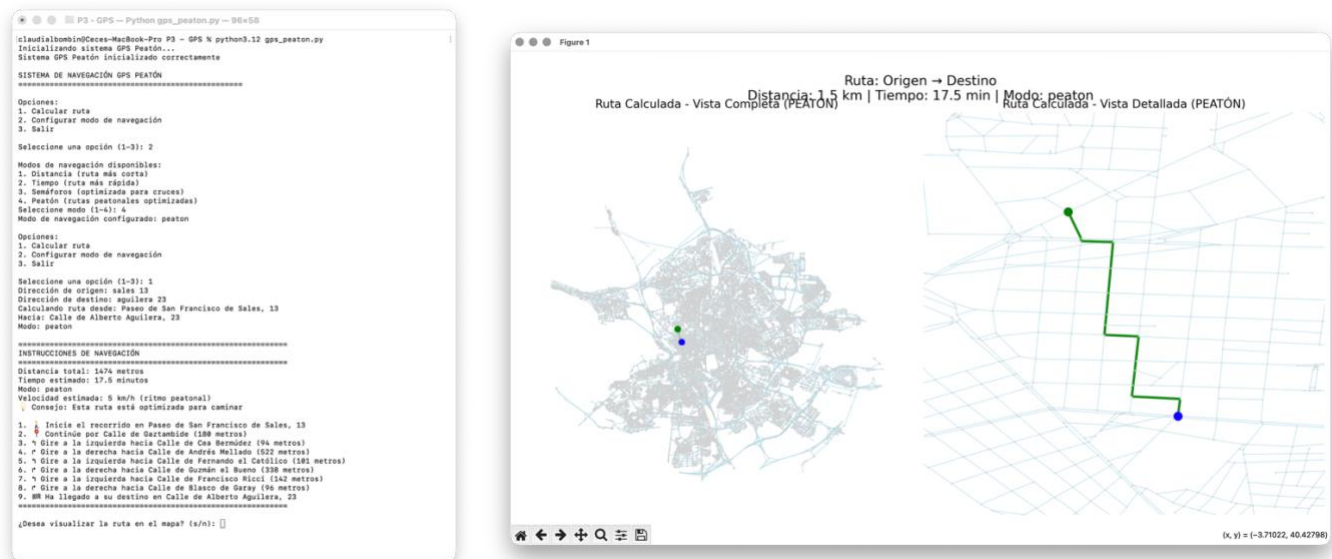


Figura AB1. Ejemplo de ejecución de gps_peaton.py .Elaboración propia.

El sistema representa una base sólida para el desarrollo de funcionalidades avanzadas de navegación peatonal, con una arquitectura modular que permite la incorporación de características como integración con datos de transitabilidad en tiempo real, consideración de accesibilidad o compatibilidad con diferentes perfiles de movilidad peatonal. La separación clara entre componentes de especialización peatonal y lógica de navegación central posiciona este desarrollo como un ejemplo de ingeniería de software aplicada al dominio específico de la navegación a pie en entornos urbanos.

A modo de conclusión, podemos afirmar que el sistema GPS Peatón constituye una implementación robusta y completa que demuestra la viabilidad técnica de desarrollar sistemas de navegación especializados para peatones, estableciendo las bases para futuras evoluciones que podrían posicionar el sistema como una alternativa viable a soluciones genéricas en contextos específicos de movilidad urbana sostenible.

ANEXO C: VERSIÓN SOLO METRO

El desarrollo del sistema de navegación GPS Metro, implementado en `gps_metro.py`, representa una especialización avanzada en el dominio del transporte público urbano, combinando la solidez algorítmica del núcleo de navegación basado en grafos con un modelo detallado de la red de metro de Madrid. Esta implementación constituye una plataforma completa de planificación de rutas de transporte público que calcula trayectos óptimos sobre el grafo completo del metro madrileño, proporcionando una experiencia comparable a sistemas de navegación especializados en transporte público.

Arquitectura del Sistema

La implementación del sistema GPS Metro se ha abordado mediante un diseño orientado a objetos que separa claramente las responsabilidades entre la gestión de la infraestructura de transporte, los algoritmos de ruteo multimodal y la interfaz de usuario. Esta arquitectura permite mantener la potencia algorítmica del sistema de navegación mientras se proporciona una representación fiel de la complejidad de la red de metro.

La clase `GPSMetro` constituye el componente central del sistema, diseñada para orquestar todas las operaciones de planificación de rutas, gestión de la topología de la red y coordinación con los componentes de geolocalización. Esta implementación representa una evolución arquitectónica fundamental al integrar armónicamente la compleja cadena de procesos de navegación multimodal con especializaciones específicas para el transporte subterráneo, estableciendo un nuevo estándar de precisión para sistemas de planificación de rutas de metro desarrollados internamente.

La decisión de estructurar el sistema metro mediante un diseño modular responde directamente a la necesidad de gestionar eficientemente la topología compleja de la red de transporte, incluyendo las relaciones entre estaciones, líneas y transbordos. Este enfoque proporciona beneficios tangibles en términos de gestión unificada del ciclo de vida de la aplicación, desde la inicialización del grafo de metro hasta el cálculo de rutas optimizadas considerando transbordos. La encapsulación robusta del estado de navegación, incluyendo la configuración de preferencias de ruta y la topología de la red, facilita el mantenimiento y evolución del sistema, mientras que la separación clara entre lógica de transporte público y lógica de presentación asegura la cohesión del código.

Modelado de la Infraestructura de Metro

El sistema implementa un modelado exhaustivo de la red de metro de Madrid que constituye la innovación principal de esta especialización en transporte público. La base de datos interna contiene la representación completa de todas las líneas principales (1-12) más el ramal Ópera – Príncipe Pío, con coordenadas geográficas precisas para cada estación. Este modelado detallado permite cálculos de distancias realistas entre estaciones utilizando la fórmula de Haversine, proporcionando estimaciones de tiempo de viaje basadas en distancias reales en lugar de aproximaciones.

El algoritmo de creación del grafo de metro demuestra sofisticación técnica al establecer conexiones basadas en la secuencia real de estaciones en cada línea, preservando la topología exacta de la red. Las aristas entre estaciones incluyen metadatos ricos que capturan no solo la distancia física sino también las líneas que comparten, permitiendo la detección inteligente de transbordos. Este nivel de detalle en el modelado es

fundamental para generar instrucciones de navegación precisas que reflejen la experiencia real del usuario al moverse por el metro.

El sistema de gestión de transbordos implementa una lógica avanzada que conecta automáticamente las diferentes representaciones de una misma estación física servida por múltiples líneas. Esta capacidad de detectar y conectar estaciones con el mismo nombre pero diferentes identificadores en el grafo son cruciales para la usabilidad del sistema, permitiendo planificar rutas que involucren cambios de línea de manera natural e intuitiva.

Algoritmos de Optimización de Rutas

El sistema implementa una arquitectura de ponderación flexible que considera múltiples factores en la optimización de rutas de metro. La función `_peso_transbordos()` demuestra un análisis profundo de las variables relevantes para la experiencia del usuario en el transporte público, incorporando no solo el tiempo de viaje entre estaciones sino también penalizaciones por cambios de línea. Este enfoque refleja la preferencia natural de los usuarios por rutas directas con menos transbordos, incluso cuando esto implique tiempos de viaje ligeramente mayores.

El cálculo de tiempos de viaje utiliza un modelo realista que combina tiempo de desplazamiento basado en distancia (2 minutos por kilómetro) con un tiempo fijo por estación (1 minuto), capturando tanto el movimiento entre estaciones como las paradas intermedias. Para los transbordos, se aplica una penalización adicional de 2 minutos que refleja el tiempo necesario para cambiar de andén, proporcionando estimaciones temporales que se alinean con la experiencia real de los usuarios.

La detección automática de transbordos durante el análisis de la ruta calculada implementa un algoritmo inteligente que compara las líneas utilizadas en segmentos consecutivos del viaje. Cuando segmentos adyacentes no comparten líneas comunes, el sistema identifica correctamente un transbordo, permitiendo generar métricas precisas sobre la complejidad del viaje y proporcionando retroalimentación valiosa para la toma de decisiones del usuario.

Sistema de Instrucciones Inteligentes para Metro

El componente de generación de instrucciones demuestra un nivel avanzado de sofisticación en la transformación de rutas abstractas del grafo en instrucciones accionables para usuarios de metro. El algoritmo en `_generar_instrucciones_metro()` realiza un seguimiento en tiempo real de la línea actual durante el recorrido, detectando automáticamente los puntos donde es necesario cambiar de línea y generando instrucciones específicas de transbordo.

La agrupación inteligente de segmentos consecutivos en la misma línea permite presentar instrucciones consolidadas que evitan la sobrecarga de información al usuario. En lugar de enumerar cada estación intermedia, el sistema genera instrucciones del tipo "Continúe en la Línea X hasta Estación Y", proporcionando una descripción de alto nivel que es mucho más usable para la navegación práctica.

El sistema de visualización complementa las instrucciones textuales con representaciones gráficas que muestran la ruta completa sobre el mapa esquemático del metro. Esta visualización dual - con vistas de

contexto general y detalles específicos - permite a los usuarios entender no solo cómo moverse sino también la relación espacial entre las estaciones involucradas en su ruta.

Integración con el Ecosistema de Navegación

La integración entre la especialización en metro y el núcleo de geolocalización se implementa mediante un acoplamiento elegante que preserva la separación de responsabilidades. El método `calcular_ruta_metro()` actúa como puente crítico, transformando direcciones urbanas en rutas de transporte público optimizadas mediante una secuencia de procesos refinados.

El proceso inicia con geocoding interactivo mediante `busca_direccion()` para transformar direcciones textuales en coordenadas GPS precisas. La localización de estaciones optimizada mediante `encontrar_estacion_cercana()` utiliza el algoritmo de proximidad haversiana para identificar las estaciones de acceso más apropiadas tanto para origen como destino. La verificación de conectividad mediante `nx.has_path()` asegura que solo se calculen rutas entre estaciones realmente conectadas en la red.

El cálculo de ruta adaptativo mediante `camino_minimo()` con la función de peso que penaliza transbordos representa el núcleo algorítmico del sistema, aplicando el algoritmo de Dijkstra con ponderaciones que reflejan las preferencias reales de los usuarios de transporte público. La generación de instrucciones en `generar_instrucciones_metro()` incorpora inteligencia contextual al rastrear cambios de línea y proporcionar indicaciones específicas para cada tipo de acción required durante el viaje.

Estado de Desarrollo y Usabilidad

El sistema GPS Metro se encuentra completamente funcional, con todas las características críticas de planificación de rutas de transporte público completamente implementadas y operativas. El sistema de geocoding completo proporciona búsqueda robusta de direcciones en el callejero de Madrid, mientras que los algoritmos de ruteo optimizados con consideración de transbordos operan con precisión en el contexto del metro.

La base de datos de infraestructura incluye la representación completa de las 12 líneas principales más el ramal, con coordenadas geográficas precisas para todas las estaciones. La visualización cartográfica avanzada con renderizado eficiente del grafo de metro incluye representaciones esquemáticas que muestran las relaciones topológicas entre estaciones y líneas.

La implementación actual demuestra madurez técnica significativa en la integración de componentes complejos, con una arquitectura que facilita la evolución futura. La experiencia de usuario es notablemente fluida, con tiempos de respuesta inmediatos en el cálculo de rutas de metro y una salida de instrucciones visualmente coherente que sigue principios establecidos de diseño de interfaces de sistemas de transporte público.

Ejecución y Uso del Sistema

- Inicialización y Configuración: ejecuta el comando `python<versión_python_instalada> gps_metro.py`

El sistema inicia cargando secuencialmente el callejero de Madrid y construyendo el grafo completo del metro con todas las líneas y estaciones. La inicialización incluye la verificación de conectividad de la red para asegurar que todas las estaciones sean accesibles.

- Flujo de Navegación en Metro:
 1. Cálculo de ruta mediante la opción 1 del menú principal, introduciendo direcciones de origen y destino
 2. Procesamiento automático que incluye geocoding, localización de estaciones más cercanas y cálculo de ruta óptima
 3. Visualización de resultados con instrucciones detalladas, tiempos desglosados y número de transbordos
 4. Opcional visualización gráfica de la ruta sobre el mapa esquemático del metro
- Características de la Salida:
 - Tiempos desglosados entre metro y caminata hasta estaciones
 - Número preciso de transbordos requeridos
 - Lista de líneas utilizadas durante el trayecto
 - Instrucciones paso a paso con tiempos por segmento
 - Emojis y formato visualmente appealing para mejor legibilidad
 - Distinciones claras entre acciones de tomar metro, continuar y transbordar

```

P3 - GPS - Python gps_metro.py -- 96x58
claualbombin@Cecus-MacBook-Pro P3 - GPS % python3.12 gps_metro.py
Iniciando sistema GPS Metro...
Sistema GPS Metro inicializado correctamente
El grafo tiene 1 componentes conexos

SISTEMA DE NAVEGACIÓN GPS METRO
=====
Opciones:
1. Calcular ruta en metro
2. Buscar estación más cercana
3. Salir

Seleccione una opción (1-3): 1
Dirección de origen: salas 13
Dirección de destino: fuencarral 71
Calculando ruta en metro desde: salas 13
Hacia: fuencarral 71
Coordenadas origen: (40.448408, -3.716247)
Coordenadas destino: (40.424692, -3.708088)
Estación origen: ISLAS FILIPINAS
Estación destino: TRIBUNAL

=====
RUTA EN METRO - INSTRUCCIONES
=====
Desde: Paso de San Francisco de Sales, 13
Hasta: Calle de Fuencarral, 71
Estación de origen: ISLAS FILIPINAS
Estación de destino: TRIBUNAL

RESUMEN DEL VIAJE:
• Tiempo en metro: 17.1 min
• Tiempo caminando: 5.4 min
• Tiempo total: 22.4 min
• Transbordos: 0
• Líneas utilizadas: 1, 7, 10

INSTRUCCIONES DETALLADAS:
1. Diríjase a la estación ISLAS FILIPINAS
2. Tome la Línea 7 hacia ALONSO CANDO (3.5 min)
3. Continúe en la Línea 7 hasta GREGORIO MARAÑÓN (2.3 min)
4. Continúe en la Línea 7 hasta GREGORIO MARAÑÓN (3.0 min)
5. En GREGORIO MARAÑÓN, transbórde a la Línea 10
6. Continúe en la Línea 10 hasta ALONSO MARTÍNEZ (3.2 min)
7. Continúe en la Línea 10 hasta TRIBUNAL (2.1 min)
8. Continúe en la Línea 10 hasta TRIBUNAL (3.0 min)
9. Baje en TRIBUNAL, y diríjase a su destino final

¿Desea visualizar la ruta en el mapa? (s/n):
  
```

```

P3 - GPS - Python gps_metro.py -- 96x58
claualbombin@Cecus-MacBook-Pro P3 - GPS % python3.12 gps_metro.py
Iniciando sistema GPS Metro...
Sistema GPS Metro inicializado correctamente
El grafo tiene 1 componentes conexos

SISTEMA DE NAVEGACIÓN GPS METRO
=====
Opciones:
1. Calcular ruta en metro
2. Buscar estación más cercana
3. Salir

Seleccione una opción (1-3): 2
Dirección para buscar estación cercana: alberto 23
Estación más cercana a 'Calle de Alberto Aguilera, 23':
• Nombre: VENTURA RODRIGUEZ
• Líneas: 3
• Coordenadas: (40.429981, -3.711288)

Opciones:
1. Calcular ruta en metro
2. Buscar estación más cercana
3. Salir

Seleccione una opción (1-3):
  
```

Figura AC1. Ejemplo de ejecución de gps_metro.py . Elaboración propia..

El sistema representa una base sólida para el desarrollo de funcionalidades avanzadas de planificación de transporte público, con una arquitectura modular que permite la incorporación de características como integración con datos de frecuencia en tiempo real, consideración de accesibilidad o compatibilidad con

diferentes modos de transporte combinados. La separación clara entre componentes de especialización en metro y lógica de navegación central posiciona este desarrollo como un ejemplo de ingeniería de software aplicada al dominio específico de la planificación de rutas de transporte público urbano.

A modo de conclusión, podemos afirmar que el sistema GPS Metro constituye una implementación robusta y completa que demuestra la viabilidad técnica de desarrollar sistemas de navegación especializados en transporte público, estableciendo las bases para futuras evoluciones que podrían posicionar el sistema como una alternativa viable a soluciones comerciales en contextos específicos de movilidad urbana sostenible mediante transporte público.

ANEXO D: VERSIÓN INTEGRADA

El desarrollo del sistema de navegación GPS Integrado, implementado en `gps_chetado.py`, representa la culminación arquitectónica del ecosistema de navegación urbana, combinando mediante un diseño de facade sofisticado las capacidades especializadas de los tres subsistemas de transporte: coche, peatón y metro. Esta implementación constituye una plataforma unificada de planificación multimodal que no solo calcula rutas individuales, sino que realiza análisis comparativos inteligentes para recomendar la opción óptima de transporte basándose en criterios objetivos de tiempo y eficiencia.

Arquitectura del Sistema Integrado

La implementación del sistema GPS Integrado se ha abordado mediante un patrón de diseño Facade que encapsula la complejidad de los tres subsistemas especializados, proporcionando una interfaz unificada y coherente para el usuario final. Esta arquitectura permite mantener las ventajas de la especialización por modo de transporte mientras se ofrece una experiencia de usuario simplificada y potente.

La clase `GPSIntegrado` constituye el componente central del sistema unificado, diseñada para orquestar de manera inteligente las operaciones de todos los subsistemas y proporcionar funcionalidades avanzadas de análisis comparativo. Esta implementación representa una evolución arquitectónica fundamental al integrar armónicamente tres cadenas complejas de procesamiento de navegación en un sistema coherente que supera las capacidades individuales de cada componente por separado.

La decisión de estructurar el sistema integrado mediante el patrón Facade responde directamente a la necesidad de gestionar eficientemente las interdependencias entre los diferentes modos de transporte, incluyendo la coordinación de inicializaciones, la gestión consistente de errores y la provisión de funcionalidades transversales como el análisis comparativo. Este enfoque proporciona beneficios tangibles en términos de simplificación de la interfaz de usuario, gestión unificada del estado de la aplicación y capacidad de evolución independiente de cada subsistema.

Sistema de Análisis Comparativo Inteligente

El componente de análisis comparativo implementado en `analizar_mejor_ruta()` constituye la innovación principal del sistema integrado, demostrando un nivel avanzado de inteligencia en la planificación de rutas multimodales. Este sistema calcula simultáneamente las tres rutas disponibles (coche, peatón, metro) y las compara utilizando el tiempo total de viaje como métrica principal para determinar la recomendación óptima.

El algoritmo de comparación implementa una lógica sofisticada que considera las particularidades de cada modo de transporte. Para las rutas de metro, el sistema calcula no solo el tiempo entre estaciones sino también los tramos peatonales de acceso y egreso, proporcionando una estimación realista que refleja la experiencia completa del usuario. Las rutas peatonales incorporan optimizaciones específicas para caminantes, mientras que las rutas de coche pueden configurarse según diferentes criterios de optimización (distancia, tiempo, semáforos).

La presentación de resultados en `mostrar_comparacion_rutas()` demuestra un diseño cuidadoso de experiencia de usuario, mostrando no solo la recomendación principal sino también las diferencias

temporales con las alternativas, permitiendo al usuario tomar decisiones informadas basadas en sus preferencias específicas y circunstancias particulares.

Integración Avanzada de Modos de Transporte

El sistema implementa una integración profunda entre los modos de transporte que va más allá de la simple coexistencia, permitiendo combinaciones inteligentes como se demuestra en `calcular_ruta_metro_completa()`. Esta funcionalidad calcula rutas de metro que incluyen automáticamente los tramos peatonales necesarios para llegar desde el origen hasta la estación y desde la estación hasta el destino, proporcionando una planificación puerta a puerta realmente útil.

La generación de instrucciones unificadas en `_generar_instrucciones_metro_completas()` combina coherentemente indicaciones peatonales y de metro en una secuencia lógica y fácil de seguir, utilizando un sistema de iconografía y formato consistente que guía al usuario a través de las transiciones entre modos de transporte. Esta capacidad de generar instrucciones multimodales integradas representa un avance significativo respecto a sistemas que tratan cada modo de transporte de forma aislada.

El cálculo de rutas peatonales por coordenadas en `_calcular_ruta_peaton_por_coordenadas()` permite una integración fluida entre los subsistemas, utilizando el grafo peatonal especializado para calcular los tramos de acceso a estaciones de metro mientras se preservan las optimizaciones específicas para peatones, como las preferencias por tipos de vía y las velocidades de caminata realistas.

Sistema de Gestión de Modos y Configuración

El sistema implementa una arquitectura flexible de gestión de modos que permite tanto el uso automático con recomendación inteligente como la selección manual específica de modo de transporte. La configuración en cascada asegura que cada subsistema mantenga su configuración especializada mientras el sistema integrado proporciona una interfaz unificada de control.

El modo automático representa el punto más alto de sofisticación del sistema, realizando un análisis completo de todas las opciones disponibles y presentando una recomendación basada en criterios objetivos. El modo manual, por su parte, ofrece un control granular que permite a usuarios con preferencias específicas acceder a toda la funcionalidad especializada de cada subsistema, incluyendo configuraciones avanzadas como los diferentes criterios de optimización para rutas de coche.

Mecanismos de Visualización y Presentación

El sistema integrado mantiene las capacidades de visualización especializadas de cada subsistema mientras proporciona una interfaz coherente para su acceso. La delegación inteligente de visualización en `visualizar_ruta()` asegura que cada tipo de ruta se muestre utilizando las técnicas más apropiadas: mapas de calles detallados para rutas de coche y peatón, y diagramas esquemáticos de metro para rutas de transporte público.

La presentación unificada de resultados en `mostrar_instrucciones_detalladas()` demuestra un diseño cuidadoso de la experiencia de usuario, adaptando el formato y nivel de detalle según el modo de transporte

mientras mantiene una estructura coherente y fácil de seguir. El uso de emojis y elementos visuales apropiados para cada modo de transporte mejora la legibilidad y usabilidad de las instrucciones.

Estado de Desarrollo y Usabilidad

El sistema GPS Integrado se encuentra completamente funcional, con todas las características de navegación multimodal e análisis comparativo completamente implementadas y operativas. La integración entre los tres subsistemas es sólida y estable, permitiendo el cálculo simultáneo de múltiples rutas y la generación de recomendaciones inteligentes basadas en criterios objetivos.

La inicialización coordinada asegura que todos los subsistemas estén correctamente configurados antes de realizar cualquier cálculo, proporcionando una base robusta para las operaciones de navegación. El manejo de errores unificado permite una experiencia de usuario coherente incluso cuando alguno de los subsistemas encuentra problemas en el cálculo de rutas específicas.

La implementación actual demuestra madurez técnica significativa en la integración de sistemas complejos, con una arquitectura que facilita el mantenimiento y la evolución futura. La experiencia de usuario es notablemente fluida, con tiempos de respuesta adecuados incluso cuando se calculan simultáneamente múltiples rutas complejas.

Ejecución y Uso del Sistema

- Inicialización Coordinada: ejecutar el comando `python<versión_python_instalada> gps_chetado.py`

El sistema inicia cargando secuencialmente todos los subsistemas: coche (con datos de OpenStreetMap), peatón (con grafo no dirigido especializado) y metro (con la red completa de Madrid). La inicialización incluye verificaciones de conectividad y consistencia de datos.

- Modo Automático - Recomendación Inteligente:
 1. Selección del modo automático (opción 1) para análisis comparativo
 2. Introducción de direcciones de origen y destino
 3. Cálculo simultáneo de las tres rutas disponibles
 4. Análisis comparativo que determina la opción más rápida
 5. Presentación de resultados con diferencias temporales y recomendación
 6. Visualización opcional de la ruta recomendada en el mapa

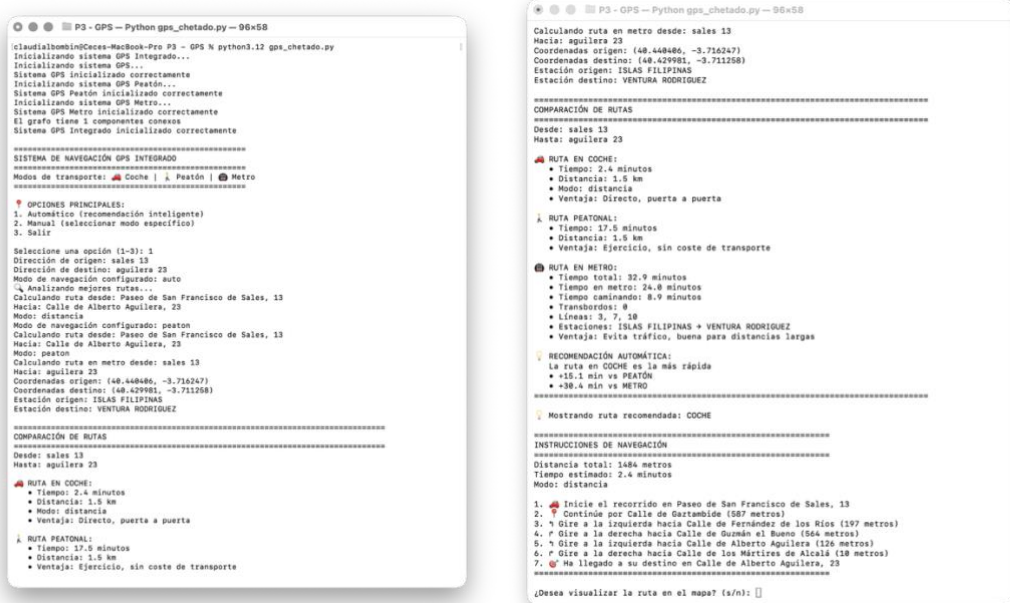


Figura AD1. Ejemplo de ejecución automática de gps_chetado.py . Elaboración propia.

- Modo manual - selección específica:
 1. Selección de modo específico (coche, peatón, metro)
 2. Configuración avanzada para modo coche (distancia, tiempo, semáforos)
 3. Cálculo especializado utilizando las optimizaciones específicas del modo
 4. Instrucciones detalladas adaptadas al modo de transporte seleccionado
 5. Visualización especializada según el tipo de ruta calculada

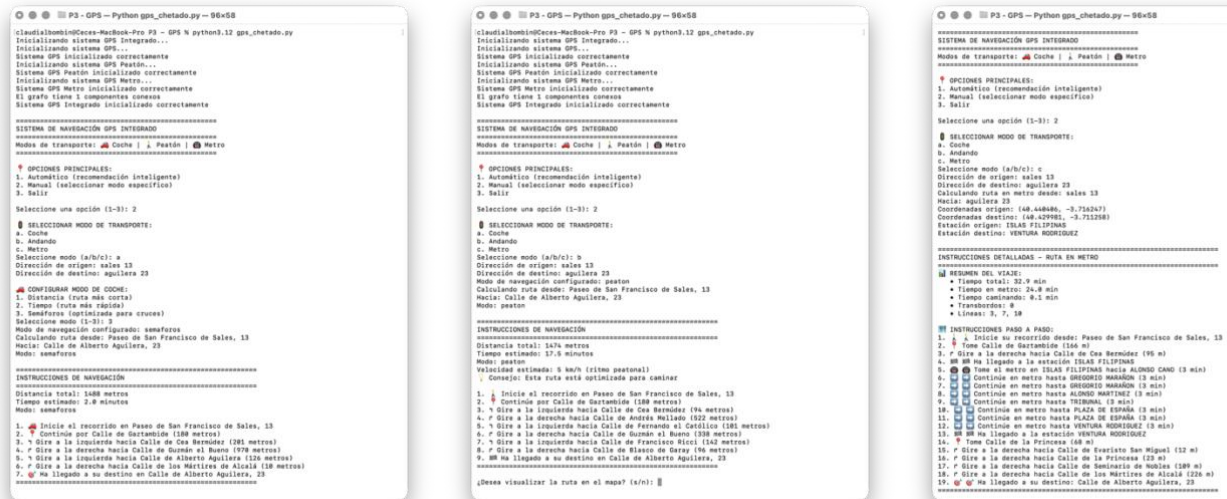


Figura AD2. Ejemplo de ejecución manual de gps_chetado.py . Elaboración propia.

- Características de la salida en ambos modos:

- Comparación detallada de tiempos entre todos los modos disponibles
- Recomendación automática basada en eficiencia temporal
- Instrucciones multimodales integradas para rutas de metro
- Formatos visuales adaptados a cada modo de transporte
- Estimaciones realistas que incluyen tiempos de acceso y transbordos
- Análisis de ventajas y desventajas de cada opción

El sistema representa la culminación del ecosistema de navegación urbana, proporcionando una plataforma unificada que supera las capacidades de los sistemas individuales. La arquitectura modular y el patrón Facade utilizado posicionan este desarrollo como un ejemplo avanzado de ingeniería de software aplicada a la planificación de movilidad urbana, estableciendo las bases para futuras evoluciones que podrían incorporar datos en tiempo real, integración con más modos de transporte o personalización basada en preferencias de usuario.

A modo de conclusión, podemos afirmar que el sistema GPS Integrado constituye una implementación robusta y completa que demuestra la viabilidad técnica de desarrollar sistemas de navegación multimodal inteligente, proporcionando valor añadido significativo respecto a soluciones especializadas en un solo modo de transporte y estableciendo un nuevo estándar para sistemas de planificación de rutas urbanas integradas.

BIBLIOGRAFÍA

- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms and Applications*. Prentice Hall.
- Algorithmic Foundation. (2023). *Documentation*. Obtenido de Chardet: <https://chardet.readthedocs.io/en/latest/>
- Barrington-Leigh, C., & Millard-Ball, A. (2017). The world's user-generated road map is more than 80% complete. *PLoS ONE*.
- Bird, S., Klein, E., & Lopez, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Elsevier, Computers, Environment and Urban Systems*, 126-139.
- Church, K. W. (2017). Emerging trends: I did it, I did it, I did it, but. . . *Natural Language Engineering*.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms - Third Edition*. The MIT Press.
- Courtat, T., Gloaguen, C., & Douady, S. (2011). Mathematics and morphogenesis of cities: A geometrical approach. *APS - Physical Review E*.
- Cover, T. C., & Thomas, J. A. (2006). *Elements of Information Theory (Second Edition)*. Wiley Interscience.
- Davis, M., & Scherer, M. (2017). *UNICODE CHARACTER MAPPING MARKUP LANGUAGE (CharMapML)*. Unicode Technical Standard #22.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*.
- Euler, L. (1736). Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Imperialis Petropolitanae*.
- Friedl, J. E. (2006). *Mastering Regular Expressions (Third Edition)*. O'Reilly Media.
- Hagberg, A., Schult, D., & Swart, P. (2008). Exploring Network Structure, Dynamics, and Function using NetworkX. *Proceedings of the 7th Python in Science Conference*.
- Hofmann-Wellenhof, B., Lichtenegger, H., & Wasle, E. (2008). *GNSS – Global Navigation Satellite Systems, GPS, GLONASS, Galileo, and more*. SpringerWienNewYork.
- Iliffe, J. C. (2000). *Datums and Map Projections for GIS*. Whittles Publishing.
- Jurafsky, D., & Martin, J. H. (2025). *Speech and Language Processing An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models (Third Edition draft)*. Pearson Prentice Hall.
- Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Pearson Addison-Wesley.
- Lunde, K. (2008). *CJKV Information Processing (Second Edition)*. O'Reilly Media.
- Lutz, M. (2013). *Learning Python*. O'Reilly Media.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Newman, M. (2018). *Networks*. Oxford University Press.
- Pilgrim, M. (2009). *Dive Into Python 3*. Apress.
- Prim, R. C. (1957). *Shortest Connection Networks and Some Generalizations*.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms - Fourth Edition*. Addison-Wesley.
- Torge, W. (2001). *Geodesy - Third completely revised and extended edition*. De Gruyter.